

National Instiute of Applied Sciences and Technology

CARTHAG UNIVERSITY

End of Studies Project

Course : **Software Engineering**

Service Level Testing Reporting Tool

Performed by

Chedli DHOUIBI

INSAT Supervisor : **Mme BAKLOUTI Fatma**
Vistaprint Supervisor : **Mr OUNIS Hatem**

Defended on : −/−/**2015**

JURY

M. President	M. X	(President)
Mme. Rapporteur	Mme. Y	(Raporteur)

Academic year : 2014/2015

Table of Contents

Table of Figures ii

Table of tables iii

General introduction 1

I Project Background 2

1 Host company . . . 3
1.1 Overview . . . 3
1.2 History . . . 3
1.3 Company organization . . . 5
1.4 Manufacturing Software Organization . . . 6
1.4.1 Organizational structure . . . 6
1.4.2 Actual teams and squads . . . 7
2 Project context . . . 8
2.1 Service Oriented Architecture . . . 8
2.2 Continuous Delivery . . . 9
2.2.1 Definition . . . 9
2.2.2 Continuous Integration . . . 9
2.2.3 Test Pyramid . . . 10
3 Problematic . . . 12
3.1 Before SOA migration . . . 12
3.2 After SOA migration . . . 12
4 Global Solution . . . 13
5 Methodology . . . 14
5.1 Agile methodologies . . . 14
5.2 Scrum . . . 14
5.2.1 Roles . . . 14
5.2.2 Artifacts : . . . 15

II Preliminary study 16

1 Study of the technical context . . . 17
1.1 Jenkins . . . 17

1.1.1 Jenkins definition . . . 17
1.1.2 Jenkins REST API . . . 17
1.1.3 Jenkins extensibility . . . 18
1.2 Stash . . . 18
1.2.1 Git definition . . . 18
1.2.2 Stash definition . . . 18
1.2.3 Stash REST API . . . 18
1.3 Nunit . . . 19
1.3.1 Nunit definition . . . 19
1.3.2 Nunit results file . . . 19
1.4 Combining technologies . . . 19
1.4.1 Jenkins and Stash . . . 19
1.4.2 Jenkins and Nunit . . . 20
1.5 Automated testing . . . 20
2 Critical review of the existing solution . . . 20
2.1 Existing solution : Nunit plugin . . . 20
2.2 Critical review . . . 22
3 Technical choices and global solution . . . 22
3.1 Data reporting systems . . . 23
3.1.1 Overview . . . 23
3.1.2 Technical choice : QlikView . . . 24
3.2 Web services . . . 26
3.2.1 Overview . . . 26
3.2.2 Technical choice : RESTful web services . . . 27
3.3 XML file parsing . . . 30
3.3.1 XML overview . . . 30
3.3.2 XML parsing methods . . . 30
3.3.3 Technical choice : DOM parsing method . . . 32
3.4 Global solution with technical choices . . . 33

III Requirement specification & analysis **35**

1 Actors . . . 36
2 Requirement analysis . . . 36
2.1 Functional requirements . . . 36
2.2 Non-functional requirements . . . 37

3 Use case diagrams . . . 38
3.1 Global use case diagram . . . 38
3.2 Detailed description . . . 40
3.2.1 Jenkins plugin module . . . 40
3.2.2 Web service module . . . 41
3.2.3 Web application module . . . 42
3.2.4 Dashboard module . . . 45
4 System sequence diagrams . . . 46
4.1 Authentication . . . 46
4.2 Add a new job to the system . . . 47
4.3 Save a Jenkins job's build data . . . 48
4.4 Perform a global search through Jenkins history for unsaved builds . . . 49
4.5 Change a Jenkins job's project, squad and team . . . 50
4.6 Change a Jenkins job's test type . . . 51

IV Solution design **52**
1 Global design . . . 53
1.1 Global architecture . . . 53
1.2 Package model . . . 55
1.3 Design patterns . . . 56
2 Domain design . . . 58
2.1 Jenkins plugin class model . . . 58
2.2 Web service class model . . . 59
2.3 XML file parsing class model . . . 60
2.4 Data persistence class model . . . 61
2.5 Web application class model . . . 62
2.5.1 Model class model . . . 63
2.5.2 Controllers class model . . . 64
2.5.3 DAO class model . . . 65
2.5.4 Utils class model . . . 66
3 Behavioral design . . . 67
3.1 Object sequence diagram : Add a new job . . . 68
3.2 Object sequence diagram : Authenticate . . . 69
3.3 Object sequence diagram : Global search through Jenkins . . . 70
3.4 Object sequence diagram : Save job's build data . . . 71

3.5 Object sequence diagram : Change project, squad and team 72
3.6 Object sequence diagram : Change test type 73
4 Database design 73

V Achievements **76**
1 Software environment 77
1.1 Design tool : Enterprise Architect (EA) 77
1.2 Implementation tools 77
1.2.1 Netbeans IDE 77
1.2.2 Eclipse IDE with Maven plugin 78
1.2.3 QlikView 78
1.2.4 SQL Server Management Studio 78
1.3 Continuous integration tools 79
1.3.1 Jenkins 79
1.3.2 Stash 79
1.3.3 Apache ant 79
1.3.4 Junit 79
1.4 Technologies and Frameworks 80
1.4.1 J2EE 80
1.4.2 JAVAX-WS 80
1.4.3 Twitter bootstrap 80
1.4.4 JQuery 80
1.4.5 AJAX 80
2 Technical architecture 81
3 Implementation achievements 82
3.1 Dashboard achievement 82
3.1.1 Velocity KPI 82
3.1.2 General dashboard view 82
3.1.3 Dashboard filters 84
3.1.4 Dashboard primary graphs 84
3.1.5 Dashboard secondary graphs 86
3.2 Web service achievement 88
3.2.1 Deploying the RESTful web service 88
3.2.2 Invoking the RESTful web service 88
3.3 Web application achievement 89

3.3.1 Authentication view 89
3.3.2 Main view 89
3.3.3 Add new job view 92
4 Continuous integration 93
4.1 Jenkins job 93
4.2 Stash repository 94
4.3 Ant script 95
4.4 Unit tests 95
5 Project documentation 96

General conclusion and perspectives **97**

Bibliography **98**

Table of Figures

I.1 MSW structure . . . 7
I.2 Service Oriented Architecture . . . 8
I.3 Continuous Integration cycle . . . 9
I.4 Test Pyramid . . . 10
I.5 Global Solution . . . 13
I.6 Burn-down chart example . . . 15

II.1 Jenkins dashboard . . . 21
II.2 Nunit plugin . . . 21
II.3 Reporting chain . . . 24
II.4 Magic quadrant for reporting platforms . . . 25
II.5 Hierarchical parsing method . . . 31
II.6 Event-based parsing method . . . 32
II.7 Global solution after technical choices . . . 34

III.1 Global use case diagram . . . 39
III.2 Jenkins plugin use case diagram . . . 40
III.3 Web service use case diagram . . . 41
III.4 Web application use case diagram . . . 42
III.5 Dashboard use case diagram . . . 45
III.6 Authenticate sequence . . . 46
III.7 Add new job sequence . . . 47
III.8 Save job build data sequence . . . 48
III.9 Global search through Jenkins sequence . . . 49
III.10 Change project squad and team sequence . . . 50
III.11 Change test type sequence . . . 51

IV.1 Global architecture . . . 53
IV.2 Jenkins layer . . . 54
IV.3 Back end layer . . . 54
IV.4 Front end layer . . . 54
IV.5 Global packaging model . . . 55
IV.6 Design patterns . . . 57

IV.7 Jenkins plugin class model . . . 58
IV.8 RESTful web service class model . . . 59
IV.9 XML file parser class model . . . 60
IV.10 Data persistence class model . . . 61
IV.11 Web application package model . . . 62
IV.12 Model class model . . . 63
IV.13 Controllers class model . . . 64
IV.14 DAO class model . . . 65
IV.15 Utils class model . . . 66
IV.16 Add new job object sequence diagram . . . 68
IV.17 Authenticate object sequence diagram . . . 69
IV.18 Global search through Jenkins object sequence diagram . . . 70
IV.19 Save build data object sequence diagram . . . 71
IV.20 Change project, squad and team object sequence diagram . . . 72
IV.21 Change test type object sequence diagram . . . 73
IV.22 Database design . . . 74

V.1 Netbeans IDE logo . . . 77
V.2 Eclipse IDE logo . . . 78
V.3 QlikView logo . . . 78
V.4 Sql Server Management Studio logo . . . 78
V.5 Jenkins logo . . . 79
V.6 Stash logo . . . 79
V.7 Apache Ant logo . . . 79
V.8 Junit logo . . . 79
V.9 Technical architecture . . . 81
V.10 Global velocity dashboard . . . 83
V.11 Velocity dashboard filters . . . 84
V.12 Velocity KPI graph . . . 85
V.13 Velocity per service KPI graph . . . 85
V.14 Velocity per test type KPI graph . . . 85
V.15 Tests per build graph . . . 86
V.16 Total builds per month graph . . . 86
V.17 Total tests per month graph . . . 87
V.18 Velocity details table . . . 87

V.19 The deployed web service : GET method 88
V.20 Invoking the RESTful web service with PowerShell 88
V.21 Authentication view 89
V.22 Main page 90
V.23 Main page buttons 90
V.24 Console 91
V.25 Performing global search 91
V.26 Fetching a job's information 92
V.27 Information fetching results 92
V.28 Jenkins job name 93
V.29 Jenkins job configuration to include Stash 93
V.30 Invoking Ant script from Jenkins job 94
V.31 Stash repository 94
V.32 Utils class model 95
V.33 SLT reporting page 96

Table of tables

I.1 MSW teams and squads 7

II.1 REST HTTP methods 28
II.2 Difference between XML parsing methods 32

III.1 Save build data of a given job from within Jenkins 40
III.2 Save build data of a given job with external references 41
III.3 Add a new job to the database 42
III.4 Change test type 43
III.5 Change project, squad, and team 43
III.6 Trigger a global search through Jenkins history links for unsaved builds 44
III.7 Authenticate 44
III.8 Consult dashboard 45

General introduction

Vistaprint is an international company that mainly sells printable products such as business cards, cups and t-shirts. It offers an online platform for customers to manually configure their printable products. This is what probably explains the big interest Vistaprint is showing towards software engineering and technology in general. In fact, multiple IT teams are deployed all over the globe so as to ensure the platform's maintenance and upgrade. Consequently, Vistaprint tends to be very strict when it comes to being technologically up-to-date in terms of software engineering best practices. The main reason behind all of this is to simply stay on top of the very competitive market.

One of the software engineering best practices is a concept called **Continuous Delivery**, where the software project is continuously tested and delivered as it is being developed. This practice includes a massive amount of software testing, which led vistaprint to leaning towards test automation platforms, which basically are special software used to control the execution of tests and the comparison of actual outcomes with predicted outcomes.
The main issue with the test automation platform Vistaprint is currently using, is the lack of a solid reporting system. In fact, the generated data after each test execution is very rich in useful information, yet it has always been disposed of. This is what created the need of a reporting solution.

The proposed solution aims to fill the reporting gap by perfectly integrating the automation platform and at the same time ensuring the loose coupling between the two.

In this report, we will first present the project's background, explaining in depth the project's context, the root causes of the raised issues and the global solution.
We will then present the preliminary study, where we specify the technical context, the involved technologies, as well as a thorough critical review of the existing solution.
Later on, we will present the requirement specification and analysis, where we will include the system actors, the use case diagrams and the system sequence diagrams.
After that, we will define the project's design, going from packaging diagrams to object sequence diagrams, as well as presenting the global architecture.
In the final chapter, we will illustrate the realized achievements during the project's development, the implementation tools and involved technologies, and finally the project's continuous integration.

Chapter I

Project Background

Plan

1 Host company 3

1.1 Overview 3

1.2 History 3

1.3 Company organization 5

1.4 Manufacturing Software Organization 6

2 Project context 8

2.1 Service Oriented Architecture 8

2.2 Continuous Delivery 9

3 Problematic 12

3.1 Before SOA migration 12

3.2 After SOA migration 12

4 Global Solution 13

5 Methodology 14

5.1 Agile methodologies 14

5.2 Scrum 14

Introduction

In this chapter we will be presenting the project's background. We will start with a general presentation of the host company, a presentation of the team in which this project has been requested (the client), then we move on to the project's context, the raised issues and the global requirement. We will then finish by presenting the technical background and the development methodology.

1 Host company

In this section we will present an overview of the company, a briefing of its history and a description of its organizational units.

1.1 Overview

Vistaprint is a multinational company and it belongs to Cimpress group. It is located in 15 different countries including Tunisia. It provides printed and promotional customizable materials and marketing services to small businesses and consumers.

From a simple paper provider, Vistaprint has become a leader in the e-commerce field. It exposes its items and services through the website **Vistaprint.com**. Its main products are Business Cards, Postcards, Websites, T-shirts, Hats, Pens, Sticky Notes, Window Decals, and Car Door Magnets.

Beyond its printing business activities, Vistaprint is a technology-focused company. The fact that its e-commerce platform operates in a large scale and provides mass customization of products, leads Vistaprint's technology managers to think a lot of complexity in resolving problems optimally. That's why, Vistaprint highly depends on computer science and software engineering skills [1].

1.2 History

Created in 1994 by Robert Keane in Paris, Vistaprint began with providing small businesses with graphic design and desktop printing supplies via direct mailed catalogues.

By 1998, the company had begun to grow up into an internet e-commerce company by starting to create its own e-commerce platform. It was the year of the first expansion beyond France. The company reached the British and German Markets via the acquisition of "PaperDirect Europe" which, like Vistaprint at the time, was a specialty catalogue of desktop publishing papers.

Since 1999, it stated to deliver its products via the internet while focusing on its starting business, and the first US office was opened in Framingham, Massachusetts. 2000 was the worst year for the company, Vistaprint lost its venture capital financing offer from Geocapital Partners after that investor reads a Wall Street Journal article questioning the viability of Internet based business models. Moreover, and in order to survive the burst of the "dot-com crash", the company was forced to sell off some pieces of the business and to reduce the workforce by 50 (from 70 to 20 employees). Vistaprint survived this crisis and emerged as a profitable company in 2002.

Three years later, Vistaprint was floated on the NASDAQ stock exchange. Since that time, the company's revenue had grown up while continuing the expansion towards global markets in Europe, Japan and Australia, and today, its revenue exceeds the one billion USD declaring it as a large company. In addition to that, Vistaprint has enlarged its business to surpass the business card printing, offering as a consequence hundreds of options and marketing solutions for small businesses : including dozens of printed products, apparel, signage, email marketing, websites and internet marketing.

Vistaprint has served over 15 million customers in 120 countries, and received an average of more than 18,000 orders per day. And during the last years, the company has been adding about 200,000 new customers to its client base each month and counts actually 4.100 employees.

In the beginning of 2014 the revenue of Vistaprint was 370.8 million dollars and announced an agreement to acquire "People & Print" Group, a leading German printing company specialized in "upload and print".

After the several acquisitions of many companies like the leading Norwegian consumer photo product company "FotoKnudsen", the leading web-to-print company Pixartprinting ; Vistaprint changed its name to Cimpress in November 2014 [2].

1.3 Company organization

Vistaprint is divided into a multiplicity of independent sub-organizations, each distinguished by their functions and responsibilities. In this section we will individually describe each of these organizational units.

1. **Capabilities :**
   It's the organization that delivers capabilities to the company. In fact, we mean by the word capability, the combination of people, process and technology to produce a successful new project or initiative. This organization includes 5 areas :

   - Capability Development (CapDev) : the software development organization.
   - Capability Operations (CapOps) : the group that builds technology infrastructure and manages the world-wide technology operations.
   - Capability Support : the team that builds technology infrastructure, manages technology operations, and delivers internal tools.
   - Capabilities Planning (CapPlan) : this team facilitates planning within Capabilities and across the company's partners.
   - Fulfill Demand (FD) : this team is devoted to fulfillment software.

2. **Manufacturing + Supply Chain (M+S) :**
   It's responsible and accountable for the development, implementation and results of the company's manufacturing strategy.

3. **CEO :**
   This organization includes the Executive officers of the company and it is composed of members of our management team.

4. **Finance :**
   Finance is the pillar in charge of all financial transactions and budgeting for the company.

5. **Human Resources :**
   They ensure that "Vistaprint" finds, hires, rewards, develops, and retains great team members. They do this by executing on a global HR vision with audacious goals and a strategic roadmap.

6. **Legal :**
   Legal is the team responsible of enabling and supporting the achievement of Vistaprint's ongoing business objectives and corporate responsibilities by providing the highest quality, creative legal services to the organization in a productive, solution-oriented, and cost-effective manner.

7. **Design Sales and Service :**
   It comprises the process of serving Vistaprint customers across the globe by listening and advocating on their behalf, as well as delivering services they value.

8. **Marketing :**
   The one responsible of communicating the value of a product or service to the customer. Their main mission is to provide the plans, tools and tips each one on his specific field to take the business to the next level [3].

1.4 Manufacturing Software Organization

Manufacturing Software (MSW) is the organization within Vistaprint devoted to fulfillment software : manufacturing, shipping and logistics, outbound material handling,etc.

Our project comes within this organizational unit. In other words, they play the client role throughout the project's development. This is the reason behind the presentation of their structure and their actual teams and squads [4].

1.4.1 Organizational structure

MSW's organizational structure has three containment levels : teams, squads and projects. The structure is described below :

- MSW organization is split into different teams.
- A team is a collection of people that work in related areas, generally consisting of one or more squads.
- A squad is the basic development building block. It is similar to a Scrum team. It has the skills and tools needed to design, develop, test and release to production. It is self-organizing, and decides its own way of working. Each squad works on several projects.
- A project is a software project that is under development [4].

The following figure shows MSW's organization structure.

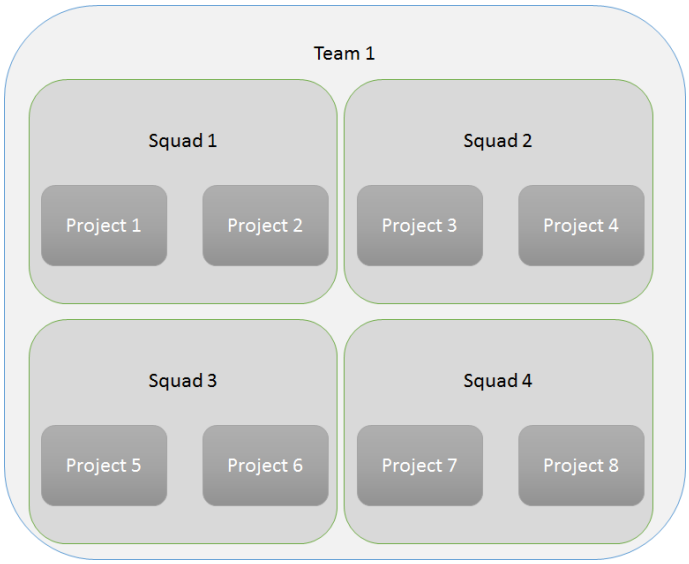


Figure I.1 – MSW structure

1.4.2 Actual teams and squads

The following database table shows the actual teams and squads of the MSW organization.

Table I.1 – MSW teams and squads

Team	Key	Squad
MSW-WIDE	MSWQE	Manufacturing Software QE
	VIPAUT	Viper Automation
	MSWQR	Manufacturing Software QE Release
	MSWUX	Manufacturing Software User Experience
	MSW	MSW infrastructure
	KISSETUP	MSW KIS setup
Production	ORDERS	MSW Orders
	PBT	Binning
	MSWINST	Manufacturing Software Instrumentation
	MSWQ	Manufacturing Software Shopfloor Quality
	MSWPP	Manufacturing Software Production Planning
Logistics	QP	Quoting and Planning
	GLOB	Global Customs
	MSWLC	Manufacturing Software Logistics Carrier
	MSWSC	Manufacturing Software Logistics Supply Chain
PMI	MIB	Manufacturing Software IVB
Manufacturing Integration	PMI	Production Manufacturing Integration

2 Project context

Vistaprint never misses a chance to renovate and be up-to-date technology-wise.
On one hand, this can be particularly noticed when it comes to making software. It insists on applying most of the software engineering best practices, so as to help maintain the best quality throughout the software's life-cycle. In fact, my project stands within one of those principles : Continuous Delivery.

On the other hand, Vistaprint is also trying to keep up the technological pace when it comes to the company-wide organization and technical architecture. That is the exact reason behind the migration towards a Service Oriented Architecture (SOA). This migration happens to be one of the root causes of the raised issues, as it will be explained later on in this section.

2.1 Service Oriented Architecture

Service oriented architecture (SOA) is an approach used to create an architecture based upon the use of services, where each service represents a specific function, and where multiple services can be bound together to form a bigger composite service.

One of the keys to SOA's success is the loose coupling of services. The simple fact that each service operates independently hugely encourages service reuse, which in return makes it unnecessary to start from scratch should an upgrade occur.

In addition to that, with the wide use of web services, SOA has proved to be one of the best ways of bringing architectural agility to one's organization, as well as saving both time and money [5].

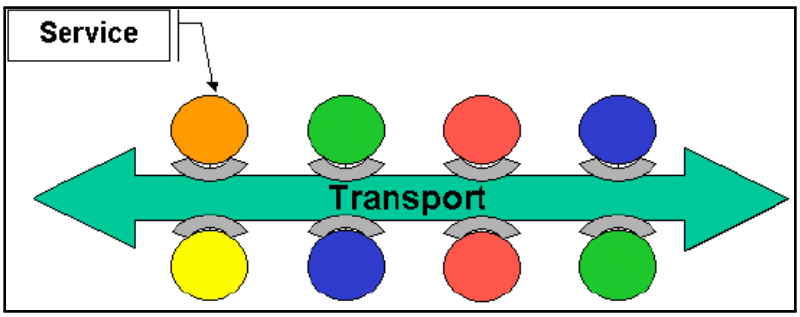


Figure I.2 – Service Oriented Architecture

2.2 Continuous Delivery

As we mentioned earlier, our project falls in the Continuous Delivery concept. In this section we will define the concept, as well as its main components.

2.2.1 Definition

Continuous Delivery (CD) is a pattern of software delivery where the system is always in a state of being ready to be deployed to production. It maximizes the delivery of new business value by decoupling changes from each other and delivering them to production as soon as they are ready.
Continuous Delivery is characterized by the use of Continuous Integration (CI) and a high degree of automation. Automation includes automated software building and automated software testing [6].

2.2.2 Continuous Integration

As mentioned above, Continuous Integration is the first step in the Continuous Delivery pipeline. It is primarily based on a principle suggesting that delaying integration of changes only delays the detection of defects, thereby making them more expensive than necessary [7].

More generally, Martin Fowler defined Continuous Integration as a software development practice where members of a team integrate their work frequently. Each of these integrations is verified by automated builds to detect integration errors as quickly as possible [8].

We also have to mention that the best way to detect software defects in a faster and more efficient way is to include automated tests in the building process. The well-known Unit tests are surely included, but they're not the only type there is. In fact, Vistaprint follows a testing model called the **Test Pyramid**.

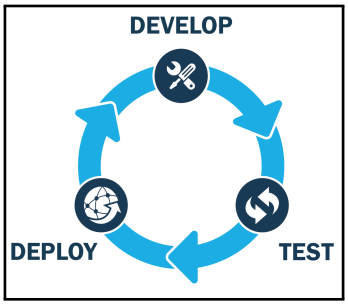


Figure I.3 – Continuous Integration cycle

2.2.3 Test Pyramid

Vistaprint uses the so-called **Test Pyramid** to guide the different squads in the trade-offs they make with regards to test automation at different levels of integration [9].

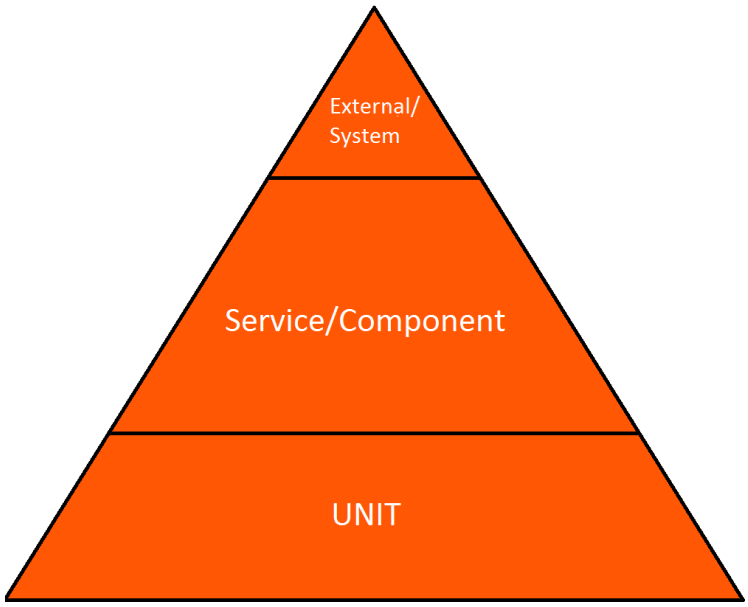


Figure I.4 – Test Pyramid

The Test Pyramid shows three different aspects of testing available when developing and releasing software. It also provides a graphical representation of trading off costs and risks/benefits associated with doing more or less of a certain type of testing.

The first aspect is **quantity of testing**, implied by the horizontal axis. That is, tests belonging to the base of the pyramid are expected to be more numerous than tests shown at the top.

The second aspect is **level of integration**, shown on the vertical axis. Generally, low levels of integration focus on testing of small, isolated units, while high levels of integration focus on testing complex systems with many internal interfaces. Simply put, the bottom two tiers are internal in a single team, while the top layer is external, as it includes system-wide, end-to-end testing.

The third aspect is **test category** or **test objective**, and this is only implied in the diagram. Specifically, each integration tier in the pyramid will generally contain tests devised to satisfy differing objectives. Some of the possible categories are Smoke Testing, Regression testing, Performance testing, Security testing, Build verification testing, and many others [10].

What we need to retain here is that Vistaprint uses three types of tests : Unit tests, Service level tests, and system tests, also referred to as UI (User Interface) tests.

- **UI tests :**
  UI tests are tests that are run at the User Interface level. It is basically an end-to-end test that checks the system as a whole.
  As a general rule, we should always do as few UI tests as possible. As a matter of fact, they prove to be expensive to write and time consuming (they take a long time to run). However, they remain quite crucial as the other test types cannot cover all the application testing needs [11].

- **Service tests :**
  Service level testing is mainly used in service-oriented architectures, but is not restricted to that. In fact, all applications are essentially made up of various services. From this point of view, a service is a procedure the application does in response to a set of inputs. The main objective behind this type of tests is testing the services of an application separately from its user interface [12].

- **Unit tests :**
  A unit test is an automated piece of code that invokes a unit of work in the system and then checks a single assumption about the behavior of that unit of work. A unit of work is a single logical functional use case in the system that can be invoked, in most cases, by some public interface.
  The main objective behind this type of tests is to isolate each part of the program and determine whether each individual part is correctly working [13].

3 Problematic

Having detailed the project's global context, we can now move on the raised issues.
The main raised issue is the absence of a test reporting tool. The root cause behind this issue is the SOA migration. In order to better understand the relationship, we have to go back to the pre-migration period.

3.1 Before SOA migration

As part of the Continuous Delivery cycle, developers were automating tests through a powerful automation tool called UAP (Unified Automation Platform). In fact, not only does UAP automate tests, but it also performs all of the data management functions including a complete reporting platform, with the help of the Application Lifecycle Manager (ALM).

In fact, UAP is tightly coupled with the test management system ALM. After executing tests, UAP delegates the test result storage to ALM, which in return feeds that data to several deployed dashboards.

At this point there were no reporting issues. However, there was a major drawback with this seemingly powerful combination : it is essentially focused on automating UI (User Interface) tests. When it comes to other test types, the performance difference is significant.
At the time, though, it was not a real problem. It was rather an advantage, since the majority of the automated tests were UI tests.

3.2 After SOA migration

As a natural consequence of the migration, the number of services increased exponentially, along with the service level tests. Being UI-testing oriented, the UAP/ALM combination was no longer a good choice. It was soon replaced by another couple : Jenkins and Nunit. The newly adopted combination was a good boost in terms of performance. However, the reporting part took a major hit.
In fact, Jenkins provides very poor reporting, that is neither flexible nor complete. There is no global view and no long term archiving. This, for a team that's used to the use of multiple dashboards, is a major problem.

4 Global Solution

In order to overcome this problem, there is an obvious need of a reporting tool that perfectly integrates with the technical context.

In a very general way, this tool will have to interact with Jenkins to retrieve testing data, save that data in a database for long term usage, and provide a dashboard and several reports for the final users. Also, the solution has to fit in the adopted Service Oriented Architecture. So it has to be, or at least a part of it has to be, a service that is accessible through the network. Moreover, it also has to provide a Jenkins plugin that might be useful when security forbids network access.

The figure below shows a global view of the requested solution.

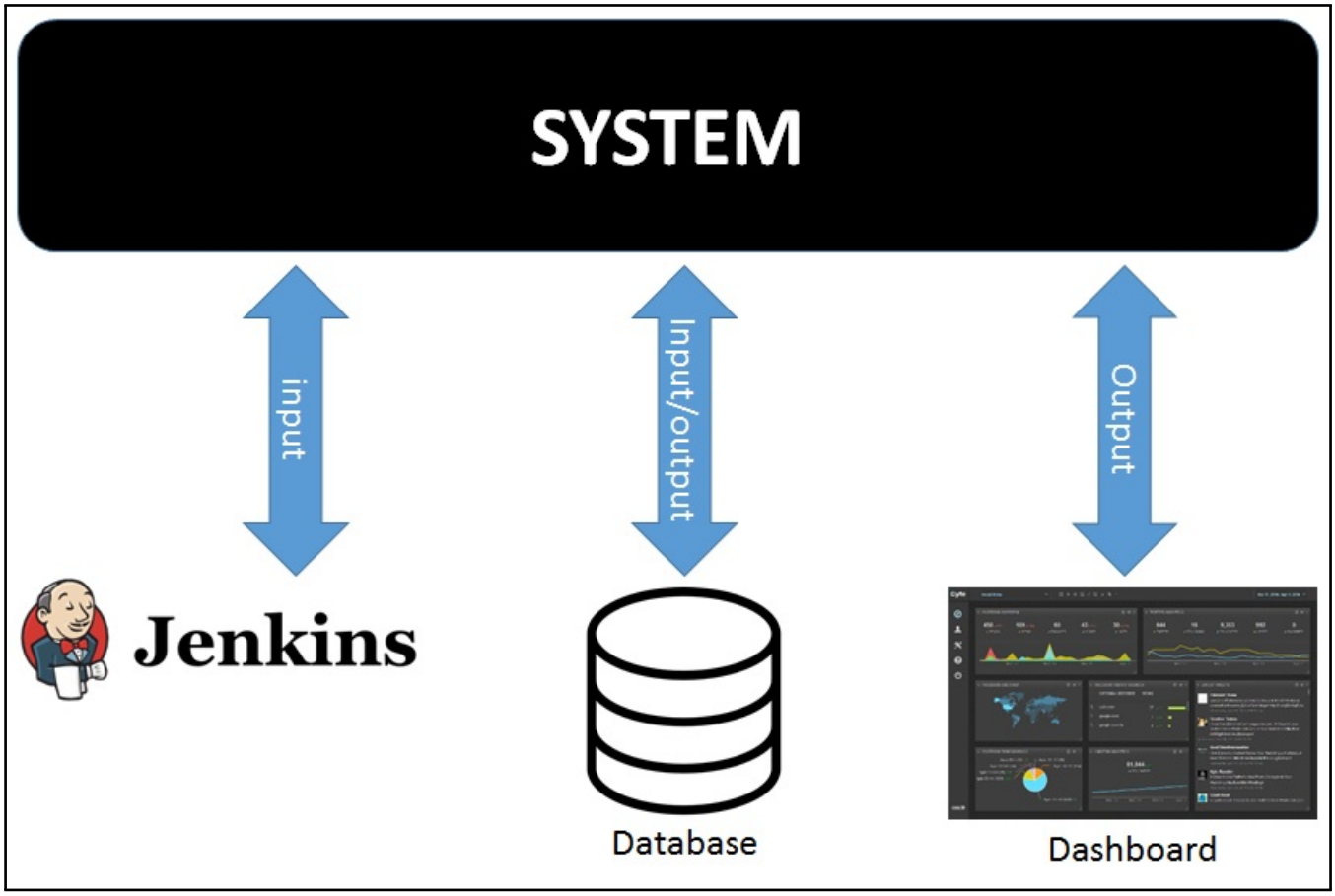


Figure I.5 – Global Solution

5 Methodology

To reduce complexity of problems and meeting business requirements in shorter time and in cost-effective way, software development requires to follow a working methodology that guarantees control throughout the development cycle and provides a clear visibility into projects. At Vistaprint, especially within MSW teams, we opt for agile methodologies and more specifically the Scrum methodology.

5.1 Agile methodologies

The agile methods are a sort of software development process. They are characterized by their iterative cycle and depending on changing customer requirements, therefore there is a frequent feedback and interaction between the development team and the stakeholders. These methods, due to their iterative approach, they minimize risks, give businesses maximum return on their investment and lead to user satisfaction [14].

5.2 Scrum

Scrum is an iterative approach to software development tightly aligned with agile principles and the Agile Manifesto. Scrum is made up of a series of time blocks called sprints, which focus on delivering working software. A sprint is typically two to four weeks in length and is defined by a goal or theme that helps to clarify the objective of the sprint. Sprints are isolated from change, allowing the team to focus on delivering working software without distraction. Scrum focuses on helping people, who are committed to develop a project, to deliver that project in best circumstances [15].

5.2.1 Roles

The major roles in Scrum are :

- **Product Owner :**
  They are the customer ambassador so they are the only person who sets and prioritizes requirements for each sprint. The product owner is always looking after the customer's best interest and their satisfaction.
- **Scrum Master :**
  Their main responsibility is to ensure that Scrum rules and practices are understood and followed by all members of the Scrum team. They resolve issues between members and any other problems that may block the course of a sprint, by setting up daily meetings.

The scrum master is always looking for the team's interest in order to maximize their productivity.

- **Development Team :**
  This team includes programmers, testers, front-end designers and any other members belonging to other disciplines that are related to the project [15].

5.2.2 Artifacts :

The main artifacts of Scrum methodology are :

- **Product backlog :**
  It consists of a list of all remaining work that the development team still need to do. The product backlog is managed by the product owner and is constantly prioritized.
- **Sprint backlog :**
  The sprint backlog is a part of the product backlog, so it lists all remaining work for a sprint.
- **Burn-down chart :**
  It is a visual way to track the product development effort remaining in a sprint. Generally, the visual of a burn-down chart is a plan where the X axis represents the remaining effort required to complete the sprint and the Y axis represents the number of days till the sprint deadline. The figure below shows an example of a burn-down chart [15].

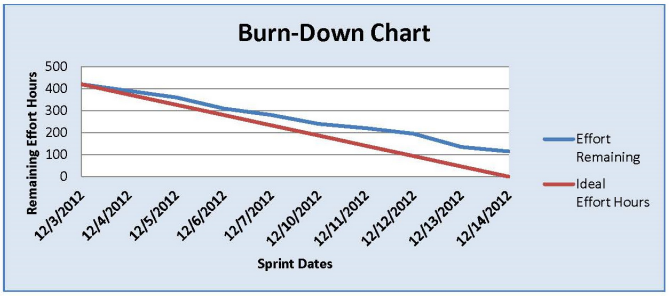


Figure I.6 – Burn-down chart example

Conclusion

Throughout this chapter we have had a global view over the project's background, including the raised issues and the global solution. In the next chapter we will be presenting the preliminary study, where we discuss the technical context in more depth, as well as present the different technical choices to make.

Chapter II

Preliminary study

Plan

1 Study of the technical context 17

1.1 Jenkins 17

1.2 Stash 18

1.3 Nunit 19

1.4 Combining technologies 19

1.5 Automated testing 20

2 Critical review of the existing solution 20

2.1 Existing solution : Nunit plugin 20

2.2 Critical review 22

3 Technical choices and global solution 22

3.1 Data reporting systems 23

3.2 Web services 26

3.3 XML file parsing 30

3.4 Global solution with technical choices 33

Introduction

In order to design and integrate a new module, it is crucial to understand the current work environment.
In this chapter we will be presenting the project's technical context, a critical review of the current solution as well as a detailed study about the potential solution's technologies.

1 Study of the technical context

After the SOA migration, Vistaprint has adopted many technologies like Jenkins for script automation, Stash as Git repository and Nunit as testing tool. The whole pack is integrated together to form a solid automation platform. In the following section, we will present each of these technologies, as well as describing their interactions.

1.1 Jenkins

In this section we will define Jenkins and its REST API.

1.1.1 Jenkins definition

By definition, Jenkins is an application that monitors the execution of repeated jobs, such as building and testing a software project. It provides a full continuous integration system, making it easier for developers to integrate changes to the project, and making it easier for users to obtain a fresh build [16].
As mentioned before, Vistaprint uses Jenkins for job automation. That is, build automation, test automation and deployment automation.

1.1.2 Jenkins REST API

Jenkins provides a remote access API to its functionalities. It provides consumable data in one of three possible formats : XML, JSON or Python. The remote access API is offered in REST style. That is, there is no single entry point for all features. In fact, all the features are available through REST GET requests. Through REST we can retrieve information for programmatic consumption, trigger new builds or even create new jobs.
The data model that Jenkins maintains internally can be thought of as a big tree. When we make a remote API call, we will get a subtree of it. The subtree is rooted at the object for which we made the remote call [17].

1.1.3 Jenkins extensibility

One of Jenkins' main strengths is its extensibility. In fact, Jenkins makes it extremely easy for users to develop their own plugins that better suit their needs.
Jenkins defines extension points, which are interfaces or abstract classes that model an aspect of a build system. Those interfaces define contracts of what needs to be implemented [18].

1.2 Stash

Before we start defining Stash, we have to first define Git, along with the version control system.

1.2.1 Git definition

Git is a distributed version control system that was first conceived for Linux Kernel development. Its emphasis on speed, data integrity and support for distributed non-linear workflows have made it one of the most commonly adopted system [19]. Version control systems are tools that manage multiple versions of files and programs. They allow file locking (a file can only be edited by one person at a time) and file change tracking.
These tools require that all files are saved in a central location, which is referred to as repository. This repository, which is usually provided by the version control system, contains all previous and current versions of files. Once a file is created or updated, the changes are committed to the repository. Thus the latest file versions are always available to all users [20].

1.2.2 Stash definition

Stash is a collaborative Git solution that's secure, fast and enterprise grade. One of the things that make it a good choice is the massive scaling ability. In fact, Stash utilizes a clustering technique, which provides high availability and high performance. It also provides administrators full control over how it fits into their environment [21].
Vistaprint uses Stash as their Git solution. It contains all the code files and software pieces that have been developed. It is directly linked to the continuous delivery cycle, and more importantly to the continuous integration step.

1.2.3 Stash REST API

Stash offers a REST API that provides a standard interface for interacting with it. The API provides access to Stash via URI paths. To retrieve information, we only have to make HTTP requests (mainly GET requests), and then parse the response (JSON format) [22].

1.3 Nunit

In this section we will define the Nunit framework and globally explain the test results.

1.3.1 Nunit definition

Nunit is an open source framework designed for writing and running tests in Microsoft .Net programming languages.
Nunit provides a console runner (nunit-console.exe), which is used for batch execution of tests. The console runner works through the Nunit Test Engine, which provides it with the ability to load, explore and execute tests. When tests are to be run in a separate process, the engine makes use of the Nunit-agent program to run them [23].

As result of the test execution, Nunit generates an XML result file. This XML file represents a huge part of our project (data source), which explains the detailed analysis in the following section.

1.3.2 Nunit results file

On the whole, the XML result file will have information about the work environment, the test's global statistics, as well as detailed test suites and test cases (results, duration, exceptions, category, etc). A test suite is a composition of several tests. It contains detailed instructions for each collection of test cases, along with information on the system configuration during testing. A test case is set of conditions characterized by a known input and by an expected output, which are configured before the test case is executed. Usually, the input helps test a precondition, while the expected output helps test a post-condition [23].

1.4 Combining technologies

In this section we will present how these technologies interact with one another.

1.4.1 Jenkins and Stash

Jenkins is used to execute a Jenkins job that builds, tests then deploys a software project. Stash is used as a Git repository where the source files reside.
This being said, the relationship that ties Jenkins to Stash becomes very obvious. That is, before building the source code, Jenkins extracts the files from the Stash repository.
The repository's URL can be found in the Jenkins job configurations file (accessible through REST API).

1.4.2 Jenkins and Nunit

As part of the automated process done by Jenkins, we find the automated tests, which basically are Nunit tests. After each automated test execution, Jenkins generates an XML file that contains the detailed Nunit test results.
The XML result file can be found in the Jenkins job's workspace, and is also accessible through REST API.

1.5 Automated testing

At this point, we clearly understand the roles of the automation actors, and the relationship that ties them. What we have to retain from the whole automation process is the part that matters most to this project : automated testing.

Automated software testing is a process in which software tools execute scripted tests on a software application before it is released into production. The objective of automated testing is to simplify as much of the testing effort as possible with a set of scripts. Automated testing tools are capable of executing tests, reporting outcomes and comparing results with earlier test runs. Tests carried out with these tools can be run repeatedly, at any time of day [24].

For our case, this step of the process is really self-explanatory and basically goes as follows :
- Jenkins extracts source files from Stash repository.
- Jenkins executes the Nunit tests.
- Jenkins generates the XML result files and keeps them in the job's workspace.

With this we end the technical context section, after which we will present the existing solution with a critical point of view.

2 Critical review of the existing solution

We can now proceed to studying then criticizing the existing solution.

2.1 Existing solution : Nunit plugin

First, we have to mention that Jenkins does provide an internal dashboard. It simply offers a global trend graph, along with global details about which tests failed.

The following figure is a sample of the Jenkins dashboard.

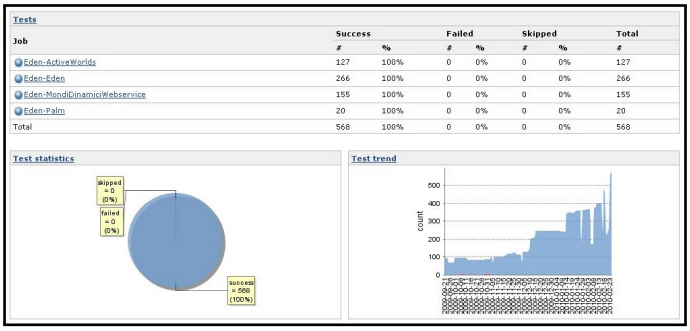


Figure II.1 – Jenkins dashboard

However, being a Java software itself, Jenkins only understands Junit tests. It is not, in any way, configured to understand Nunit test results. In fact, Jenkins treats Nunit tests as regular scripts that take source files as input, then generate XML files as output. It has no clue what is truly going on behind the scenes. The XML files are usually overwritten by new ones, thus going to waste. In order to overcome this problem, the team tried to find a quick workaround, which was no other than the Nunit plugin.

The Nunit plugin is a Jenkins plugin that allows users to include Nunit results in Jenkins Dashboard. What it basically does is to transform the Nunit XML results file into multiple Junit XML result files, and thus trick Jenkins into thinking they are actually Junit tests [25].

The following figure describes the global Nunit plugin's behavior.

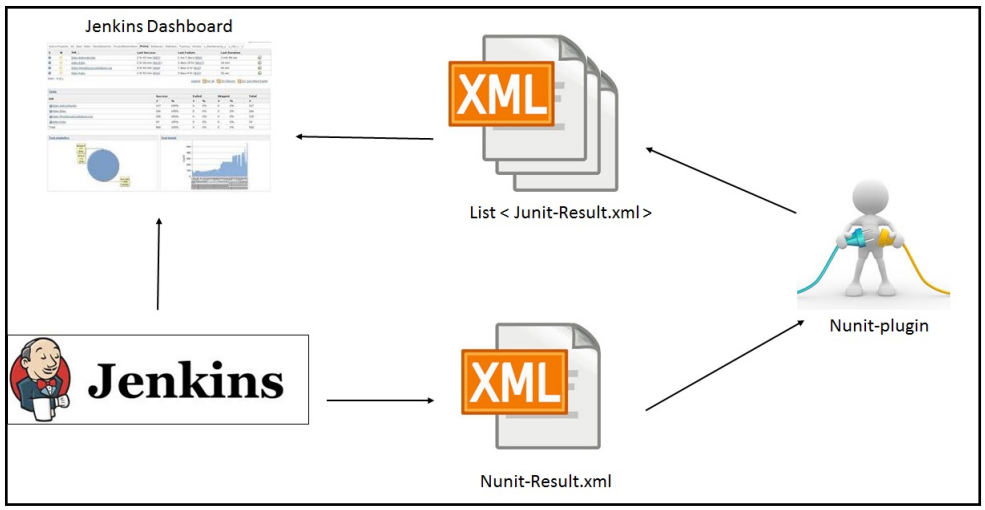


Figure II.2 – Nunit plugin

This method seems about right for a temporary solution, but we can't say as much for anything beyond that. In the following section we will be explaining why this can't be a long term solution.

2.2 Critical review

The existing solution primarily has three major drawbacks. One of which resides in the way the Nunit plugin operates. As we said earlier, the plugin transforms the Nunit result file into Junit result files. Sadly enough, this transformation is far from being optimized. In fact, the Nunit and Junit result files' structures are so different that the transformation quite frequently costs some major data loss or even worse, data distortion.

The second drawback resides in Jenkins' internal dashboard. This latter is known to be very primitive, with very basic and inflexible features. Compared to the complete reporting system the team used to have, we're allowed to say that this dashboard does not at all meet the team's expectations.

Last but not least, the third drawback resides in the absence of a unified global view. What we mean by unified global view is a unified dashboard that provides a global view over all the data that has ever been generated at any given time.
Regarding the unified part, Jenkins can't achieve it because it only provides a separate graph for each job and has no sense of unification. As for the global view part, Jenkins can't achieve it because it simply has no actual data persistence system. In fact, Jenkins uses the XML files as immediate data sources for the dashboard. These XML files have a relatively very short lifetime, which explains Jenkins' inability to provide a global view.

These drawbacks gave birth to a need of yet another reporting alternative that provides all of the previously mentioned points, and that perfectly integrates in the current automation process.

3 Technical choices and global solution

The aim of this section is to provide an overview of recent developments and standards in the field of each of the technologies included in the proposed solution.

3.1 Data reporting systems

In the following section we will present an overview of data reporting systems, then an explanation for the data reporting platform choice.

3.1.1 Overview

A data reporting system is basically a software that provides a formatted and organized presentation of data. Nowadays, this simple definition is undergoing a fundamental shift. During the past ten years, Reporting platforms have tended to be highly governed, where only IT consumers could access the information. Now, a wider range of business users are requesting access to interactive analytical reports, with no need of IT skills. As these demands grow more and more, IT wants to satisfy this requirement without sacrificing governance.

While the need for reporting systems remains, there is a significant change in how companies are satisfying the new business-user-driven requirements. They are increasingly shifting from using the traditional platforms that are the enterprise standards, to more decentralized data discovery. The market is transitioning to platforms that can be rapidly implemented and can be used by business users for quick insights, as well as IT users for quick analytics on how to deliver more business profit. Also, in support of wider user adoption, companies and independent software vendors are increasingly embedding traditional reporting, dashboards and interactive analysis into business processes or applications [26].

As a result of the market dynamics discussed above, reporting systems are defined as platforms that deliver several critical features :

- Provide a common look and feel interface.
- Provide a sufficient security level along with administration tools.
- Allow data exploration via the manipulation of chart images, with the color, brightness, size, shape and motion of visual objects representing aspects of the dataset being analyzed. These tools enable users to analyze the data by interacting directly with a visual representation of it.
- Provide highly interactive dashboards with visual exploration and embedded advanced analytics.
- Provide the ability to create highly formatted, print-ready and interactive reports.
- Provide collaboration and social integration through discussion threads.

From another angle, data reporting can be presented as an architecture, tool, technology or system that gathers and stores data, analyzes it using analytical tools, and delivers information and knowledge, facilitating reporting, and ultimately allowing organizations to improve decision making. To put it shortly, data reporting can be defined as the process that transforms data into information and then into knowledge[27].

The following figure presents the data reporting chain, adapted to our context.

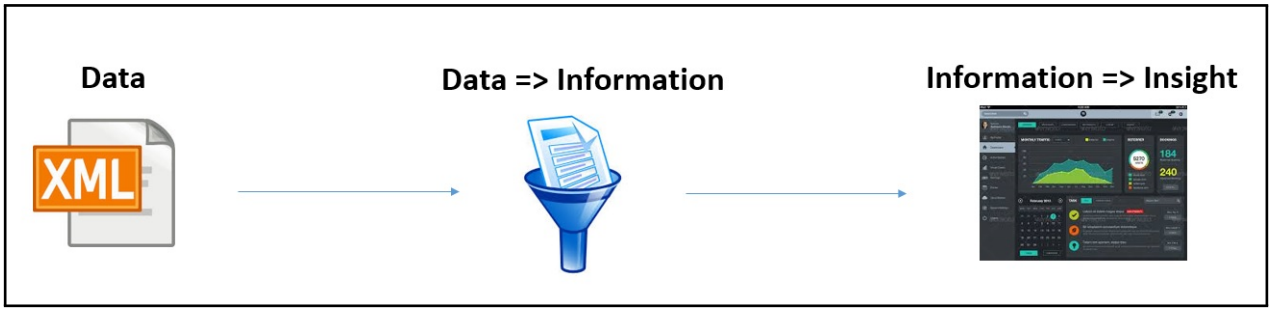


Figure II.3 – Reporting chain

There are currently several existing solutions on the market that satisfy these requirements. They greatly vary, nonetheless, on strengths and weaknesses.

3.1.2 Technical choice : QlikView

QlikView is a market leader in data discovery. QlikView is a mature, self-contained, tightly integrated development platform used by IT or more technical users for building intuitive and interactive dashboard applications faster and easier than traditional reporting platforms. It gives business users the ability to build their own dashboards while giving IT the ability to govern, manage, scale and embed them [28].

QlikView's position as a Leader in this Magic Quadrant is driven by strong vision around governed data discovery. The following figure presents the magic quadrant of reporting platforms for the year 2015 [29].

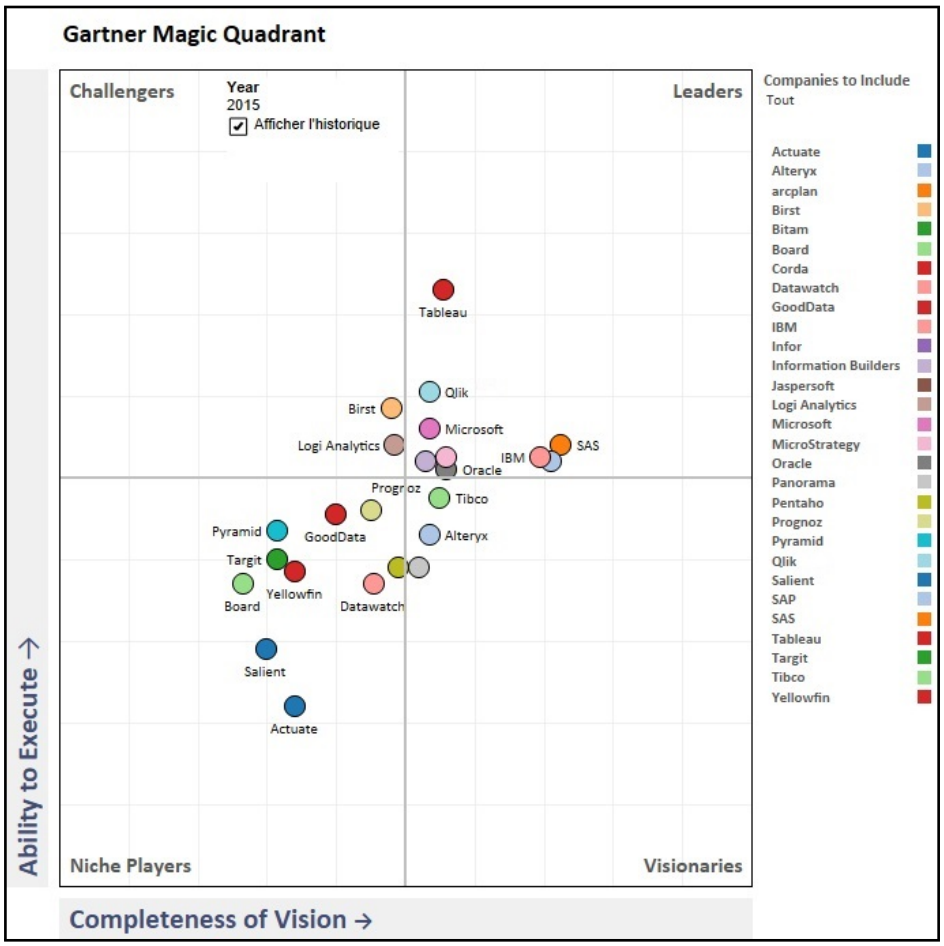


Figure II.4 – Magic quadrant for reporting platforms

- **Strengths :**
  QlikView offers a highly interactive dashboard development product portfolio that spans business-user self-service, centralized dashboard application development, and IT needs for enterprise features to support governed data discovery.

  Ease of use, particularly for dashboard consumers, is a key reason behind the choice of QlikView, in addition to low implementation time and effort through intuitive interfaces. This combination has been a key driver of its success.

  Moreover, QlikView is known for its user enablement, which includes documentation, online training, tutorials, user communities and conferences, and for wide availability of skills. In fact, the QlikView Community offers an online collaboration hub and center of excellence for prospects, customers, partners and employees [30].

- **Weaknesses :**
  A common complaint with QlikView is that it can be difficult to learn and operate when you first get started because of its many facets and intricacies. Additionally, the back end isn't quite as user-friendly and intuitive as the front end and does occasionally require IT assistance for some data management and mapping. Also, when the data load crashes the error tracing can sometimes be complicated.
  Additionally, QlikView is demanding in regard to hardware, which can be very costly [31].

Based on the reporting platforms magic quadrant, and after considering the different strengths and weaknesses of QlikView, we have come to the conclusion that QlikView is the best choice for our case.

3.2 Web services

In the following section we will present an overview of web services, then an explanation for the web services technology choice.

3.2.1 Overview

By definition, Web Services are self-contained, modular, distributed, dynamic applications that can be described, published, located, or invoked over the network to create products, processes, and supply chains. These applications can be local, distributed, or Web based. Web services are built on top of open standards such as TCP/IP, HTTP, Java, HTML, and XML.

The concept of web services (WS) has gained great importance during the last few years. WS could be especially useful for the creation of dynamic business applications and to allow Java EE and .NET technologies to interoperate.

New WS standards have been developed through the cooperation of several corporations. Numerous existing concepts such as business process management, security, directory services, routing and transactions are being adapted for WS [32].

Using web services offers many benefits such as :

- **Reusability :**
  A Web service is a component which can be remotely accessed using HTTP. Web Services provide a means to make a pre-existing code available through the network. As a result,

the program's functionalities can be invoked by other applications.

- **Interoperability :**
  Web Services enable the share of data and the communication between different applications. For example, .NET applications can interact with Java web services. Thus, applications become technology independent.
- **Standardized Protocol :**
  Web Services uses industry standard protocol for the communication. This standardization of protocol gives the business many advantages like wide range of choices, reduction in the cost due to competition and increase in the quality [32].

Web services have had multiple implementations based on different architectures. Most notably, we find the RESTful web services based on REST architecture.

3.2.2 Technical choice : RESTful web services

REST is the abbreviation of **Representational State Transfer** and designates an architecture style used to create networked applications. It uses a stateless, client-server, cacheable communications protocol which is almost always the HTTP protocol. Its original feature is to work by using mere HTTP to make calls between machines instead of choosing complex mechanisms such as CORBA, RPC or SOAP [33].

- **RESTful resources :**
  The pieces of information the RESTful technology exposes are referred to as resources. Each resource is linked to a global URI. These resources are accessed by components of the network which communicate through HTTP and exchange representations of these resources. In order to interoperate with a resource, an application must possess both the resource's identifier and the required method.
  However, the application must be capable of interpreting the data format returned from the resource, which is often an HTML or XML document, though it may also be an image, plain text, or any other content [33].

- **RESTful principles :**
  The REST architectural style is based on four principles :
  - Resource identification through URI : Resources are identified by URIs.
  - Uniform interface : A fixed set of four CRUD operations is responsible for the manipulation of resources.

- Self-descriptive messages : Resources' content can be presented in several formats. Metadata about the resource can be used to detect transmission errors and perform authentication or access control.
- Stateful interactions through hyperlinks : Stateful interaction can be achieved using several techniques (e.g. hidden form fields) [33].

- **RESTful HTTP methods :**
  Restful applications use HTTP requests to post data, read data and delete data. Thus, REST uses HTTP for all four CRUD operations (Create/Read/Update/Delete). The following figure summarizes the four HTTP methods, and shows the possible HTTP response codes.

Table II.1 – REST HTTP methods

HTTP verb	CRUD	list of items (/items)	Single item (/item(id))
POST	Create	– 201 : Created.	– 201 : Created. – 404 : Not found. – 409 : Conflict (resource already exists).
GET	Read	– 200 : OK (returns list of items).	– 200 : OK (returns single item). – 404 : Not found (ID not found or invalid).
PUT	Update	– 404 : Not found.	– 200 : OK . – 204 : No content. – 404 : Not found.
DELETE	Delete	– 404 : Not found.	– 200 : OK . – 404 : Not found.

- GET method : The HTTP GET method is used to retrieve a representation of a resource. GET returns a representation in XML or JSON along with a response code.
- PUT method : The HTTP PUT method is most-often utilized for data updating. It contains the newly-updated representation of the original resource. However, PUT can also be used to create a resource. Again, the request body contains a resource

representation.
- POST method : The POST method is most-often utilized to create new resources.
- DELETE method : The DELETE method is pretty easy to understand. It is used to delete a resource identified by a URI [34].

- **RESTful security :**
  REST does not offer any security features. But this problem can be avoided as REST can be used on top of HTTPS (secure sockets[33]).

- **RESTful strengths :**
  RESTful has some aspects which can be viewed as positive. First, RESTful Web services appear to be simple because REST applies many existing well-known standards (HTTP, XML, URI, and MIME) and needs an infrastructure that has already become ordinary.
  Second, HTTP clients and servers are compatible with all programming languages and operating system/hardware platforms.
  Last but not least, only a small effort is needed to build a client of a Restful service. Services can be tested using simply a mere web browser and the development of a client software becomes superfluous [33].

- **RESTful weaknesses :**
  RESTful has also some aspects which can be viewed as negative. On one hand, encoding a large amount of input data in the resource URI is impossible because the server either refuses such requests or crashes. It may also be challenging to encode complex data structures into URI as there is no commonly accepted marshalling mechanism.
  On the other hand, Restful web services currently have no standard vocabulary to describe the web service interface. Both the service consumer and service producer must have an out of-band agreement. Services can be described using Web Application Description Language (WADL). It is an XML-based file format that provides a machine readable description of REST web services. However, WADL is not yet widely supported [33].

RESTful web services have appeared as a lightweight alternative approach to design web services. A central advantage of a RESTful API is its flexibility. Various applications can be provided with system's resources through data formatted in a standard manner. This technology allows to abiding by integration requirements which are crucial for the conception of systems enabling easy combination of data. This is the reason RESTful web services are the best choice in our case.

3.3 XML file parsing

3.3.1 XML overview

XML, which stands for Extensible Markup Language, is the most common format of text oriented documents. When it comes to software engineering, XML has proven to be inevitable. It is essentially useful with document storage, as well as data transfer between applications. Both its flexibility and its extensibility have made several technologies revolve around it (e.g. Jenkins and Nunit).
XML provides a method to analyze and parse XML documents, using XSD. XSD stands for XML Schema Definition, and is a recommendation from the World Wide Web Consortium. It specifies how to formally describe the elements of an XML document. Moreover, it can be used to express a set of rules to which the XML document must conform.

XML Schema is used to create a vocabulary for an XML instance, and uses namespaces heavily. An XML namespace is a collection of XML elements and attributes identified by an Internationalized Resource Identifier, often referred to as an XML vocabulary. One of the primary motivations for defining an XML namespace is to avoid naming conflicts when using and reusing multiple vocabularies.

Also, the XSD Datatype Specification defines numerous datatypes for validating the element content and the attribute value. These datatypes can be used to validate only the scalar content of elements, and not the non-scalar or mixed content. The text enclosed between the <opening> and </closing> element tags, and the value of the attributes are often referred to as scalar data, but it can also be a list of scalar data. These datatypes are intended for use in XML Schema definition and other XML-related documents [35].

Having thoroughly explained the XML structure and data types, we move on to the XML file parsing.

3.3.2 XML parsing methods

In order to extract the information found in the XML file, we need an XML file parser. A file parser is a program which analyses files to identify the component parts. While reading a file, a parser checks the file's syntax for both conformity and data extraction. In order to achieve this, these parsers make use of different parsing methods. There are mainly two parsing methods : hierarchical method and event-based method [36].

– **Hierarchical method :**
Based on this method, the XML file parser builds a hierarchical structure that contains objects representing the document elements, with properties accessible through specific methods.
This method makes use of the DOM (Document Object Model) concept. The following figure illustrates the parsing process.

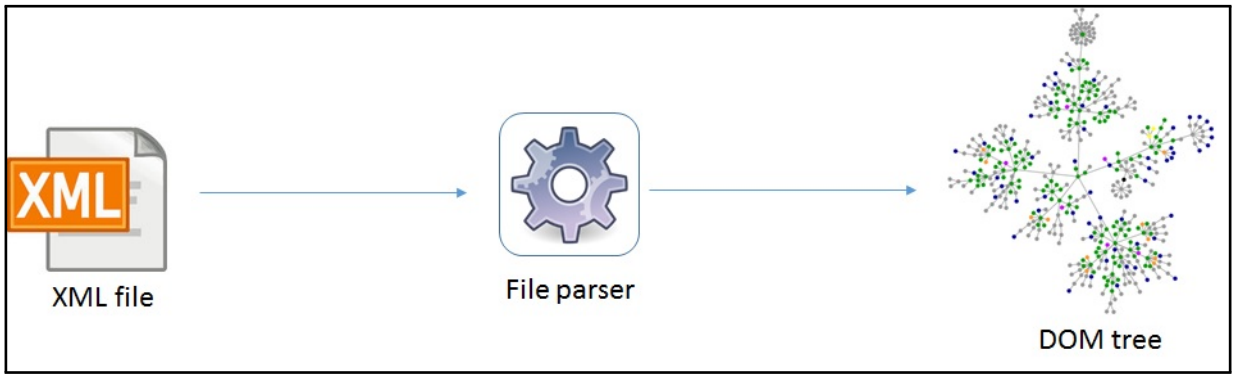


Figure II.5 – Hierarchical parsing method

– **Event-based method :**
Event-based parsers react to multiple events (e.g. document start, document end) and sends the results back to the system using the parser. The following figure illustrates the parsing process.

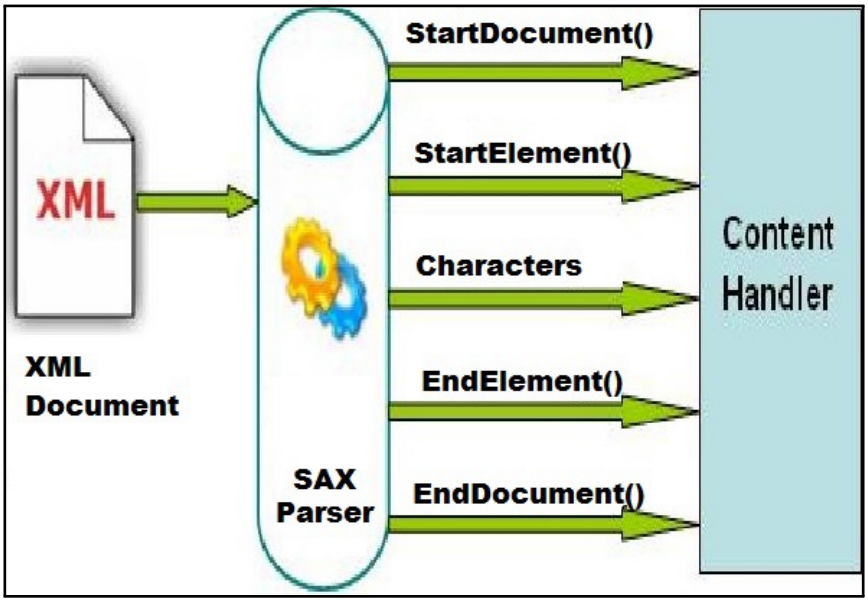


Figure II.6 – Event-based parsing method

– **Comparison :**
The following table presents the comparison between the two previously mentioned methods [37].

Table II.2 – Difference between XML parsing methods

	Advantages	**Disadvantages**
Event-based method	– Simple and fast. – Reduces parsing to only the requested parts of the file, based on the events.	– The parsing is sequential. – the entire document needs to be parsed.
Hierarchical method	– Generic. – Easy implementation. – Easy data manipulation through the use of nodes.	– Might require a lot of system resources depending on the file's size.

3.3.3 Technical choice : DOM parsing method

After this brief analysis, we can deduce that both methods can be efficient and the choice uniquely depends on our needs.
For the XML file parsing, we have chosen to go along with the hierarchical parsing method (DOM), as it better suits our needs, especially that we already have to transform the XML elements into objects.

3.4 Global solution with technical choices

After studying the different involved technologies and specifying the technical choices based on that study, it became easier to get a better global view over the proposed solution. Briefly put, we have chosen QlikView as our reporting platform, RESTful as our web service technology and DOM as our XML Parsing method. For the data storage, we are using Microsoft Sql server, as it is an imposed standard in the Microsoft-oriented company.

The following figure is a resumption of the global solution (previously presented) in addition to the technical context and the technical choices mentioned in this chapter.

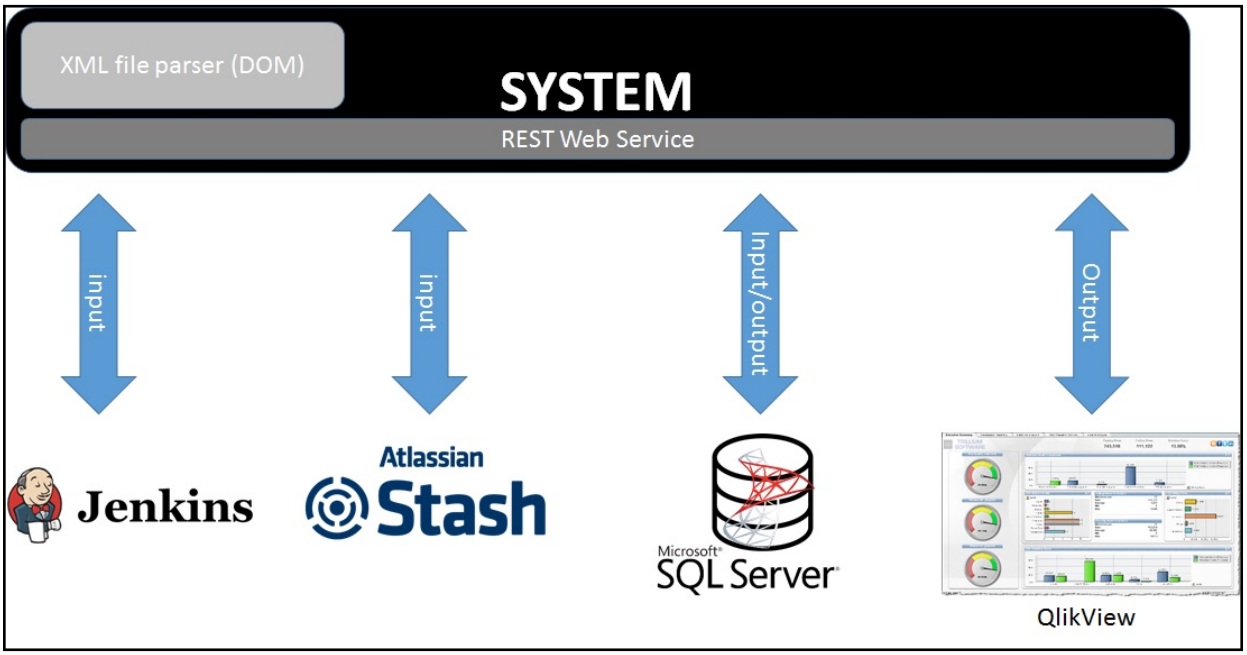


Figure II.7 – Global solution after technical choices

Conclusion

In this chapter we have gone through the technical context of the project, the detailed study of the potentially required technologies, then we specified our major technical choices.
The better understanding of the different technologies and standards mentioned here has helped us better understand the functional and non-functional requirements of our project.

Chapter III

Requirement specification & analysis

Plan

1 Actors 36

2 Requirement analysis 36

2.1 Functional requirements 36

2.2 Non-functional requirements 37

3 Use case diagrams 38

3.1 Global use case diagram 38

3.2 Detailed description 40

4 System sequence diagrams 46

4.1 Authentication 46

4.2 Add a new job to the system 47

4.3 Save a Jenkins job's build data 48

4.4 Perform a global search through Jenkins history for unsaved builds . . 49

4.5 Change a Jenkins job's project, squad and team 50

4.6 Change a Jenkins job's test type 51

Introduction

This chapter represents an analytical phase in which we fully describe our system's behavior. We first describe the system's main actors, then we iterate over the different functional and non-functional requirements. Afterwards we analyze those requirements through use case diagrams and system sequence diagrams in which we describe every possible interaction between the actors and the system to be.

1 Actors

In our system we have three actors :

- **User :**
  The simple user can only register their Jenkins job in order to be recognized by the system, and consult the final dashboard.

- **Jenkins :**
  Jenkins is the main actor of our system. It will automatically trigger our system if certain conditions are met. We will go through the details of this interaction in the following sections.

- **System administrator :**
  A system administrator is the super user that has the ability to outweigh Jenkins and change certain job related information and the way our system treats them.

After defining the system actors, we move on the the requirement analysis.

2 Requirement analysis

The requirements are divided in two groups : Functional requirements and non-functional requirements.

2.1 Functional requirements

Functional requirements are basically the contract that ties the system developers with the clients. That is why our system has to provide the following features :

- **User :**
  - Configure a Jenkins job to automatically save data (through our system) after every build (This is realized by the addition of a PowerShell script as a post-build action).
  - Consult a test per build velocity dashboard.

- **Jenkins :**
  - In order to ensure both flexibility and availability, Jenkins has to have two available methods for accessing the system : A web service (to fit in the SOA) and a Jenkins plugin (for security issues).
  - Save a job's related build data with or without external references.
  - Save a job's related Nunit test results with or without external references.
  - Load a job's related data through accessing Jenkins REST API.
  - Load a job's related Git repository information through Stash REST API.
  - Automatically figure out the project, squad and team the job belongs to.

- **System administrator :**
  The system administrator inherits all of the user's functionalities in addition to the following ones :
  - Manually register a Jenkins job in order to be recognized by the system.
  - Manually trigger a global search through Jenkins history links to find unsaved builds and save them.
  - Manage a Jenkins job related data (project, squad, team, test type) through CRUD operations (Create, Read, Update, Delete).
  - Authenticate to the system through the company's Active Directory (if a user belongs to a certain group).

2.2 Non-functional requirements

Our main objective is to build a very performing solution. However, a system that only satisfies the functional requirements rarely satisfies the client. For this reason we have to also focus on non-functional requirements when developing the solution :

- **Ergonomics :**
  The system has to provide a clear, intuitive and legible front-end.

- **Consistency :**
  The system has to always provide coherent data and matching the data provided by other systems.

- **Performance :**
  The system has to minimize the response time while saving and manipulating the massive data provided by the external actors.

- **Flexibility :**
  The system has to provide several usage methods so as to suit the user's needs and requirements.

- **Extensibility :**
  Integration of new modules or features can be achieved without affecting already existing modules.

- **Maintenance :**
  The system has to be easily maintainable. A good method to ensure this requirement is the clean coding patterns (e.g. variable names, function size), and a good documentation.

- **Portability :**
  The system has to be portable in way that it runs under any working environment.

After the requirement analysis, specification proves to be crucial so as to get a better understanding of the required functionalities. This analysis will be based on use case and system sequence diagrams.

3 Use case diagrams

Use case diagrams allow us to better describe the interactions between the different actors and the system.

3.1 Global use case diagram

In this diagram we regroup all of the use cases, divided into sub-modules to reduce complexity and have a better global view.

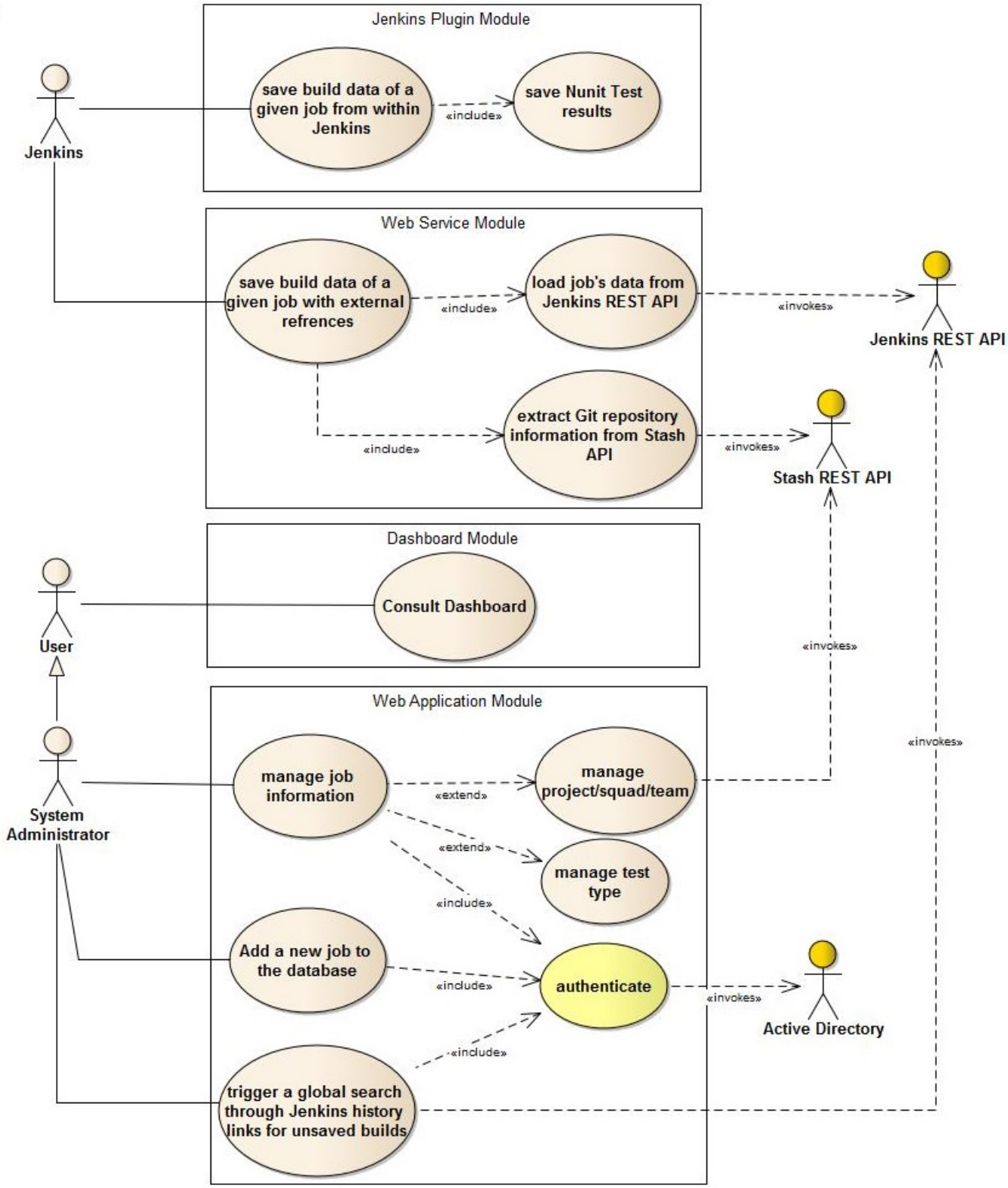


Figure III.1 – Global use case diagram

3.2 Detailed description

3.2.1 Jenkins plugin module

- **Use case diagram :**

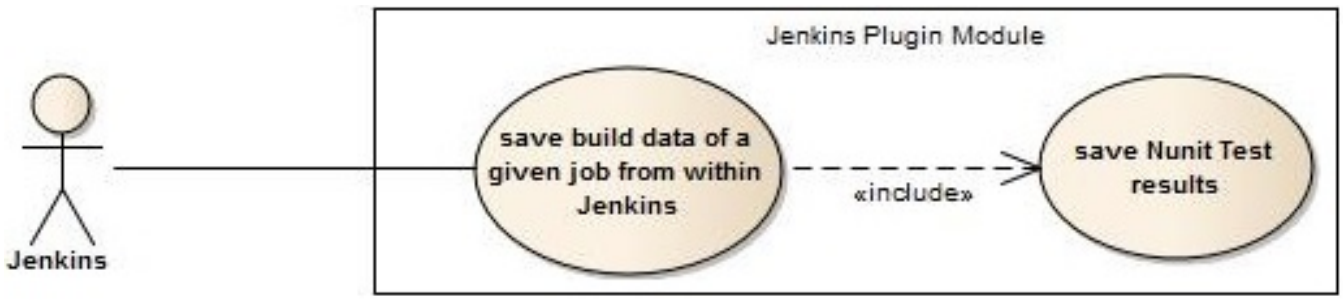


Figure III.2 – Jenkins plugin use case diagram

- **Case : Save build data of a given job from within Jenkins**
  In this case, Jenkins uses the plugin to save a given job's build data and then save the Nunit test results.

Table III.1 – Save build data of a given job from within Jenkins

Use case	Save build data of a given job from within Jenkins.
Primary actor	Jenkins
Brief	The system saves a job's build data through a Jenkins plugin.
Preconditions	The Jenkins job has to be successfully built by Jenkins and has generated Nunit test results XML files.
Post-conditions	The build data and the test results (if available) are both saved in the system.
Triggers	This case is triggered whenever a Jenkins job finishes building.
Basic flow	– The system receives build data through the Jenkins plugin. – If the test results files are available, the system receives the XML files through the plugin. – The system extracts the information from the XML files and transforms them into structured data. – The system saves the provided data in the database.
Alternative(s)	– The Jenkins job didn't complete correctly. – No Nunit test results were generated.

3.2.2 Web service module

- **Use case diagram :**

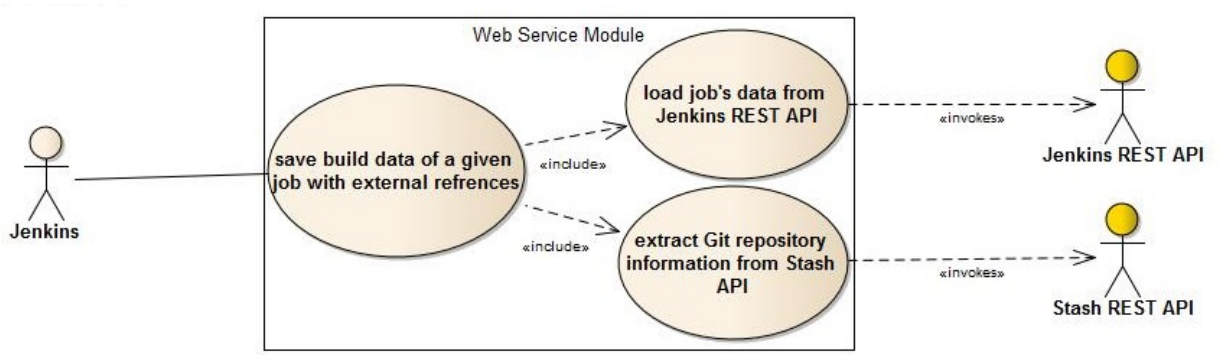


Figure III.3 – Web service use case diagram

- **Case : Save build data of a given job with external references**
  In this case, Jenkins uses a web service exposed by our system to save a given job's build data and then save the Nunit test results.

Table III.2 – Save build data of a given job with external references

Use case	Save build data of a given job with external references.
Primary actor	Jenkins
Brief	The system saves a job's build data through a web service call.
Preconditions	The Jenkins job has to be successfully built by Jenkins and has generated Nunit test results XML files.
Post-conditions	The build data and the test results (if available) are both saved in the system.
Triggers	This case is triggered whenever a Jenkins job finishes building.
Basic flow	– Jenkins provides the web service with the current job's URL. – The system uses that URL to access Jenkins REST API and retrieve job's information, build data and Nunit test result files. – If the job is new to the system, the system uses the provided Git information to access Stash REST API and retrieve the job's related data (project,squad and team). – The system extracts the information from the XML files and transforms them into structured data. – The system saves the provided data in the database.
Alternative(s)	– The Jenkins job didn't complete correctly. – No Nunit test results were generated.

3.2.3 Web application module

- **Use case diagram :**

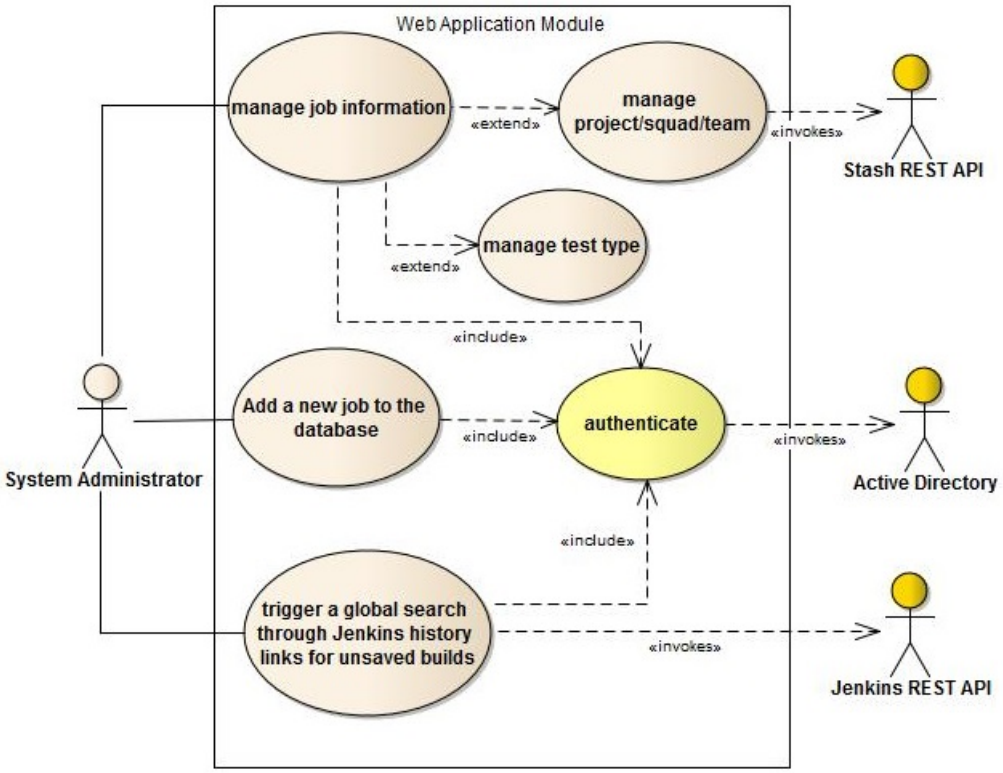


Figure III.4 – Web application use case diagram

- **Case : Add a new job to the database**
  In this case the user adds a new job to the database.

Table III.3 – Add a new job to the database

Use case	Add a new job to the database
Primary actor	System administrator
Brief	Adds a new job to the database so that it is recognized by the system.
Preconditions	The user has to be authenticated (belongs to the administrators group).
Post-conditions	A job is successfully added to the database and is recognized by the system.
Triggers	When the user clicks on the "add new job" link and fills a certain form.
Basic flow	– The administrator chooses to add a new job. – The system redirects the administrator to a form page where they provide the job's URL and the test type. – The system then saves the newly introduced job

- **Case : Change test type**
  In this case the system changes a job's test type.

Table III.4 – Change test type

Use case	Change test type.
Primary actor	System administrator
Brief	The system changes a job's test type
Preconditions	The job has to be recognized by the system. The user has to be authenticated (belongs to the administrators group).
Post-conditions	The job's test type has to be upgraded.
Triggers	Choosing a job the test type of which has to be changed.
Basic flow	– The administrator chooses the job from a list. – The administrator chooses a new test type (System, Service or Unit). – The system saves the new test type.

- **Case : Change project, squad, and team**
  The system changes the project, the squad and the team the job belongs to.

Table III.5 – Change project, squad, and team

Use case	Change project, squad, and team.
Primary actor	System administrator
Brief	The system changes the project, the squad and the team the job belongs to. The administrator either chooses from a list or introduces new ones.
Preconditions	The job has to be recognized by the system. The user has to be authenticated (belongs to the administrators group).
Post-conditions	The job's project, squad and team association has to be upgraded.
Triggers	Choosing a job the project, the squad or the team of which has to be changed.
Basic flow	– The administrator chooses the job from a list. – The system offers the choice between selecting a project/squad from a list and introducing new ones. – If the administrator selects a project/squad from a list, then the team is automatically updated (the one the project and squad belong to). – Else if the administrator chooses to introduce a new project/squad association, then the system provides an interface through which they provide the Stash (Git) URL. – The system looks up the correspondent project and squad, then adds them to the system's database. – The administrator either selects a team from a list or introduces a new one.

- **Case : Trigger a global search through Jenkins history links for unsaved builds**
  The system searches through Jenkins history links for unsaved builds then saves them.

Table III.6 – Trigger a global search through Jenkins history links for unsaved builds

Use case	Trigger a global search through Jenkins history links for unsaved builds.
Primary actor	System administrator
Brief	The system searches through Jenkins history to save unsaved builds.
Preconditions	The user has to be authenticated (belongs to the administrators group).
Post-conditions	All of the recognized jobs have their build data up-to-date and fully saved.
Triggers	The administrator manually triggers this functionality.
Basic flow	– The administrator chooses the job to be searched through. – The system calls Jenkins REST API to get all the available builds. – The system compare those builds' data with those saved in the database. – If it finds any unsaved build data it saves it, along with the Nunit results. – The system provides a log console for the administrator to visualize results.

- **Case : Authenticate**
  In the case, the system authenticates the credentials provided by the user.

Table III.7 – Authenticate

Use case	Authenticate
Primary actor	System administrator.
Brief	The system authenticates the username and password provided by the user, then either grants or revokes access depending on the authentication result. It only grants access to users having an Active Directory account and belonging to the administrators group.
Preconditions	The user has to have an Active Directory username and password, and that belong to an Active Directory group called "SLT Admins".
Post-conditions	Access is granted or revoked depending on the authentication results.
Triggers	This case is triggered when a user wants to access the application.
Basic flow	– The system provides an interface for the user to introduce a username and a password. – The system access the company's Active Directory to check the validity of the user's credentials and whether or not they belong to the administrators group. – If the validity is verified, the user is granted access. – Else, the access is revoked and a message is shown explaining the reason.

3.2.4 Dashboard module

- **Use case diagram :**

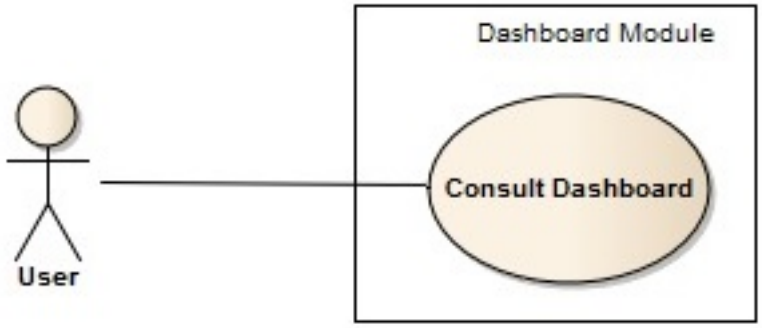


Figure III.5 – Dashboard use case diagram

- **Case : Consult dashboard**
  In this case, the system provides a dashboard for the user.

Table III.8 – Consult dashboard

Use case	Consult dashboard
Primary actor	User
Brief	The system provides a dashboard that presents tests per build velocity graphs, test per build average graphs, test pass rate graphs, and build details reports, along with a builds complete history.
Preconditions	No preconditions.
Post-conditions	The user can view the provided graphs with up-to-date data.
Triggers	The dashboard is automatically updated once a day.
Basic flow	– The user visits the dashboard's URL. – The system shows the dashboard. – The user can select and filter data as it suits them.
Alternative(s)	No alternatives.

After specifying the different use case diagrams, we move to the system sequence diagrams, which better describe the system's interaction with the external actors.

4 System sequence diagrams

Sequence diagrams are the graphical representation of the possible interactions between the actors and the system in a chronological order.

In the following section we will be presenting the sequence diagrams of the major use cases mentioned above.

4.1 Authentication

The system administrator has to authenticate in order to access certain parts of the application. They have to provide valid Active Directory credentials that belong to an Active Directory group called "SLT Admins".

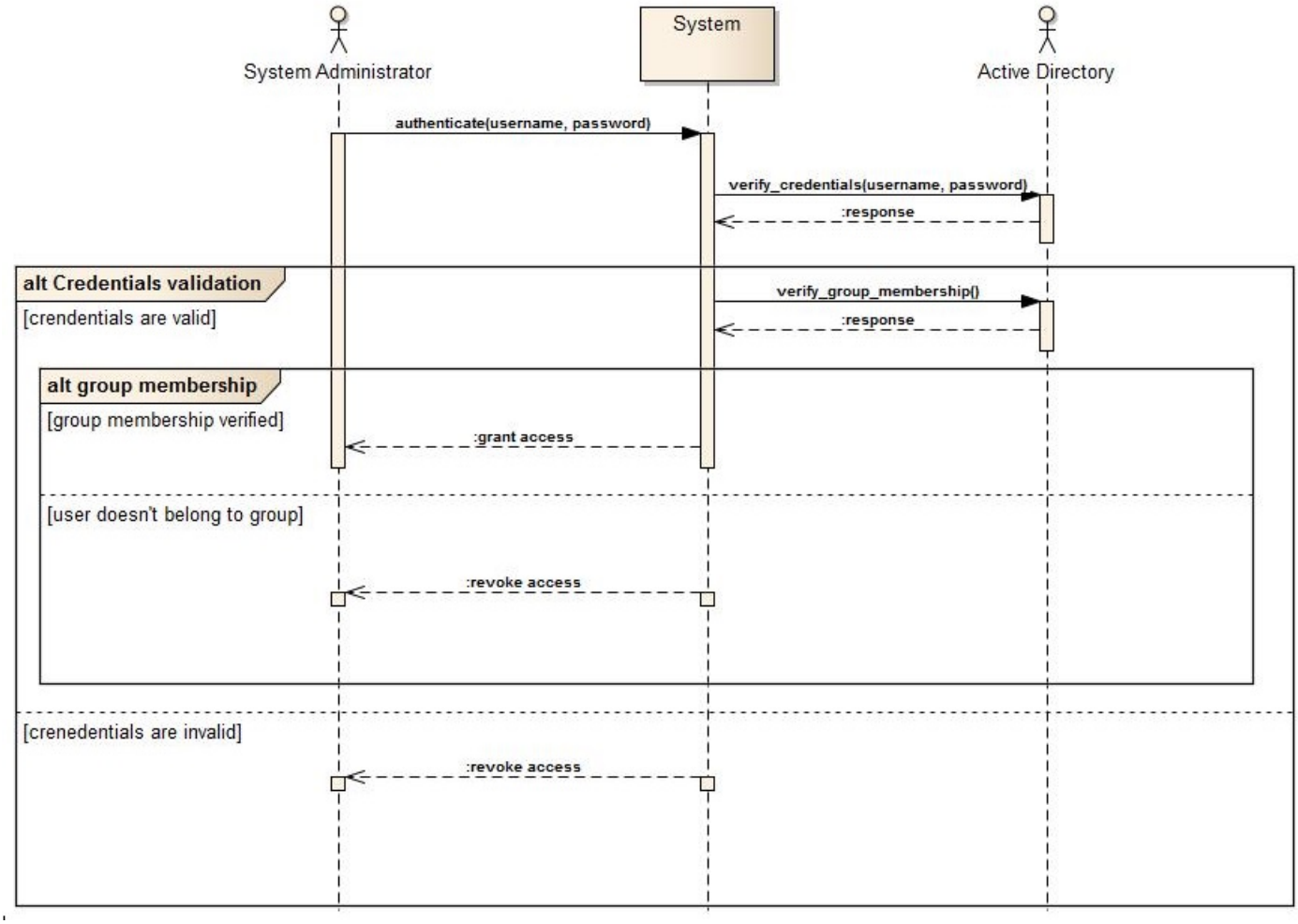


Figure III.6 – Authenticate sequence

4.2 Add a new job to the system

The system administrator can add a new Jenkins job to the system in order to be recognized in future procedures and build data updates.

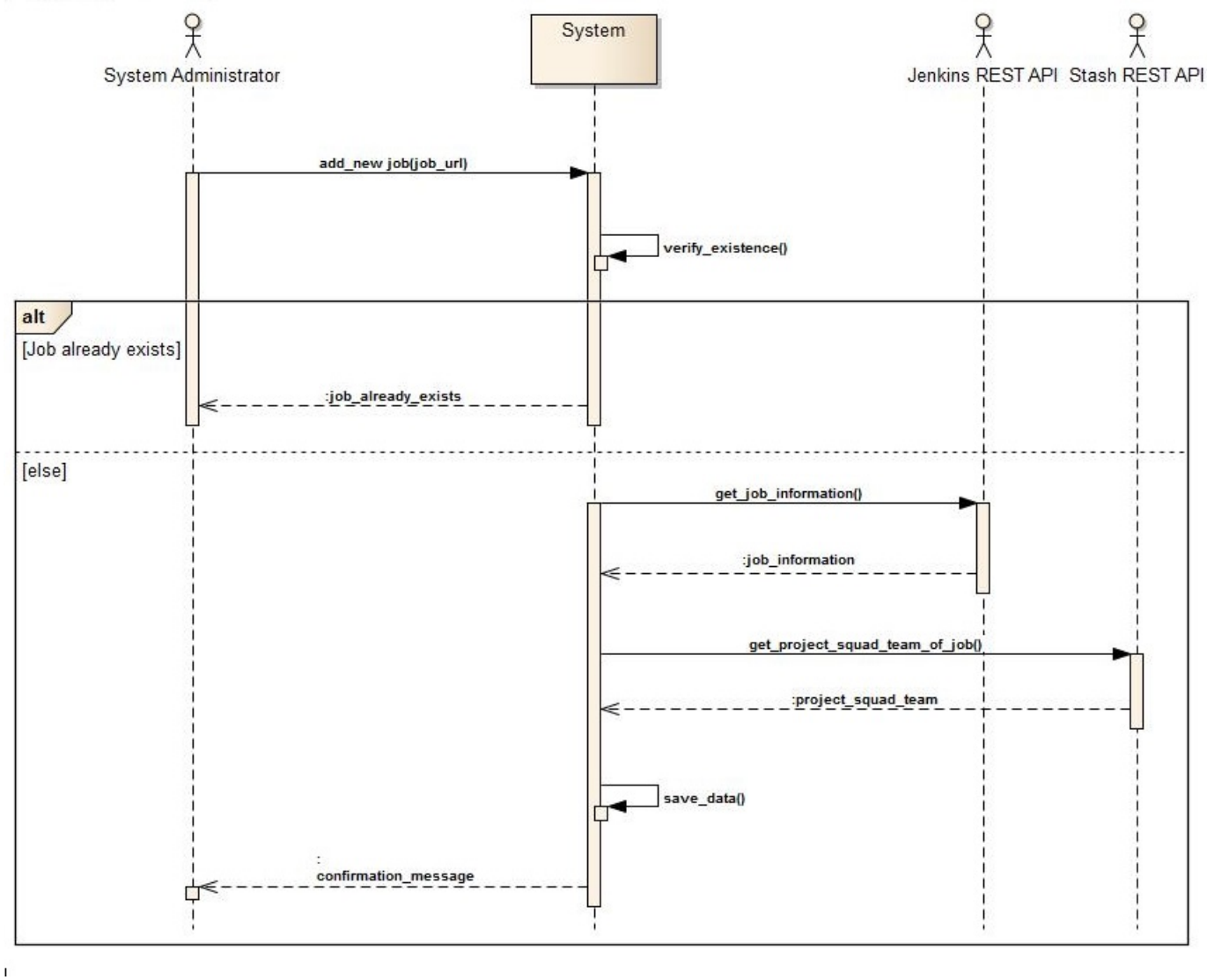


Figure III.7 – Add new job sequence

4.3 Save a Jenkins job's build data

Jenkins can save a job's build data in the system after each job build. If the job is new to the system, it will be added in the database in order to be recognized in future procedures.

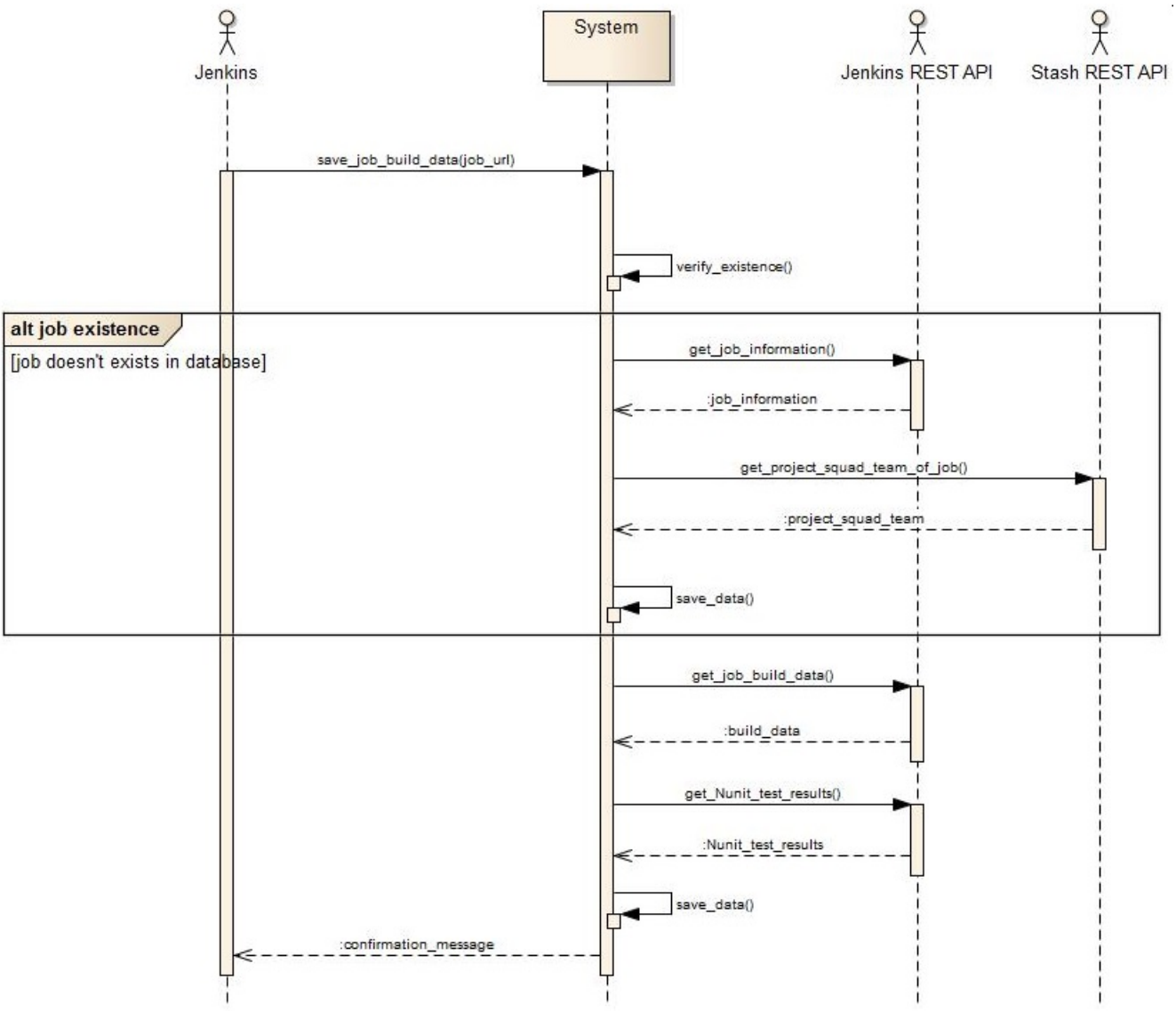


Figure III.8 – Save job build data sequence

4.4 Perform a global search through Jenkins history for unsaved builds

The system administrator can perform a global search through Jenkins history to verify if any of the recognized jobs has unsaved builds and unsaved Nunit test results.

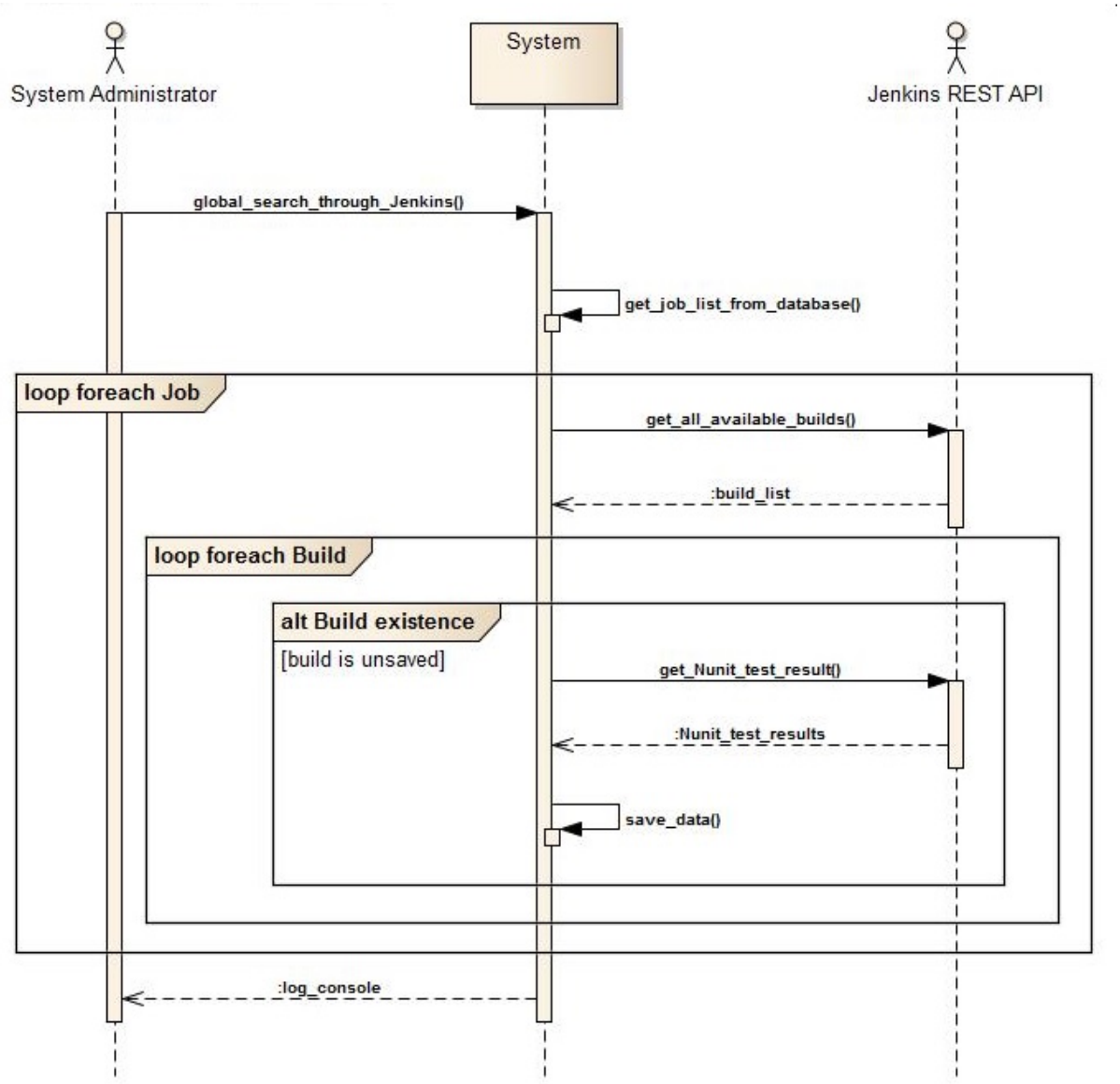


Figure III.9 – Global search through Jenkins sequence

4.5 Change a Jenkins job's project, squad and team

The system administrator can modify a job's project, squad and team. However, in order to ensure the system's consistency, they can't do so freely. The project and squad have to be matching those found in Stash repository (Git).

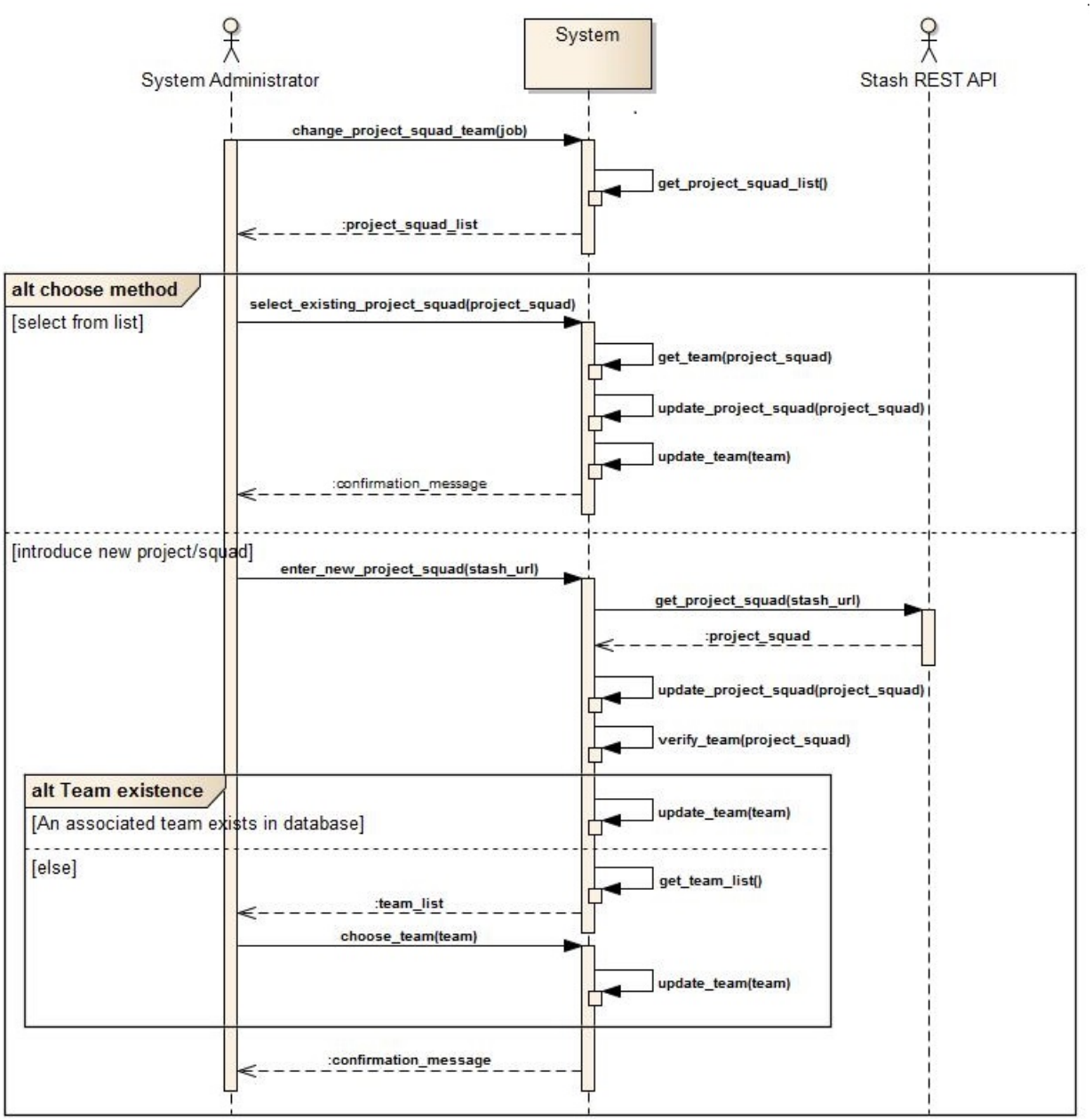


Figure III.10 – Change project squad and team sequence

4.6 Change a Jenkins job's test type

The system administrator can update a Jenkins job's test type. The system only recognizes three test types : System (UI) tests, Service tests and Unit tests.

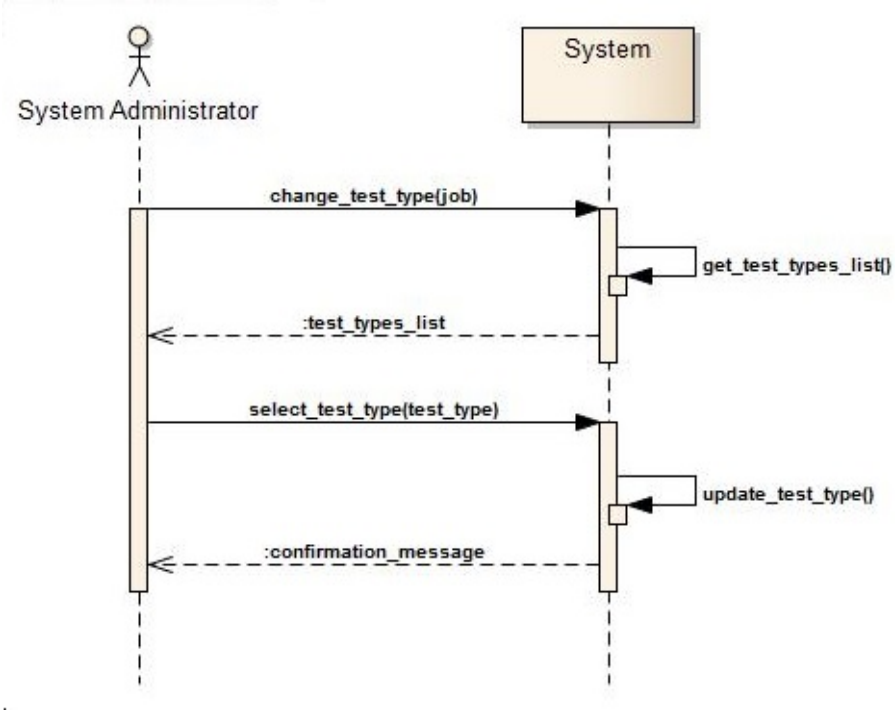


Figure III.11 – Change test type sequence

Conclusion

This chapter has helped us get a better understanding of the functional and non-functional requirements. After decorticating the clients' needs to the tiniest details through the use case and system sequence diagrams, we move on to the system design, which will be presented in the next chapter.

Chapter IV

Solution design

Plan

1 Global design 53

1.1 Global architecture 53

1.2 Package model 55

1.3 Design patterns 56

2 Domain design 58

2.1 Jenkins plugin class model 58

2.2 Web service class model 59

2.3 XML file parsing class model 60

2.4 Data persistence class model 61

2.5 Web application class model 62

3 Behavioral design 67

3.1 Object sequence diagram : Add a new job 68

3.2 Object sequence diagram : Authenticate 69

3.3 Object sequence diagram : Global search through Jenkins 70

3.4 Object sequence diagram : Save job's build data 71

3.5 Object sequence diagram : Change project, squad and team 72

3.6 Object sequence diagram : Change test type 73

4 Database design 73

Introduction

In order to satisfy every requirement, we are bound to set up a well-structured design for our solution. starting from the global design, we move on to the domain design (class models), behavioral design (object sequence models) and finally to the database and dashboard design.

1 Global design

In this section we will be presenting the global architecture with its different layers, the packaging model and the design patterns put to use in this project, namely the singleton, the Adapter, the Composite, the Decorator and the Iterator patterns.

1.1 Global architecture

The following figure presents the solution's global architecture. In this figure, the orange colored blocks are the modules we will develop, while the gray colored blocks are external modules or actors.

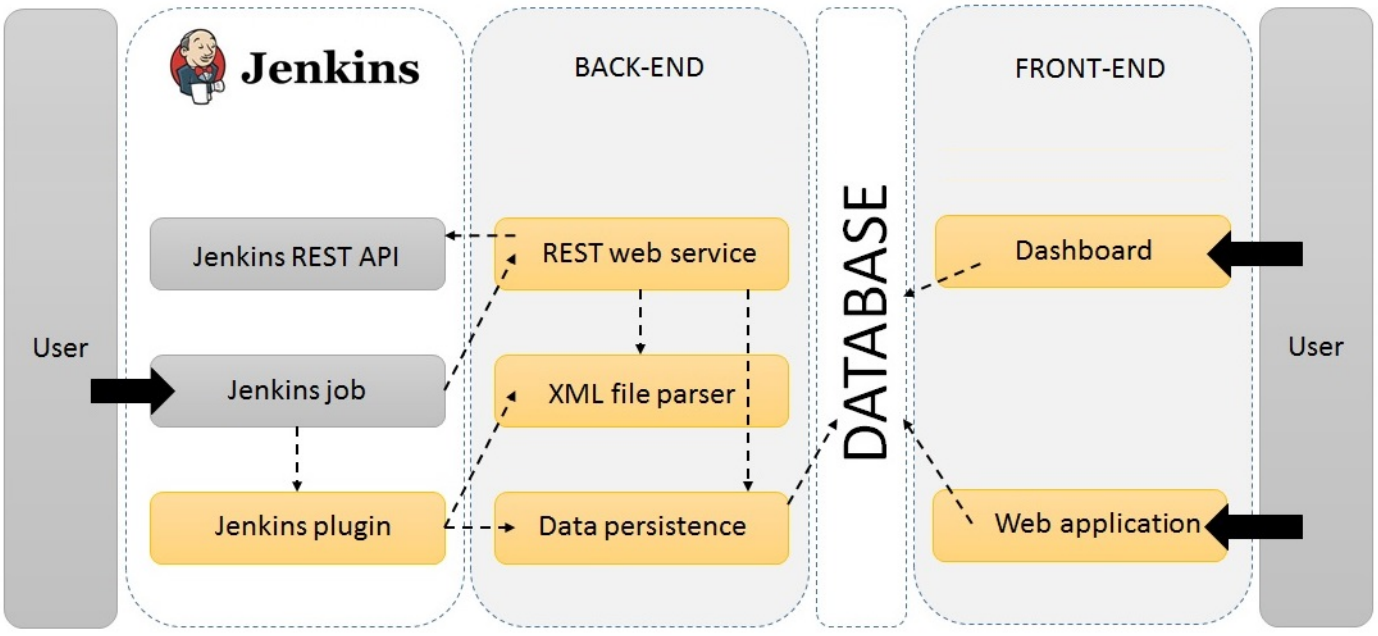


Figure IV.1 – Global architecture

We can also distinguish three logical layers : Jenkins layer, Back-end layer, and Front-end layer.

- **Jenkins layer :**
  - This layer represents the Jenkins platform. An already existing layer, to which we will add an internal Jenkins plugin that connects the platform to our system.
  - The plugin is one of the two entry points to our solution. The other one is the RESTful web service. The user can choose which entry point they want to use, by configuring the Jenkins job.
  - The Jenkins REST API is used by the web service for data collection.



Figure IV.2 – Jenkins layer

- **Back-end :**
  - The back-end is the layer where most of the logical operations take place. First we have the RESTful web service, which is the second entry point to the solution, and which can be called by a Jenkins job.
  - The XML file parser is a module that transforms the Nunit result file (XML) into a DOM tree object.
  - The data persistence module takes care of the persistence operation (Create, read, delete and update).
  - Both the XML file parsing module and the data persistence module are being simultaneously used by the Jenkins plugin and the RESTful web service.

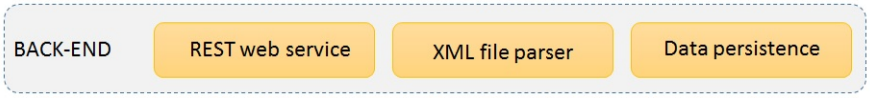


Figure IV.3 – Back end layer

- **Front-end :**
  - This layer is the main interaction point with the users. The dashboard is used to consult the data reports, and the web application is being used (by system administrators) to administer the whole system.
  - Both the dashboard and the web application are directly linked to the database.

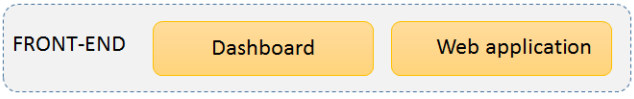


Figure IV.4 – Front end layer

1.2 Package model

The solution has to maintain a high degree of loose coupling between the different modules performing different functions, which justifies the package model illustrated in the following figure.

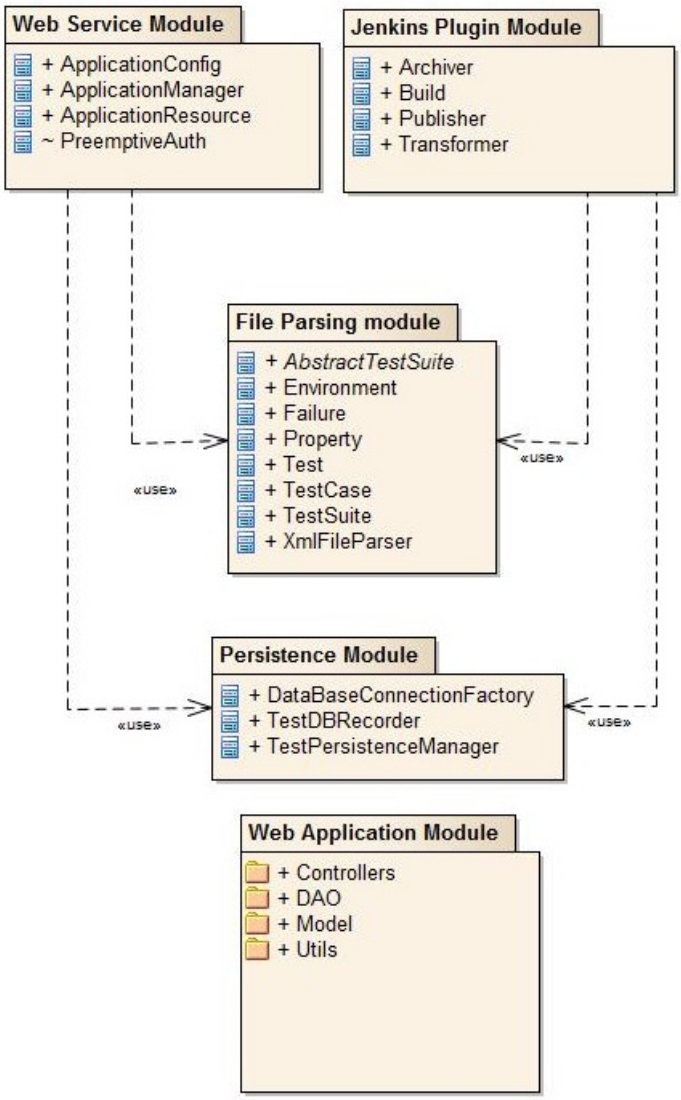


Figure IV.5 – Global packaging model

The solution, as shown in the package model, was split into five different packages : Jenkins plugin module, Web service module, XML file parsing module, Data persistence module and finally the web application module. Each of these modules will be thoroughly described in the domain design section.

1.3 Design patterns

In this section we will present the design patterns used in the design of our solution.
A design pattern is a general reusable solution to a commonly occurring problem within a given context in software design. A design pattern is not a finished design that can be transformed directly into source or machine code. It is a description or template for how to solve a problem that can be used in many different situations. Patterns are formalized best practices that the programmer can use to solve common problems when designing an application or system [38].

There are three groups of design patterns :

- **Creational patterns :** Creational design patterns are design patterns that deal with object creation mechanisms, trying to create objects in a manner suitable to the situation. The basic form of object creation could result in design problems or in added complexity to the design. Creational design patterns solve this problem by somehow controlling this object creation.
- **Structural patterns :** structural design patterns are design patterns that ease the design by identifying a simple way to realize relationships between entities.
- **Behavioral patterns :** behavioral design patterns are design patterns that identify common communication patterns between objects and realize these patterns. By doing so, these patterns increase flexibility in carrying out this communication.

After specifying what design patterns are, we will present the list of the design patterns used in this project :

- **Singleton pattern :**
  The singleton pattern is a design pattern that restricts the instantiation of a class to one object. This is useful when exactly one object is needed to coordinate actions across the system. The concept is sometimes generalized to systems that operate more efficiently when only one object exists, or that restrict the instantiation to a certain number of objects. This pattern is used in the data persistence module, specifically to the Database-ConnectionFactory class. Only one database connection is used when database operations are being performed [38].

- **Adapter pattern :**
  The adapter pattern is a software design pattern that allows the interface of an existing class to be used from another interface. It is often used to make existing classes work with

others without modifying their source code. This pattern is used in the Jenkins plugin module, which was build on another existing plugin called Nunit plugin [38].

- **Composite pattern :**
  The composite pattern describes that a group of objects is to be treated in the same way as a single instance of an object. The intent of a composite is to "compose" objects into tree structures to represent part-whole hierarchies. Implementing the composite pattern lets clients treat individual objects and compositions uniformly. This pattern is used in the XML file parsing module, so as to treat TestCase and TestSuite (a composition of testCases and TestSuites) as a single entity [38].

- **Decorator pattern :**
  The decorator pattern is a design pattern that allows behavior to be added to an individual object, either statically or dynamically, without affecting the behavior of other objects from the same class. This pattern is also used in the XML file parsing module, so as to differentiate between TestCases and TestSuites [38].

- **Iterator pattern :**
  The iterator pattern is a design pattern in which an iterator is used to traverse a container and access the container's elements. This pattern is used to iterate through the different elements of the Test properties (the DOM tree) [38].

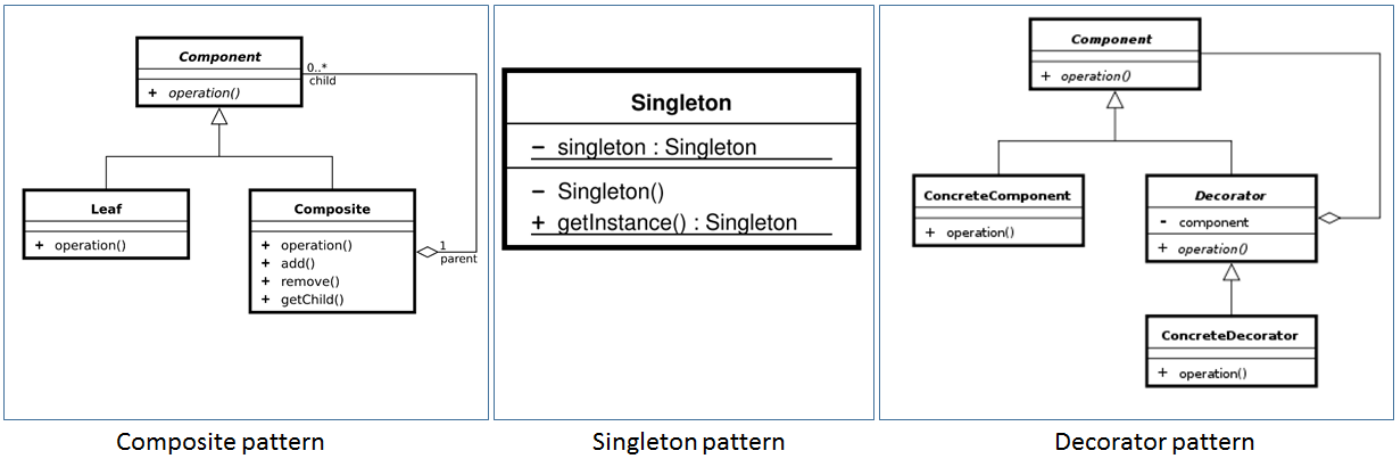


Figure IV.6 – Design patterns

2 Domain design

After specifying the global design, we move on to the domain design, where we specify the class model of each module (package).

2.1 Jenkins plugin class model

The Jenkins plugin module is the first entry point to our solution. It is integrated within the Jenkins instances, and acts as an internal tool to extract data and save it to the database. The following figure is the Jenkins plugin class model.

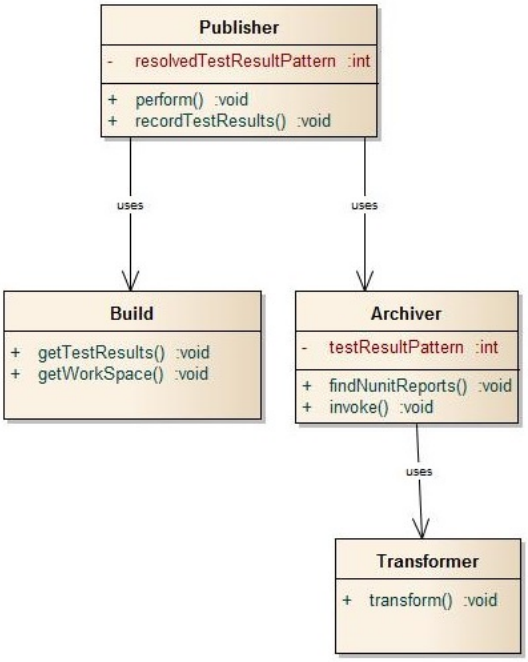


Figure IV.7 – Jenkins plugin class model

- **Description :**
  - The **Publisher** class is the main entry point to the Jenkins plugin. It is, in fact, the only part Jenkins interacts with. It triggers the whole data extraction and saving process.
  - The **Build** class here represents an actual Jenkins job build. It contains all the necessary data, including the Nunit test results.
  - The **Archiver** class is responsible for retrieving the Nunit results from the **Build** object, then triggers the **Transformer**.
  - The **Transformer** class is responsible for any data manipulation. It triggers the XML file parsing.

2.2 Web service class model

The RESTful web service module is the second entry point to our solution. It is called by a Jenkins job, and it mainly communicates with Jenkins and Stash via REST APIs. It also extracts data from Jenkins and Stash, then saves them to the database. The following figure is the web service's class model.

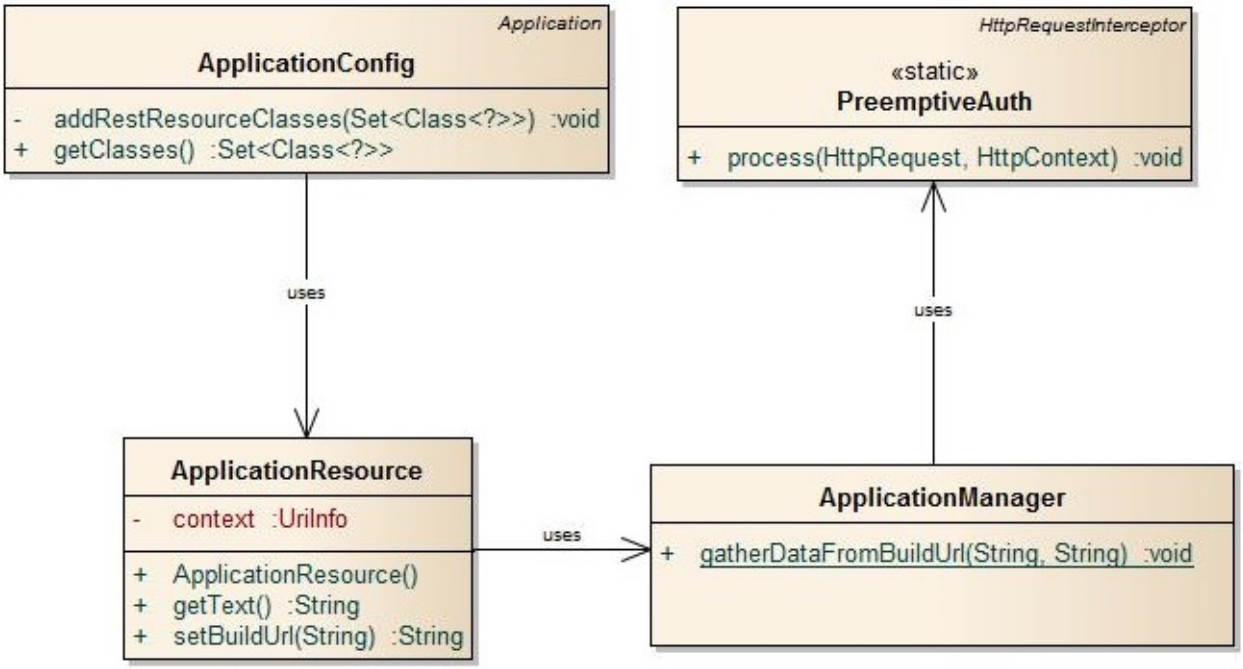


Figure IV.8 – RESTful web service class model

- **Description :**
  - The **ApplicationConfig** class is the responsible for managing the **ApplicationResource** object and making it available through a URI. It represents the housekeeper of the RESTful web service.
  - The **ApplicationResource** class represents the REST resource of the web service. It offers functions that are available through some REST HTTP methods (Get and PUT). It calls the **ApplicationManager** object for logical treatments.
  - The **ApplicationManager** class is responsible for all of the logical treatment, the external modules' calls and the data management of the web service.
  - The **PreemptiveAuth** class is responsible for authenticating the web service into the external platforms (Jenkins and Stash), so as to allow the web service to access their data through REST API.

2.3 XML file parsing class model

The XML file parsing module is the common tool through which the other modules transform XML files into an object tree, giving them the possibility to explore XML elements and properties. It uses the DOM parsing method. The following figure is the XML file parsing class model.

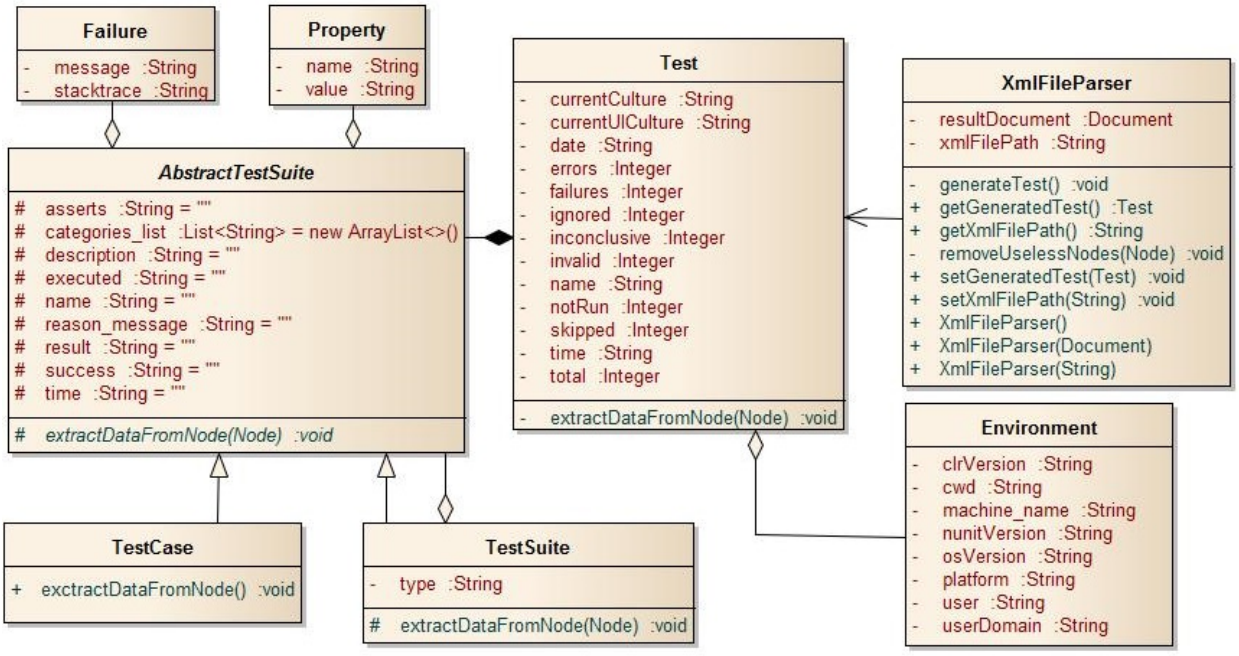


Figure IV.9 – XML file parser class model

- **Description :**
  - The **XMLFileParser** class is responsible for parsing the XML file with the DOM parsing method. It returns a **Test** object. It is the main entry point for this module.
  - The **Test** class is the parent class of the file elements' tree. It represents a single XML file, contains an **Environment** object and a list of **AbstractTestSuite** objects.
  - The **Environment** object contains the environment related properties, such as the Nunit version, the user's name, the machine's name, etc.
  - The **AbstractTestSuite** class is the abstraction of the **TestSuite** class. It is part of the "Composite" design pattern as it allows the other classes to treat **TestSuite** and **TestCase** objects as one. It has a list of **Property** and **Failure** objects.
  - The **TestSuite** class represents an Nunit testsuite. It could contain other **TestSuite** objects or other **TestCase** objects.
  - The **TestCase** class represents an Nunit testcase. It is a leaf in the DOM tree.

2.4 Data persistence class model

The data persistence module is responsible for database operations. All of the other modules, except the web application, have to go through it to access the database. it uses prepared statements for more secure and quicker results.The following figure is the data persistence class model.

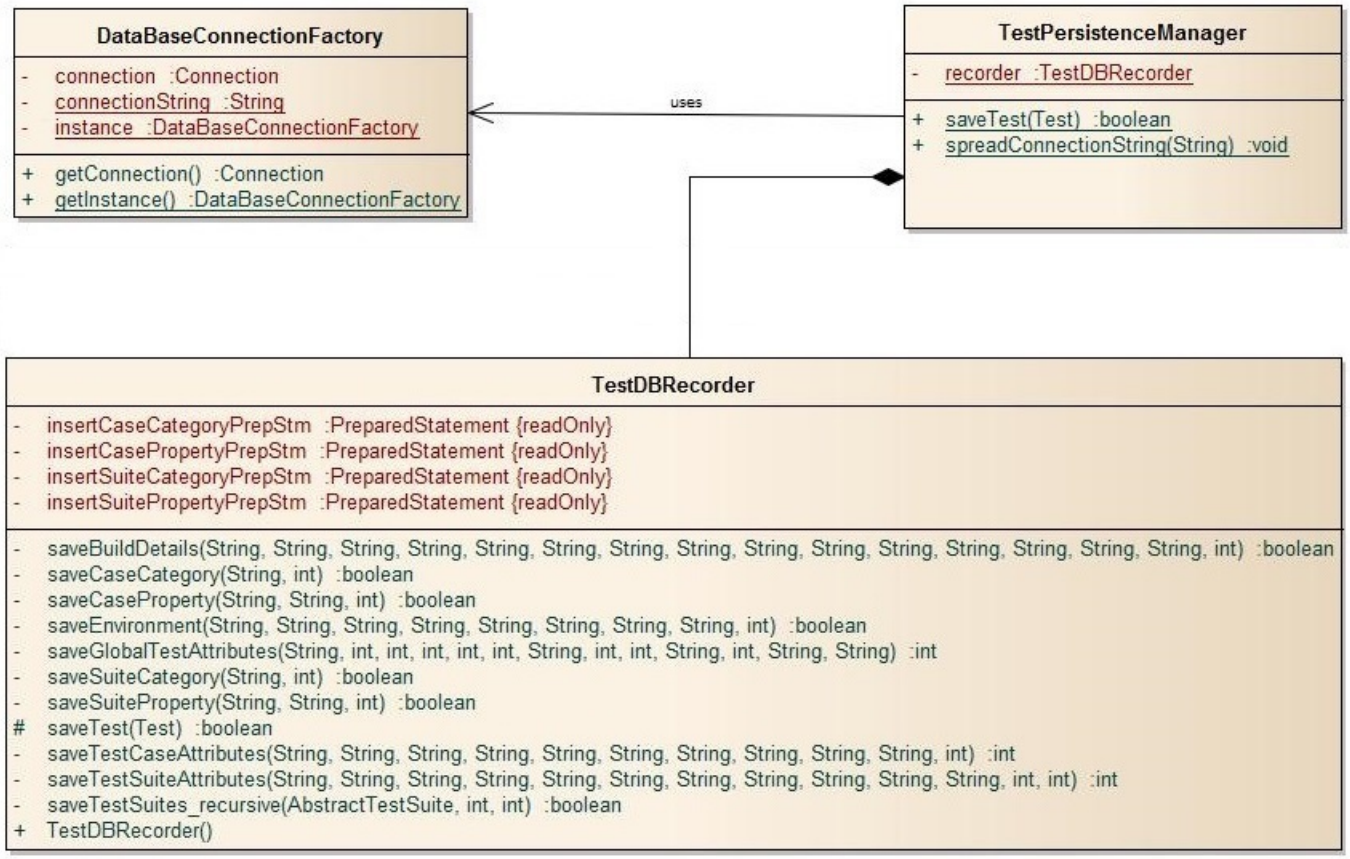


Figure IV.10 – Data persistence class model

- **Description :**
  - The **DatabaseConnectionFactory** class is responsible for providing a database connection. It used the "Singleton" design pattern so as to minimize the use of resources.
  - The **TestPersistenceManage** class is the entry point of this module. It spreads the database connection and triggers the test data saving.
  - The **TestDBRecorder** class is responsible for all of the persistence operations' logic.

2.5 Web application class model

The web application is the administrative tool through which system administrators control the application, along with the database. Their actions are however limited to a certain degree so as to maintain consistency throughout the whole system. This module follows the Model-View-Controller (MVC) pattern.

The MVC pattern is a software architectural pattern for implementing user interfaces. It divides a given software application into three interconnected parts, so as to separate internal representations of information from the ways that information is presented to or accepted from the user. This pattern has become extremely popular for designing web applications.

The central component of MVC, the model, captures the behavior of the application in terms of its problem domain, independent of the user interface. The model directly manages the data, logic and rules of the application. A view can be any output representation of information, such as a form or a diagram. The third part, the controller, accepts input and converts it to commands for the model or view.

At first we will present the global packaging diagram, then we will go in details with each package of the web application. The following figure represents the package model.

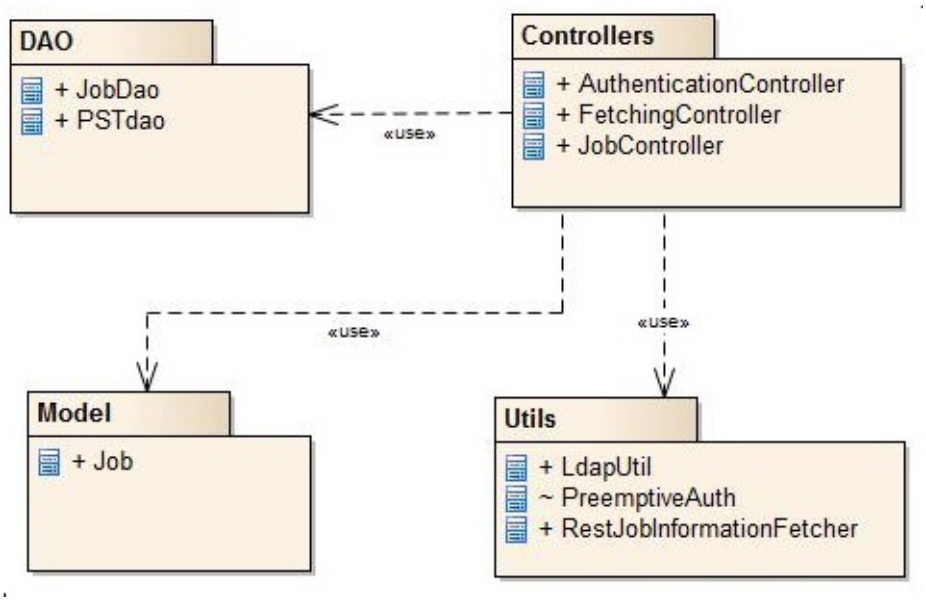


Figure IV.11 – Web application package model

In the following section, we will present each package aside.

2.5.1 Model class model

The Model package represents the model part of the MVC pattern. It stores data that is retrieved according to commands from the controller and displayed in the view. The following figure is the class model.



Figure IV.12 – Model class model

- **Description :**
  - The **Job** class represents the model of the web application.

2.5.2 Controllers class model

The controllers package represents the controller part of the MVC pattern. It sends commands to the model to update the model's state (e.g. submitting a form). It can also send commands to its associated view to change the view's presentation of the model. The following figure is the Controllers class model.

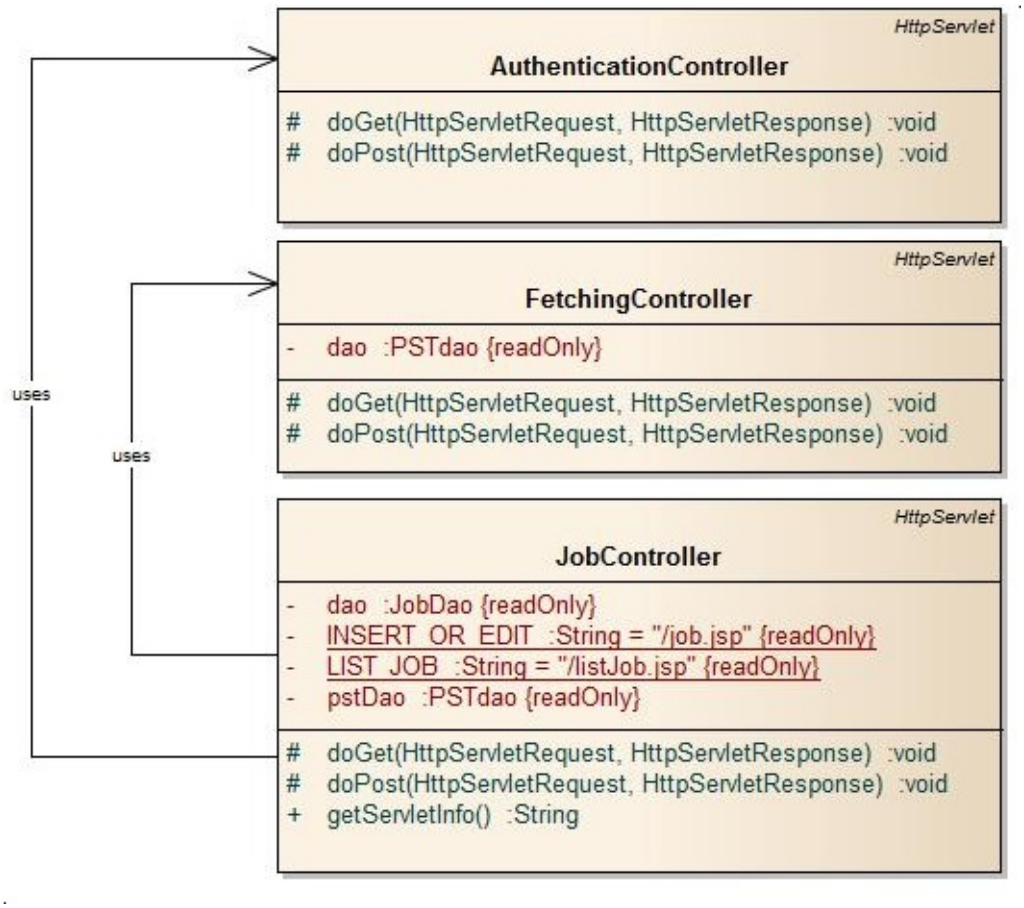


Figure IV.13 – Controllers class model

- **Description :**
  - The **AuthenticationController** class is responsible for authenticating the system administrators whenever they try to access the web application. Any url request for the application is directly investigated by this controller.
  - The **FetchingController** class is responsible for allowing and ordering data extraction from external platforms (Jenkins and Stash) through Stash APIs.
  - The **jobController** class is responsible for managing the **Job** model and most of the views. It specifically controls data creation and data updating.

2.5.3 DAO class model

The DAO (Data Access Object) package is the module responsible for accessing the database from within the web application. It uses the Singleton design pattern to manage database connections. The following figure is the DAO class module.

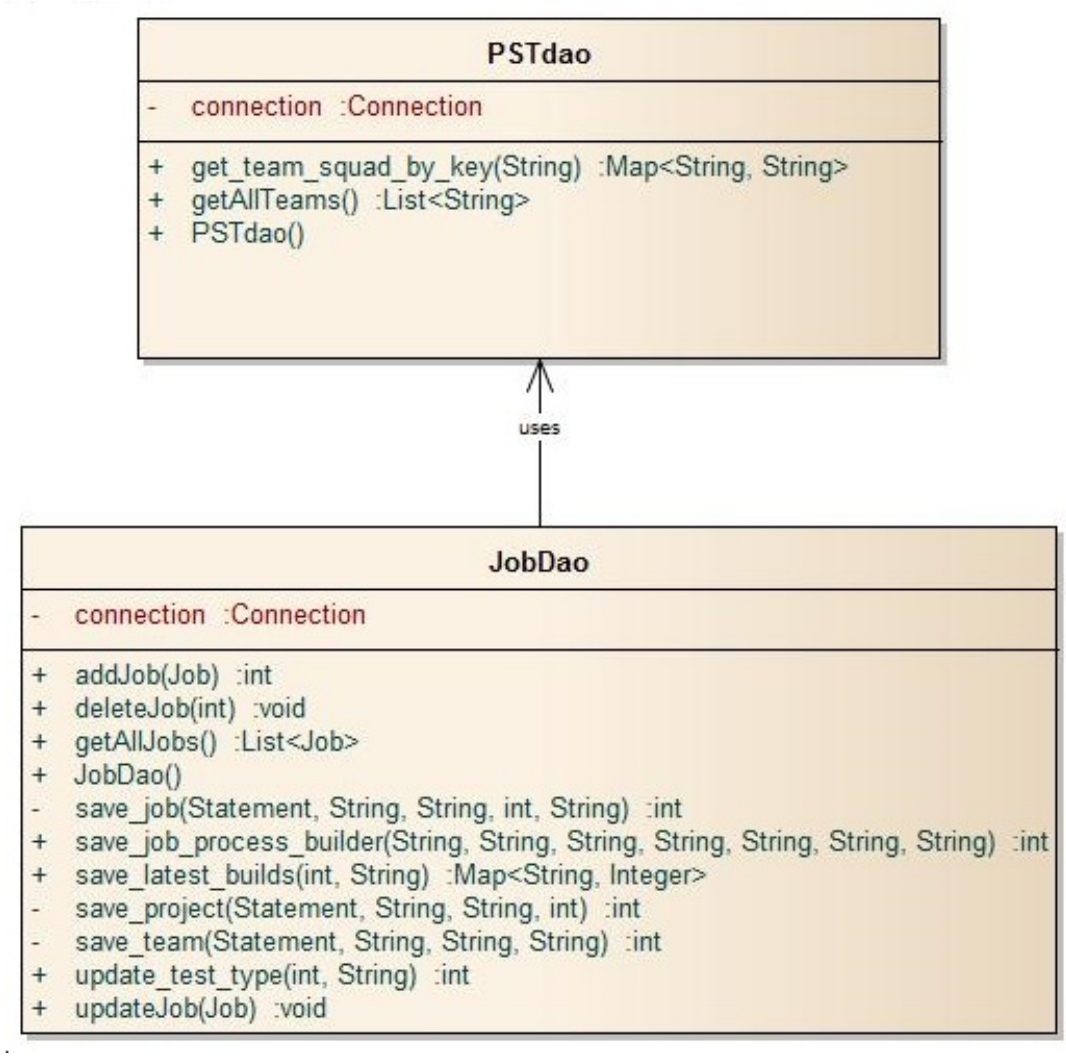


Figure IV.14 – DAO class model

- **Description :**
  - The **PSTdao** class is responsible for data persistence operations that concern projects, squads or teams.
  - the **JobDao** class is responsible for data persistence operations that don't concern projects, squads or teams (e.g. job name, test type).

2.5.4 Utils class model

The Utils package is mainly used for any external interactions between the web application and another platform, like the Active Directory or the Stash platform. The following figure is the Utils class model.

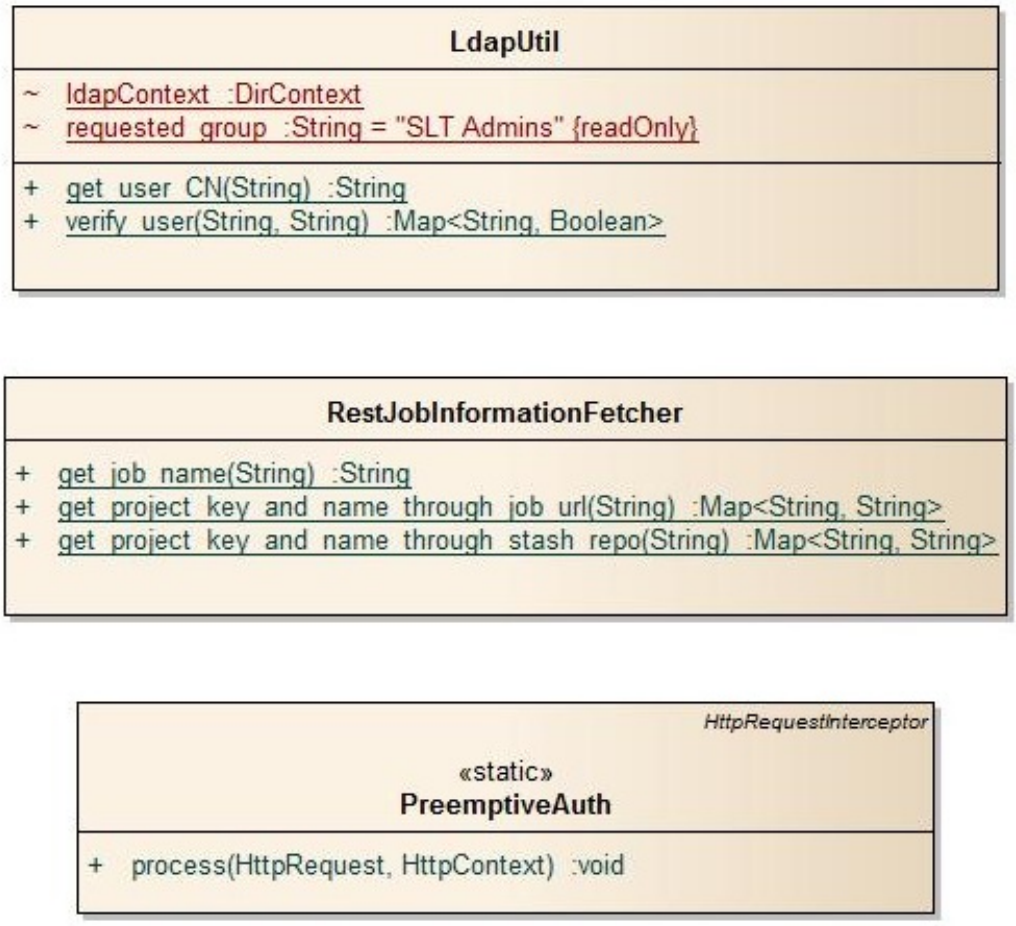


Figure IV.15 – Utils class model

- **Description :**
  - The **LdapUtil** class is responsible for accessing the LDAP Active Directory of Vistaprint and verifying user credentials and groups.
  - The **RestJobInformationFetcher** class is responsible for data extraction from external platforms : Jenkins and Stash.
  - The **PreemptiveAuth** class is responsible for authenticating the web application into the external platforms (Jenkins and Stash), so as to allow the web application to access their data through REST API.

3 Behavioral design

In this section, we will present the major object sequence diagrams. In the next few pages, we will be presenting the object sequence diagrams in the following order :

- **Object sequence diagram : Add a new job**
  The system administrator can add a new Jenkins job to the system in order to be recognized in future procedures and build data updates.

- **Object sequence diagram : Authenticate**
  The system administrator has to authenticate in order to access certain parts of the application. They have to provide valid Active Directory credentials that belong to an Active Directory group called "SLT Admins".

- **Object sequence diagram : Global search through Jenkins**
  The system administrator can perform a global search through Jenkins history to verify if any of the recognized jobs has unsaved builds and unsaved Nunit test results.

- **Object sequence diagram : Save job's build data**
  Jenkins can save a job's build data in the system after each job build. If the job is new to the system, it will be added in the database in order to be recognized in future procedures.

- **Object sequence diagram : Change project, squad and team**
  The system administrator can modify a job's project, squad and team. However, in order to ensure the system's consistency, they can't do so freely. The project and squad have to be matching those found in Stash repository (Git).

- **Object sequence diagram : Change test type**
  The system administrator can update a Jenkins job's test type. The system only recognizes three test types : System (UI) tests, Service tests and Unit tests.

3.1 Object sequence diagram : Add a new job

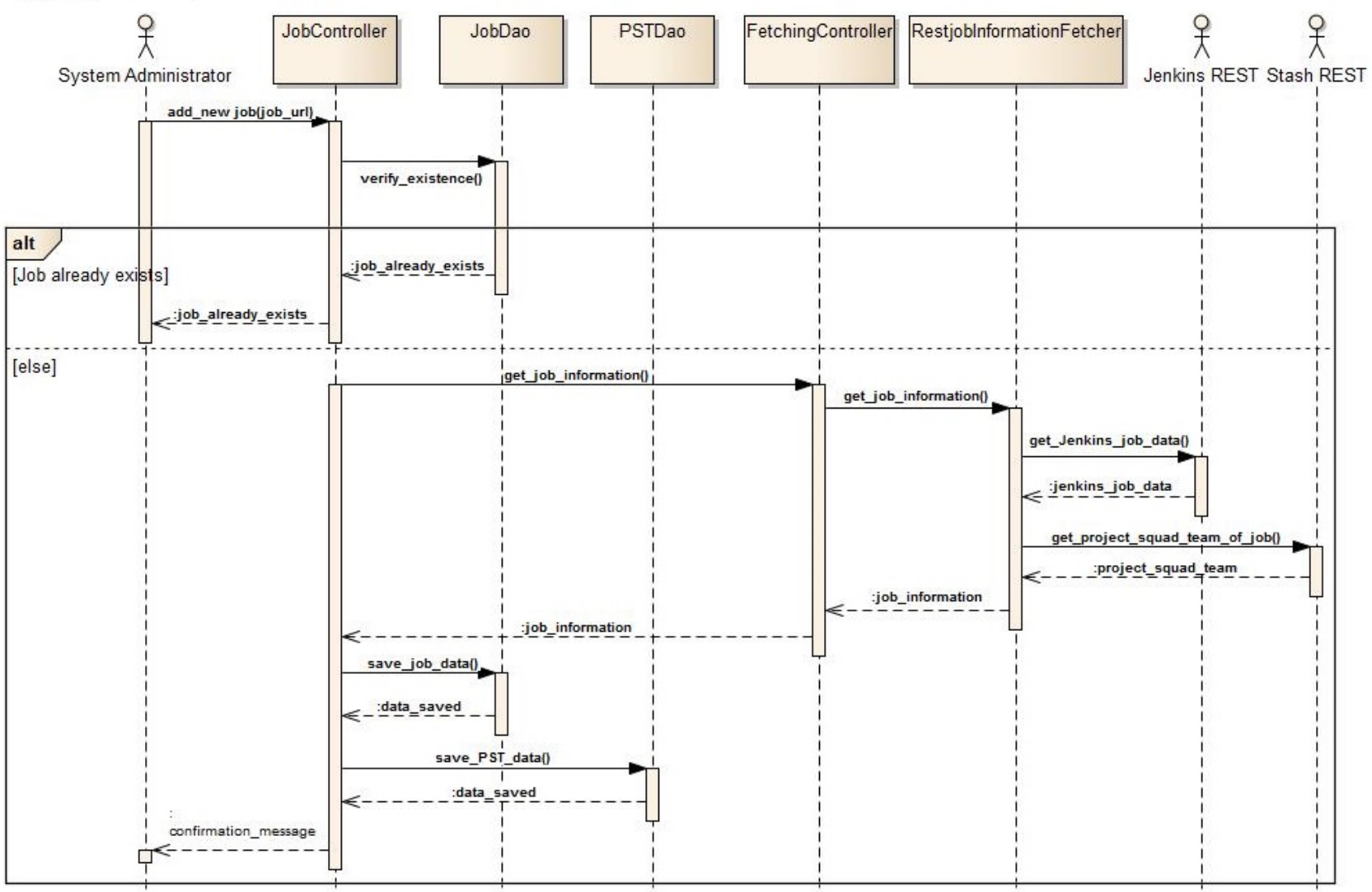


Figure IV.16 – Add new job object sequence diagram

3.2 Object sequence diagram : Authenticate

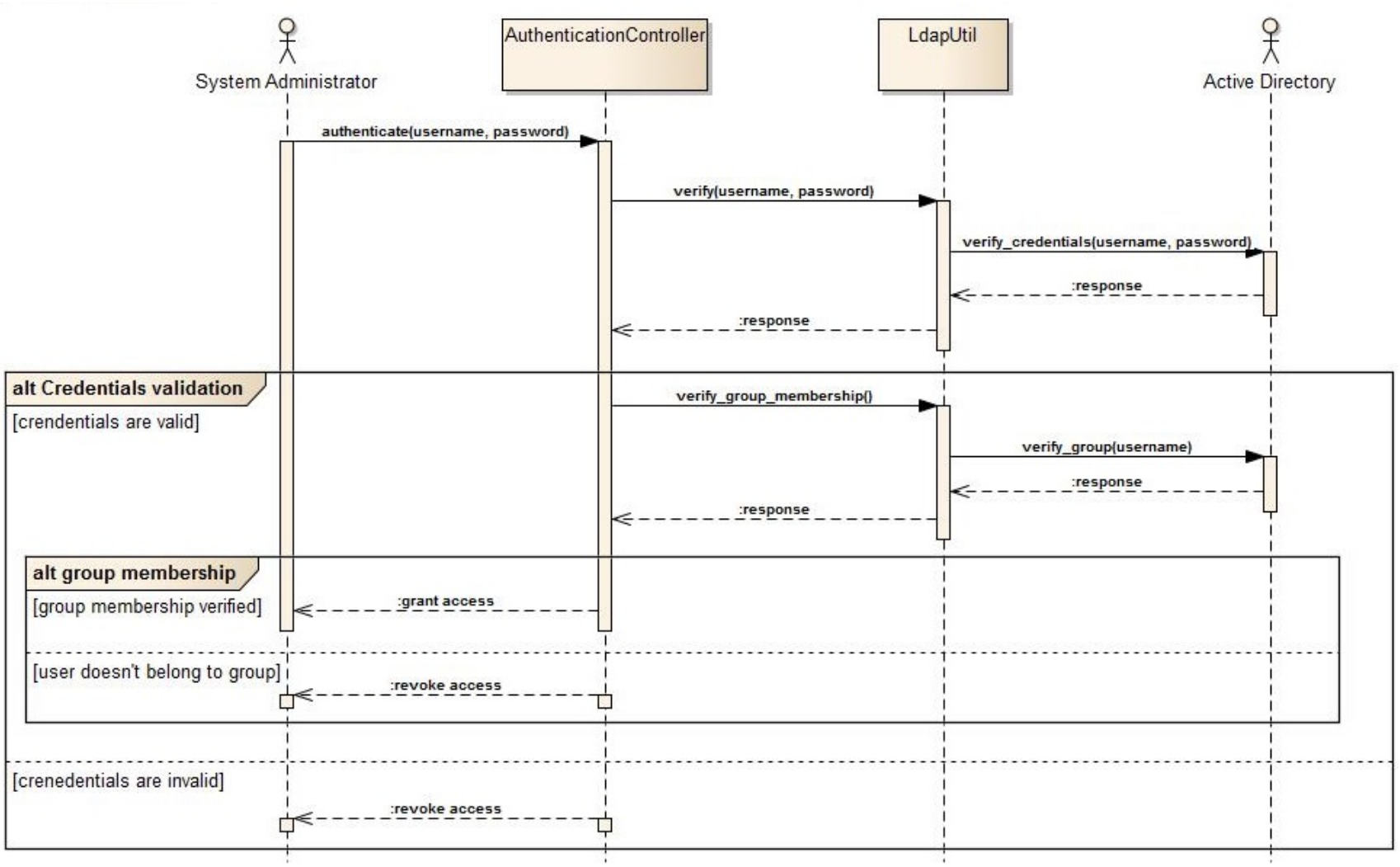


Figure IV.17 – Authenticate object sequence diagram

3.3 Object sequence diagram : Global search through Jenkins

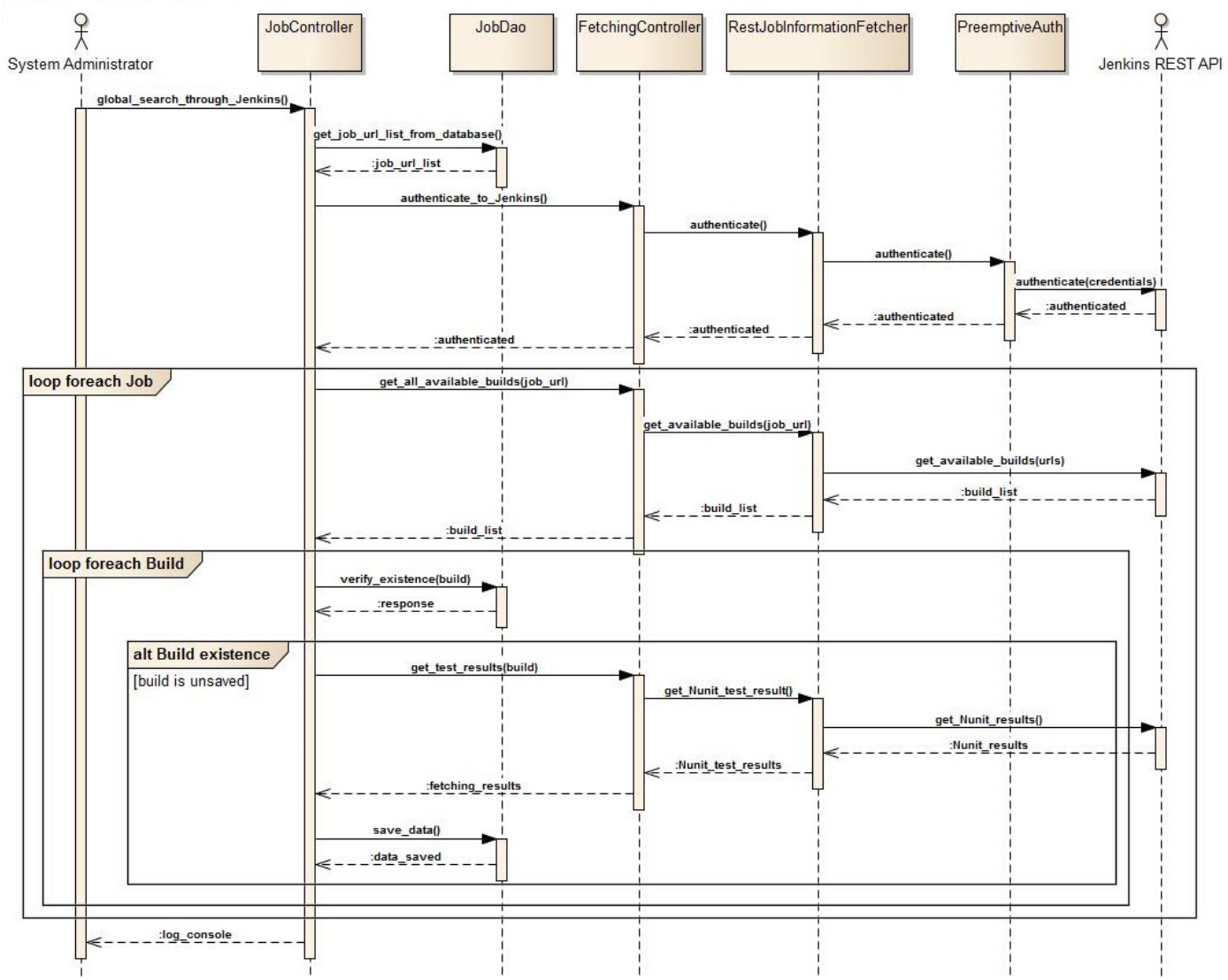


Figure IV.18 – Global search through Jenkins object sequence diagram

3.4 Object sequence diagram : Save job's build data

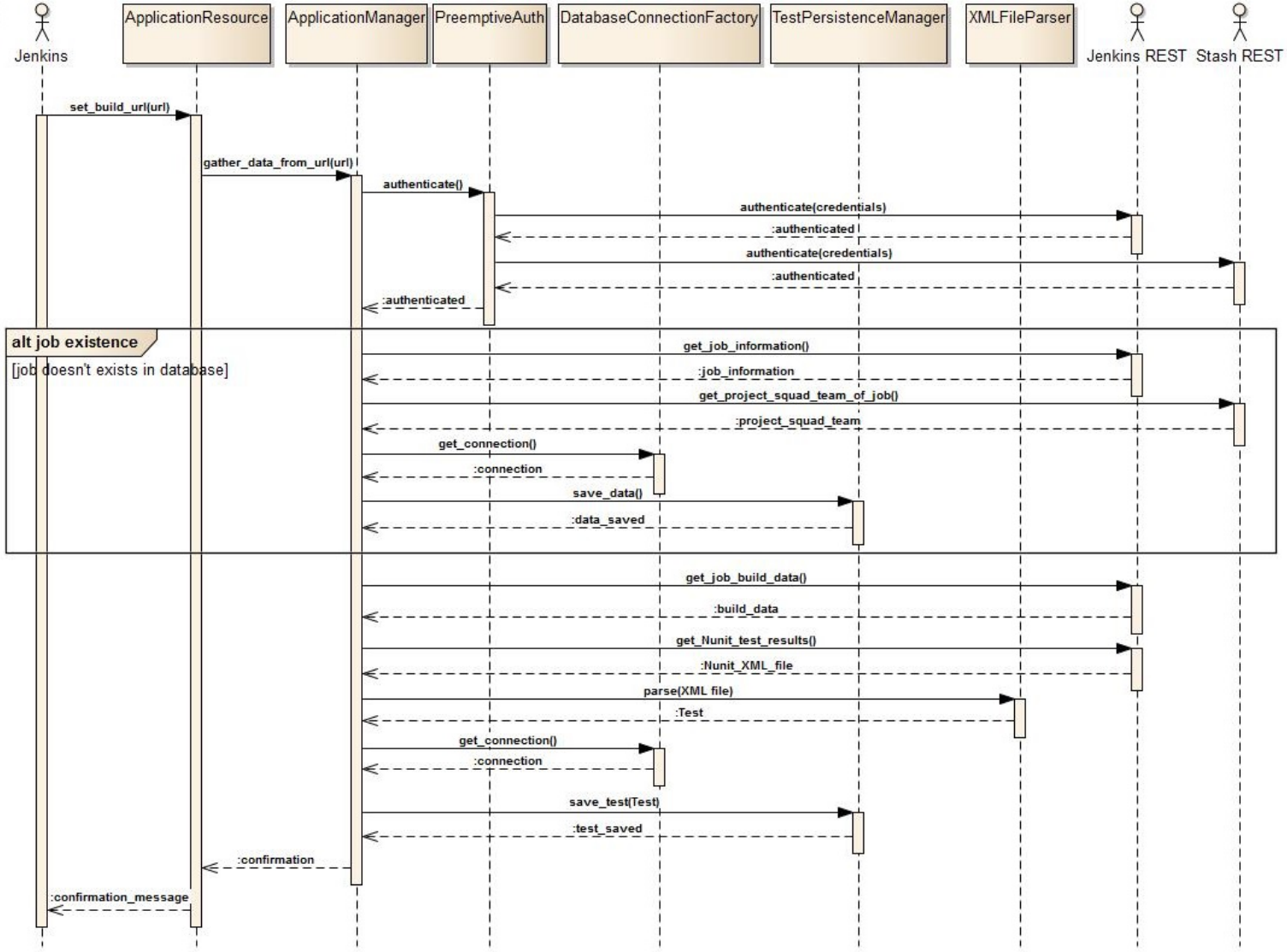


Figure IV.19 – Save build data object sequence diagram

3.5 Object sequence diagram : Change project, squad and team

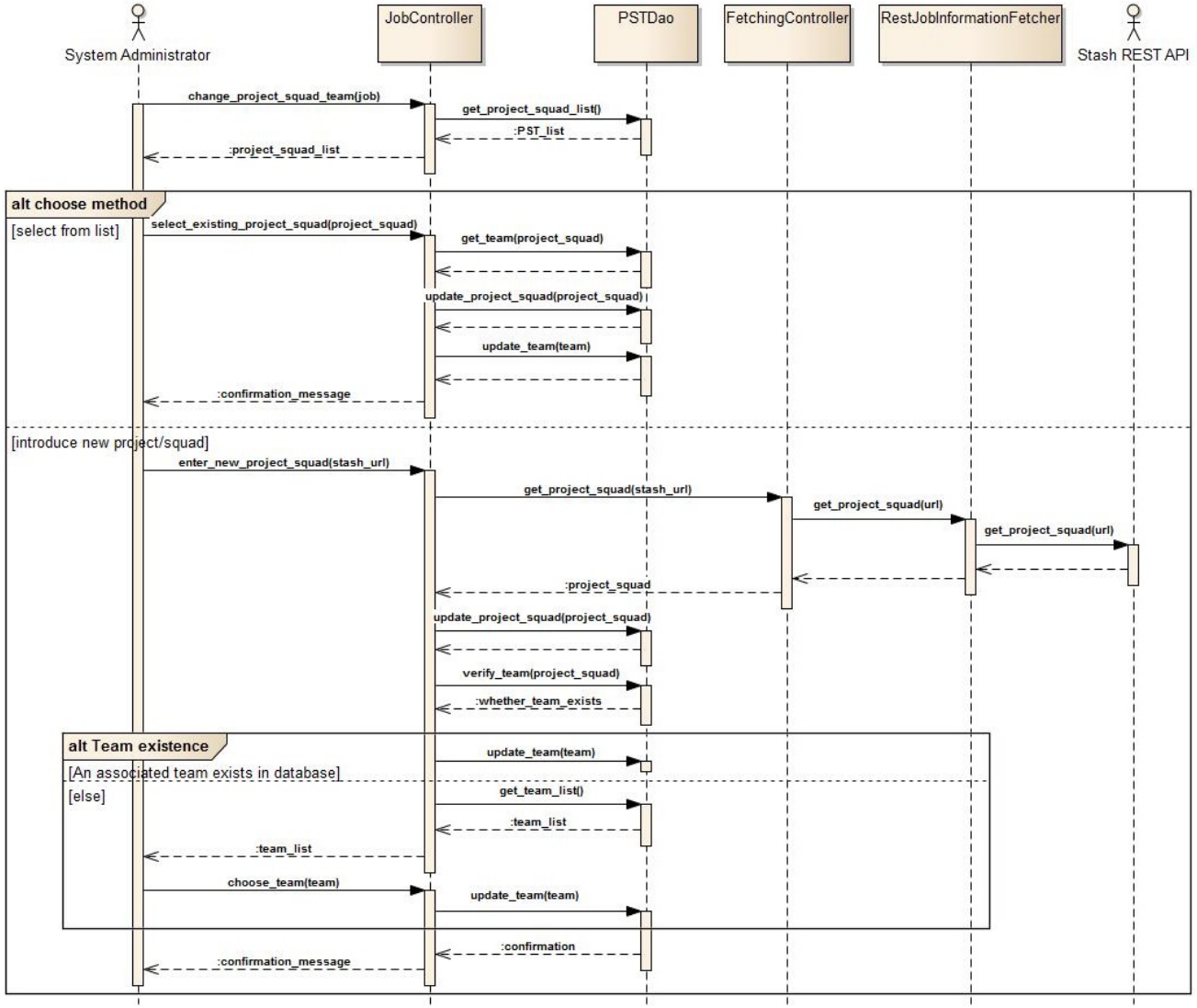


Figure IV.20 – Change project, squad and team object sequence diagram

3.6 Object sequence diagram : Change test type

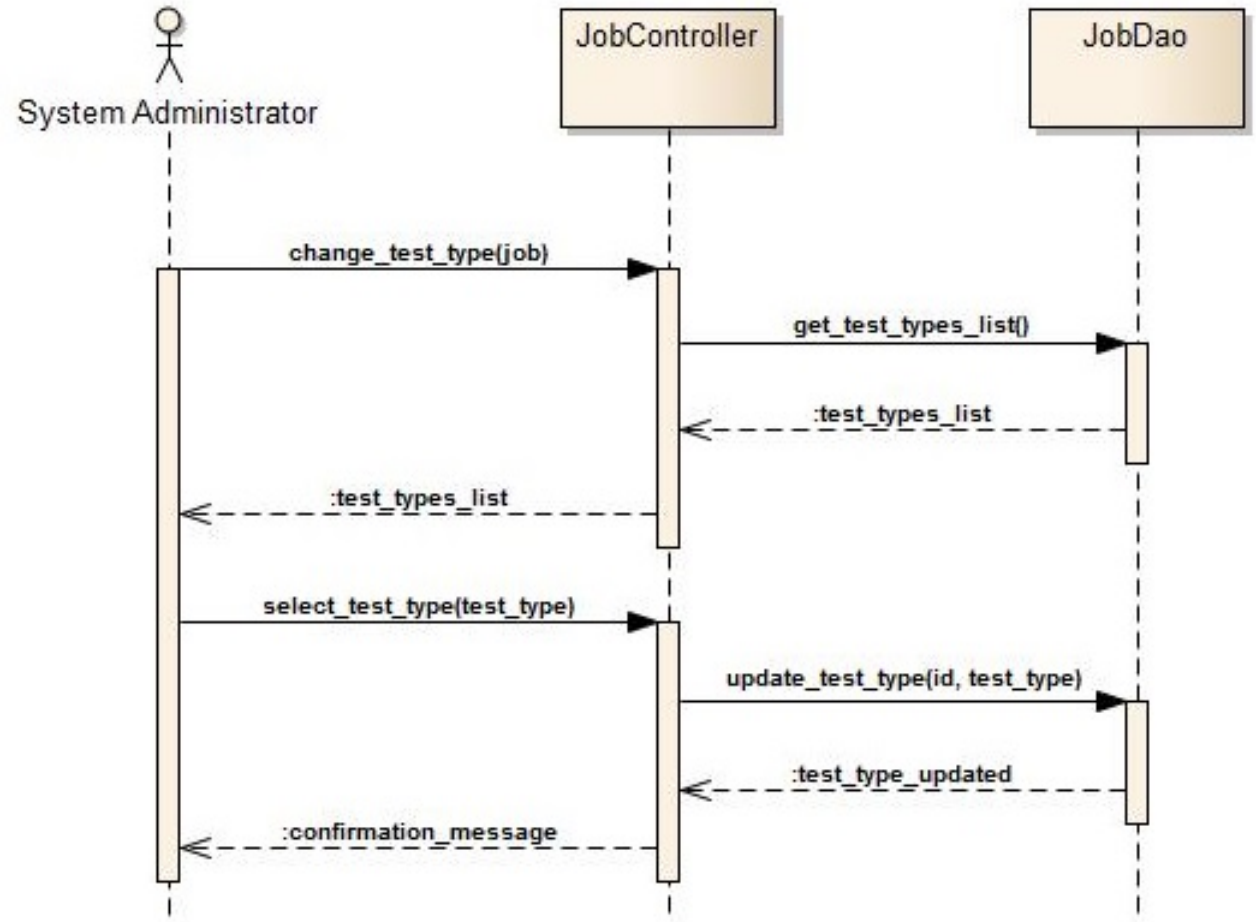


Figure IV.21 – Change test type object sequence diagram

Having detailed the behavioral design through the object sequence diagrams, we now proceed to the database design.

4 Database design

The following figure is the database relational structure. The tables and the relationships will be explained later on.

Database diagram :



Figure IV.22 – Database design

- **Tables :**
  - GlobalTest : It is the central table of the schema. It represents an Nunit results file, and containes global testing metrics (total success, total time,etc).
  - TestCase : It contains the data relative to an Nunit test case.
  - TestSuite : It contains the data relative to an Nunit test suite.
  - Environment,Property and Category : They contain the elements' properties found in the results file.
  - Build : It contains data relative to a Jenkins job's build.
  - Job : It contains data relative to a Jenkins job.
  - Project,Team : They contain data relative to projects, squads and teams.

- **Relationships :**
  - Each TestSuite belongs to either another TestSuite or a GlobalTest.
  - Each GlobalTest belongs to a Build.
  - Each Build belongs to a Job.
  - Each Job belongs to a Project.
  - Each Project belongs to a Team/Squad combination.

Conclusion

After presenting the overall solution design, we move on to presenting the realized achievements. The next chapter will be about those achievements, along with the used implementation tools and the solution's technical architecture.

Chapter V

Achievements

Plan

1 Software environment 77

1.1 Design tool : Enterprise Architect (EA) 77

1.2 Implementation tools 77

1.3 Continuous integration tools 79

1.4 Technologies and Frameworks 80

2 Technical architecture 81

3 Implementation achievements 82

3.1 Dashboard achievement 82

3.2 Web service achievement 88

3.3 Web application achievement 89

4 Continuous integration 93

4.1 Jenkins job 93

4.2 Stash repository 94

4.3 Ant script 95

4.4 Unit tests 95

5 Project documentation 96

Introduction

In this section, we will present the project related achievements we have realized throughout the internship. More specifically, we will present the software environment and tools, the implementation achievements and screenshots, as well as the continuous integration phase of the project's lifecycle.

1 Software environment

In this section we will present the different tools and technologies that were used in the project's development.

1.1 Design tool : Enterprise Architect (EA)

Enterprise Architect is a well-defined practice for conducting project analysis, design, planning, and implementation, for the successful development and execution of a defined strategy. This tool that was used for designing the different use case, sequence and class diagrams [39]. Enterprise Architect uses the UML modeling language.

UML stands for Unified Modeling Language. The Unified Modeling Language is a general-purpose modeling language in the field of software engineering, which is designed to offer a way to visualize a system's architectural blueprints in a diagram [40].

1.2 Implementation tools

In this section we specify the tools we used to develop the solution.

1.2.1 Netbeans IDE

NetBeans is a software development platform written in Java. The NetBeans Platform allows applications to be developed from a set of modular software components called modules. This tool was used for implementing the RESTful web service and the web application [41].



Figure V.1 – Netbeans IDE logo

1.2.2 Eclipse IDE with Maven plugin

Eclipse is an integrated development environment. It contains a base workspace and an extensible plug-in system for customizing the environment [42].

Maven is a build automation tool used primarily for Java projects. It addresses two aspects of building software : First, it describes how software is built, and second, it describes its dependencies [43]. This tool was used while developing the Jenkins plugin (the plugin skeleton is built with Maven).



Figure V.2 – Eclipse IDE logo

1.2.3 QlikView

QlikView is one of the most flexible Business Intelligence platforms for turning data into knowledge. It brings a whole new level of analysis, insight, and value to existing data stores with user interfaces that are clean, simple, and straightforward. QlikView was used for the Dashboard development [28].



Figure V.3 – QlikView logo

1.2.4 SQL Server Management Studio

SQL Server Management Studio (SSMS) is an integrated environment for accessing, configuring, managing, administering, and developing all components of SQL Server. SSMS combines a broad group of graphical tools with a number of rich script editors to provide access to SQL Server to developers and administrators of all skill levels [44]. This tool was used for database design and development.



Figure V.4 – Sql Server Management Studio logo

1.3 Continuous integration tools

The following tools were used for our project's continuous integration, which consists of merging, building, testing and deploying the source files after each code change.

1.3.1 Jenkins

Jenkins is an application that monitors executions of repeated jobs. It provides a continuous integration system, making it easier for developers to integrate changes to projects [16].



Figure V.5 – Jenkins logo

1.3.2 Stash

Stash is Git repository software. Git is a distributed revision control system with an emphasis on speed and data integrity [21].



Figure V.6 – Stash logo

1.3.3 Apache ant

Apache Ant is a Java library whose mission is to drive processes described in build files so as to build, test and deploy Java applications [45].



Figure V.7 – Apache Ant logo

1.3.4 Junit

JUnit is a unit testing framework for the Java programming language.



Figure V.8 – Junit logo

1.4 Technologies and Frameworks

1.4.1 J2EE

Short for Java 2 Platform Enterprise Edition. J2EE is a platform-independent, Java-centric environment from Sun for developing, building and deploying Web-based enterprise applications online. The J2EE platform consists of a set of services, APIs, and protocols that provide the functionality for developing multitiered, Web-based applications [46].
This technology was used for developing the web service and the web application.

1.4.2 JAVAX-WS

The Java API for XML Web Services (JAX-WS) is a Java programming language API for creating web services. JAX-WS is one of the Java XML programming APIs. It is part of the J2EE platform [47].
This API was used for developing the RESTful web service.

1.4.3 Twitter bootstrap

Bootstrap is a popular HTML, CSS, and JS framework for developing responsive projects on the web. It makes front-end web development faster and easier [48].
This framework is used for the web application's user interface.

1.4.4 JQuery

jQuery is a cross-platform JavaScript library designed to simplify the client-side scripting of HTML. jQuery's syntax is designed to make it easier to navigate a document, select DOM elements, create animations, handle events, and develop Ajax applications [49].
This library was used for the web application's user interface.

1.4.5 AJAX

Ajax, short for asynchronous JavaScript and XML, is a group of interrelated Web development techniques used on the client-side to create asynchronous Web applications. With Ajax, web applications can send data to and retrieve from a server asynchronously (in the background) without interfering with the display and behavior of the existing page [50].
Ajax was used for the web application's user interface.

2 Technical architecture

The following figure presents the solution's technical architecture. It specifies the deployment specification of each module.

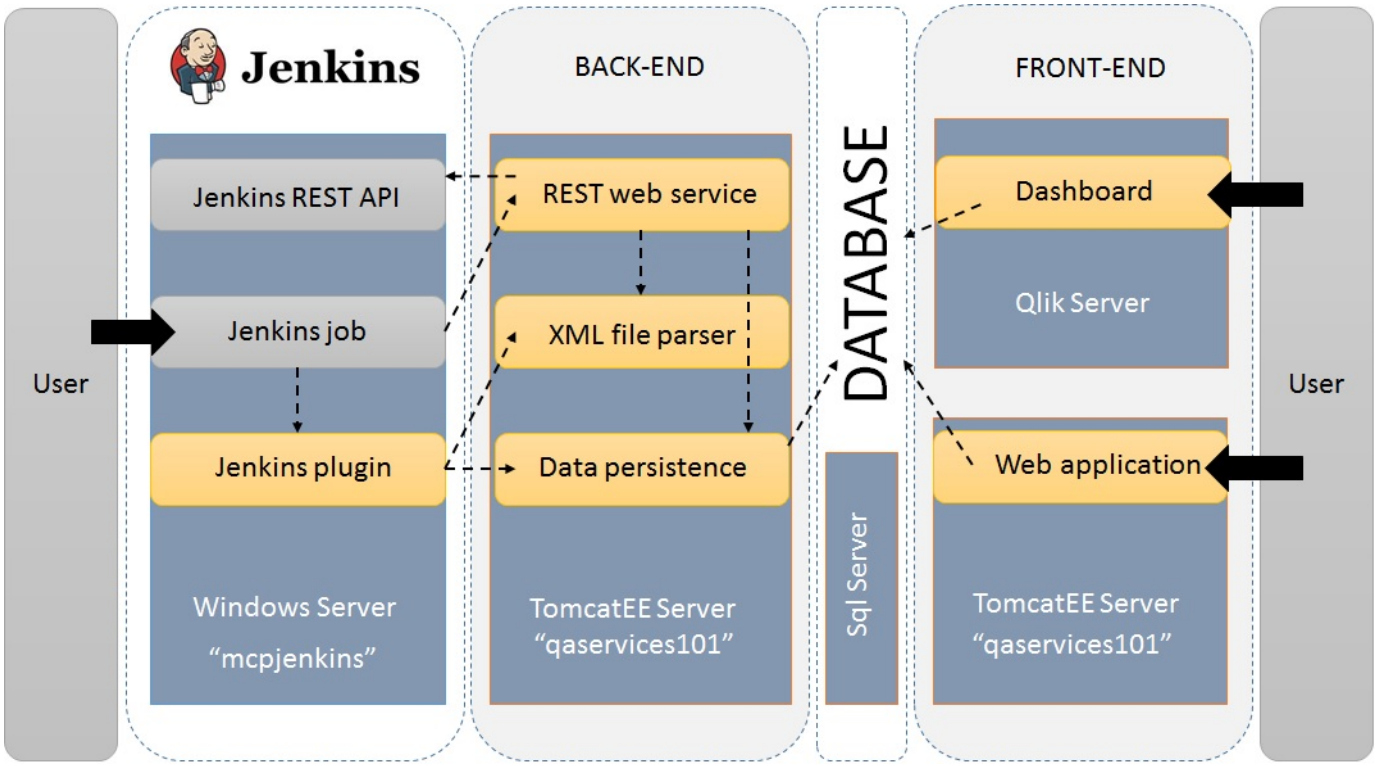


Figure V.9 – Technical architecture

- **Description :**
  - All of the inter-modular communications use the **HTTP** protocol. Data formats are either XML or Json.
  - The **Jenkins plugin** belongs to the "mcpjenkins" instance of Jenkins, which is deployed on a Windows server.
  - The **REST web service**, the **XML file parser**, the **Data persistence** and the **Web application** are deployed on a separate TomcatEE server (Application server).
  - The **Database** is deployed on a Windows Sql Server.
  - The **Dashboard** is deployed on a special Qlik Server (A server designed by Qlik to host QlikView dashboards).

3 Implementation achievements

This section includes all the implementation achievements realized throughout the project.

3.1 Dashboard achievement

In this section we will present the implemented QlikView dashboard. We will start with defining the dashboard's main KPI, then we will present a global view and specify the dashboard filters and graphs.

3.1.1 Velocity KPI

The graphs presented in the dashboard are all relative to the velocity KPI (Key performance Indicator). By definition, velocity is the speed of something in a given direction. In this case, we are measuring the speed of the test automation teams. In this meaning, we can say that the velocity KPI measures the test creation and execution speed. In other words, it tells how many tests a team has created, and how many builds of that team's Jenkins jobs have been built, by comparing the values of each month.

As an example, let's say a team has two Jenkins jobs : job X and job Y. Now let's suppose that in January, job X had 100 tests executed over 10 builds, and job Y had 100 tests over 2 builds. In February, job X had 150 tests executed over 15 builds, and job Y had 200 tests over 5 builds. If we calculate the velocity KPI for February, it would be :

$$Velocity = February_Avg - January_Avg = [(150/15) + (200/5)] - (100/10) + (100/5)] = 20$$

3.1.2 General dashboard view

The graphs featured in the dashboard are :
- Velocity per month graph
- Velocity per Service (Squad) and per test type graphs
- Average tests per build per graph
- Total tests per month and total builds per month graphs
- A details table that contains additional KPI information.

The following figure illustrates the global dashboard. Each element of this dashboard will be individually presented later on in this section.

General dashboard view :



Figure V.10 – Global velocity dashboard

3.1.3 Dashboard filters

The following figure presents the dashboard filters.



Figure V.11 – Velocity dashboard filters

- On the left pane we find the Team, Functional team and service layer. First we have to mention that a Functional team is a naming convention for squads, and service is a naming convention for projects. These filters help filter the graphs' data according to these fields.
- On the middle pane we find the test type filter. We have of course three test types, and an "undefined" element for when a user doesn't specify the test type.
- On the right pane we find the date (year and month) filters.

3.1.4 Dashboard primary graphs

The primary graphs are the graphs that directly translate the velocity KPI values. We have three primary graphs : global velocity, velocity per service (project), and velocity per test type.

These graphs show a variation of the velocity value for each month of the year, depending on the filtering values. The first graph shows the plain velocity. The second shows velocity filtered by service, and the third shows velocity filtered by test type.

- **Global velocity graph :**

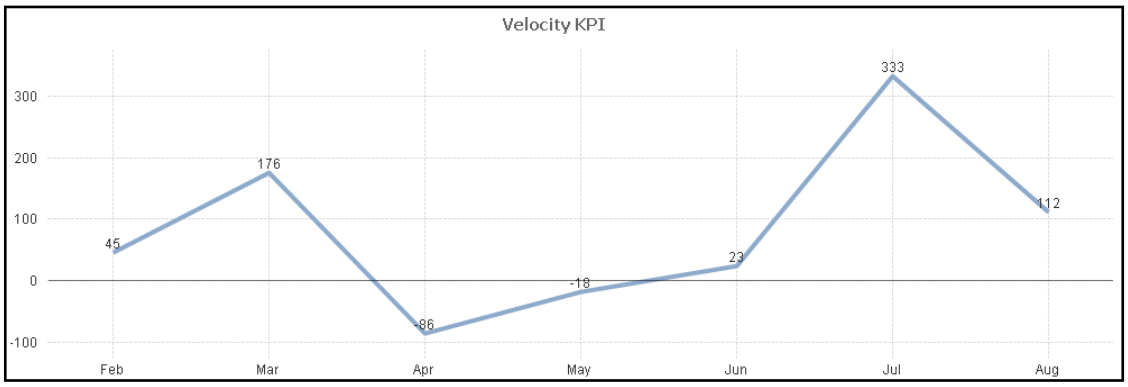


Figure V.12 – Velocity KPI graph

- **Velocity per service graph :**

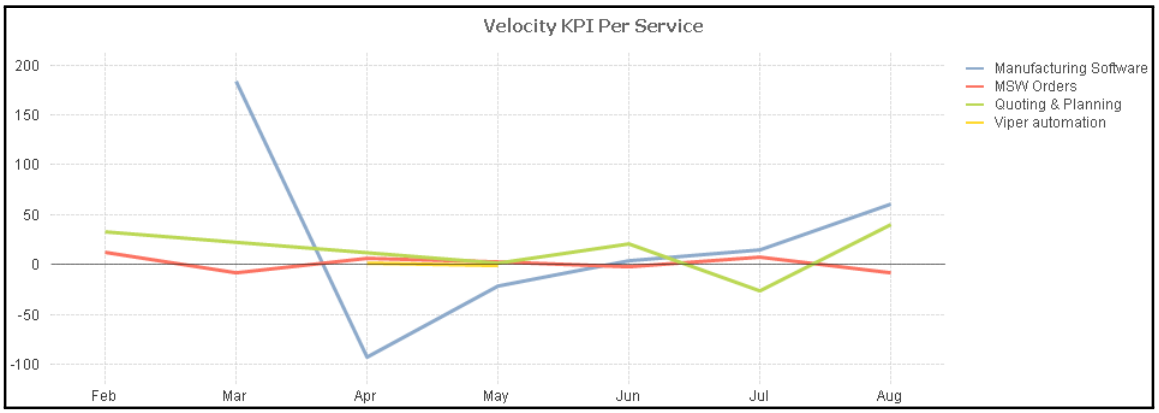


Figure V.13 – Velocity per service KPI graph

- **Velocity per test type graph :**

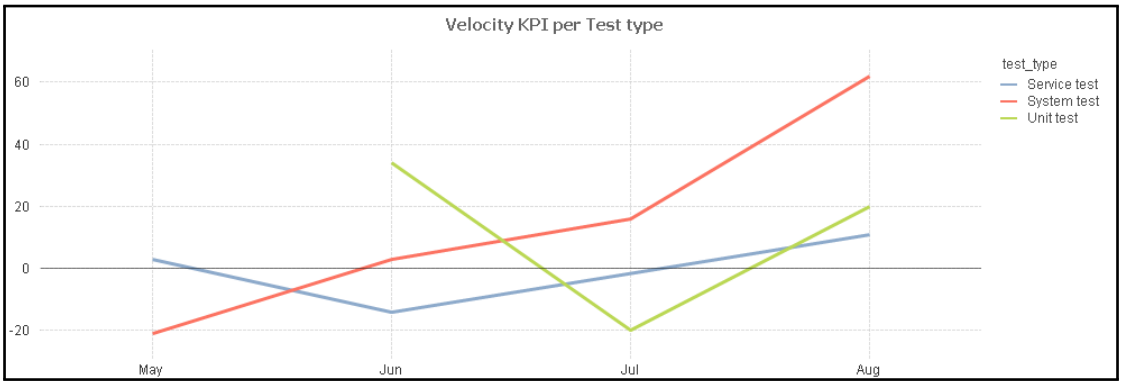


Figure V.14 – Velocity per test type KPI graph

3.1.5 Dashboard secondary graphs

The secondary graphs, unlike the primary graphs, do not directly translate the velocity KPI values. They provide additional information that is related to velocity, and helps the user to have a more global view over the generated data. These graphs are illustrated in the following figures.

- **Average of tests per build graph :**
  This graph shows the average of tests per build for each month of the year. In other words, it shows how many tests are being executed for each job build.

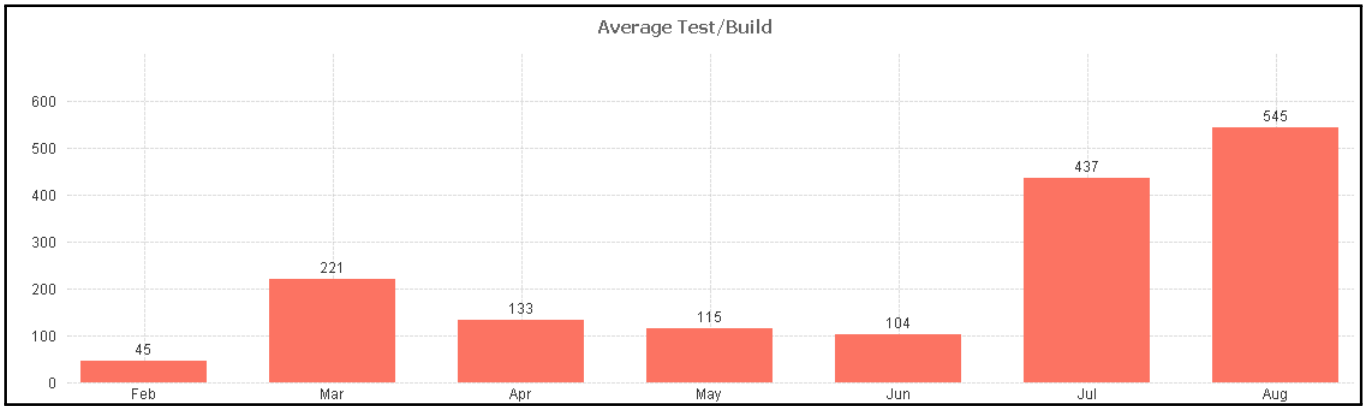


Figure V.15 – Tests per build graph

- **Total builds per month graph :**
  This graph simply shows the total number of builds that are being performed each month of the year.

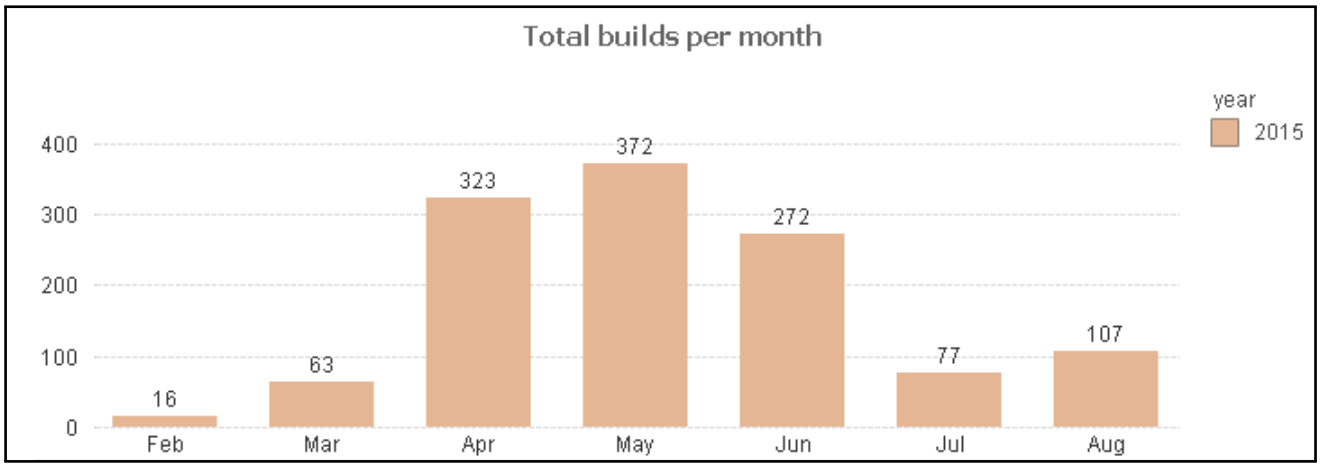


Figure V.16 – Total builds per month graph

- **Total tests per month graph :**
  This graph simply shows the total number of tests that are being executed each month of the year.

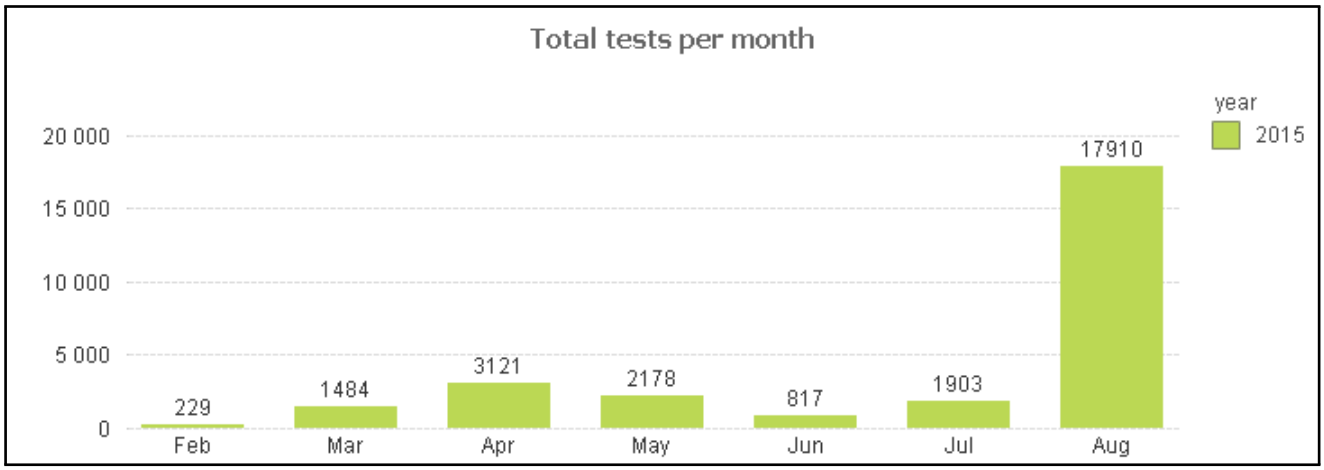


Figure V.17 – Total tests per month graph

- **Details table :**
  This table shows the details of all the items that are being taken into consideration by the different graphs. It lists the relative Jenkins jobs (Test automation job), their teams, their squads, their projects (System under test), as well as other statistical data that is listed by month and year.

Team	Squad	System under test	Test automation job	year	month	Builds	Tests	Average	Velocity
Logistics	Manufacturing Software Logistics ...	Shipping System	VP.ShippingSystem.acceptance	2015	Jul	2	674	337	337
					Aug	48	17047	356	19
	Quoting & Planning	Delivery Options	FD.DeliveryOptions.run_servicetests_and_promote	2015	Feb	2	66	33	33
					Mar	3	99	33	0
					Apr	1	33	33	0
					May	4	135	34	1
					Jun	5	102	21	-13
					Jul	5	68	14	-7
					Aug	4	136	34	20
		DeliveryOptions	ARTIFACT_VERSION	2015	Jun	1	34	34	34
					Jul	5	68	14	-20
					Aug	4	136	34	20
MSW-WIDE	Manufacturing Software QE	QE tests	FD.Quality.NonUITests	2015	Mar	12	440	37	37
					Apr	30	948	32	-5
					May	23	767	34	2
					Aug	1	15	15	15
			FD.Quality.Performance	2015	Mar	9	36	4	4
					Apr	36	100	3	-1
					May	13	36	3	0
					Jun	2	8	4	1
					Jul	3	8	3	-1
					Aug	4	8	2	-1
			FD.Quality.UITests	2015	Mar	6	768	128	128
					Apr	32	1504	47	-81
					May	7	192	28	-19

Figure V.18 – Velocity details table

3.2 Web service achievement

In this section we will present an example of calling our RESTful web service GET method, and we will show how to invoke the service from a Jenkins job.

3.2.1 Deploying the RESTful web service

The RESTful web service is deployed on "qaservices101.vistaprint.net" which is a Tomcat server. The following figure shows the response of the web service to a GET method call.



Figure V.19 – The deployed web service : GET method

3.2.2 Invoking the RESTful web service

In order to invoke the web service from within Jenkins, we make use of Windows PowerShell, which is an internal Jenkins plugin that executes PowerShell scripts. In the Jenkins job, we configure the script to be a build step. In the following figure we present the PowerShell command to provide to the Jenkins PowerShell plugin.



Figure V.20 – Invoking the RESTful web service with PowerShell

3.3 Web application achievement

The web application is deployed on a Tomcat server. It has many views that will be individually presented in this section.

3.3.1 Authentication view

The authentication view is where the system administrator authenticates themselves by providing a username and a password. This is the point where the web application checks the Active Directory about the user's credentials. They have to introduce a valid username, a correct password, and a subscription to the the group "SLT Admins" (Active Directory group). The following figure present the authentication form to be filled.

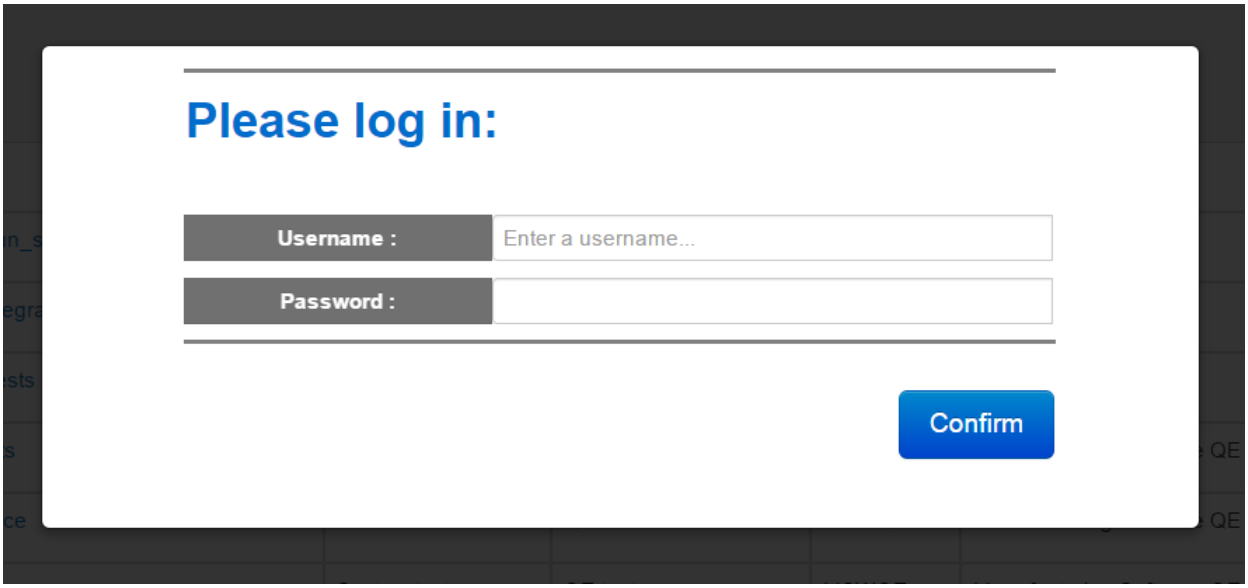


Figure V.21 – Authentication view

There are three main reasons for which an authentication could reject access to the web application :

- An incomplete form (missing username or password).
- A wrong username or a wrong password.
- The non subscription of the user to the admins group (SLT Admins).

3.3.2 Main view

The main view is the core page of the web application. In the following figures we will present concrete examples of use cases.

- **Main page :**
  The following figure presents the main page of the web application. It is basically a list of the Jenkins jobs that are recognized by the system.

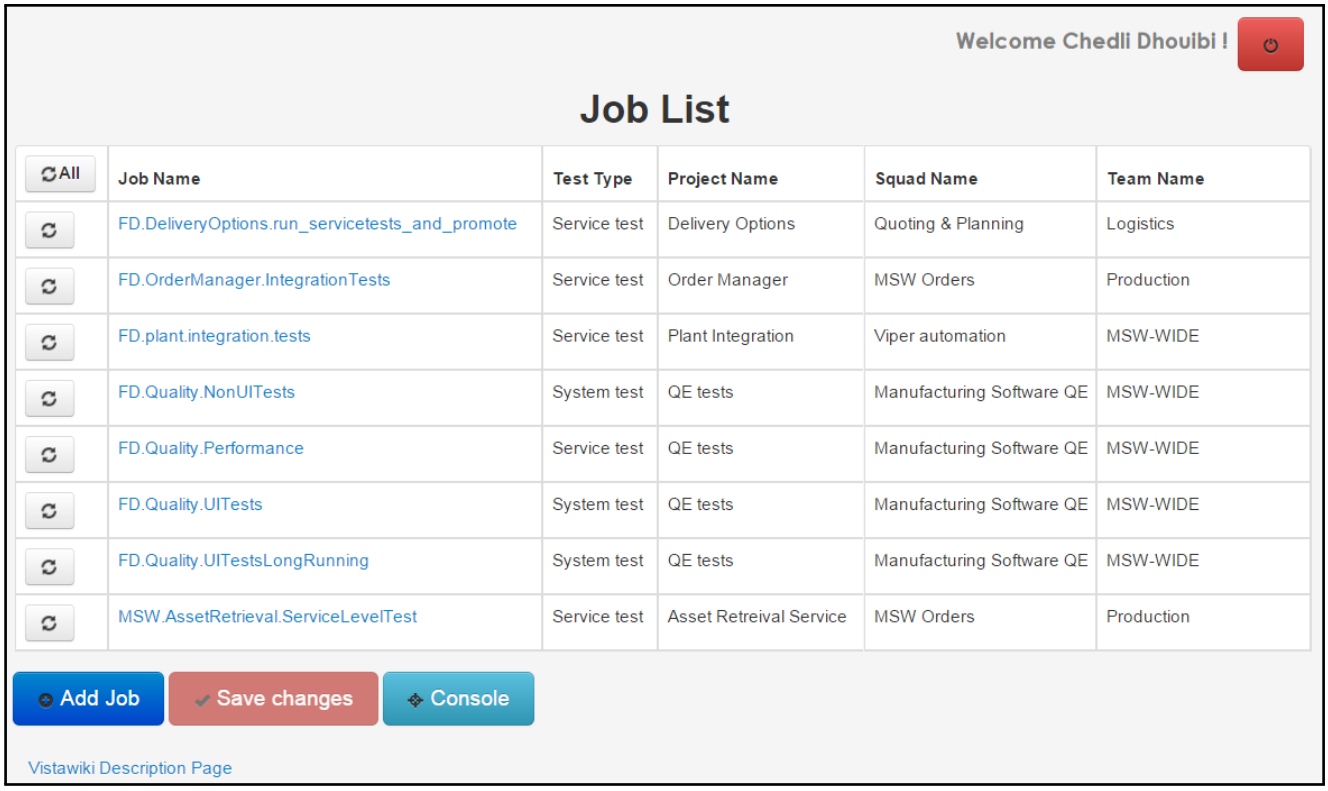


Figure V.22 – Main page

- **Main page buttons :**
  The main page mainly has three buttons. The "Add job" button redirects the user to the "Add new job" page. The "Console" button shows the log console in a modal state. The "Save changes" button saves any changes made to the job's list (e.g. test type update, team name update, etc). The link in the bottom redirects to the VistaWiki page (documentation) of the web application.



Figure V.23 – Main page buttons

- **Console :**
  The following figure presents the console which shows all of the treatment's result. It has a "clear" button that clears the console and a "close" button that closes the console. In this case it is showing the results of a global search through Jenkins for unsaved builds.

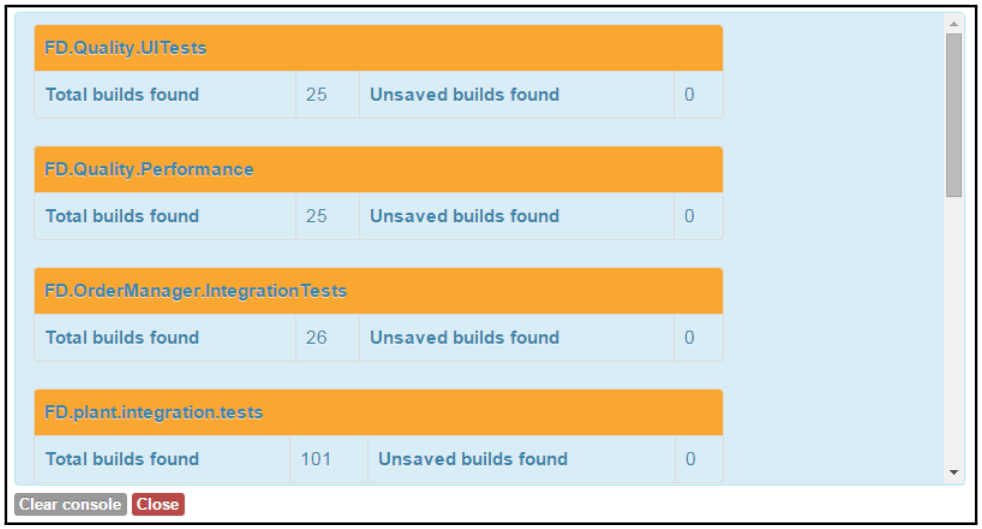


Figure V.24 – Console

- **Perform global search through Jenkins :**
  The "All" button in the top left triggers a global search through Jenkins for unsaved builds. The green flags indicate the jobs that have completed the search. The red flags indicate the jobs that are still being processed.

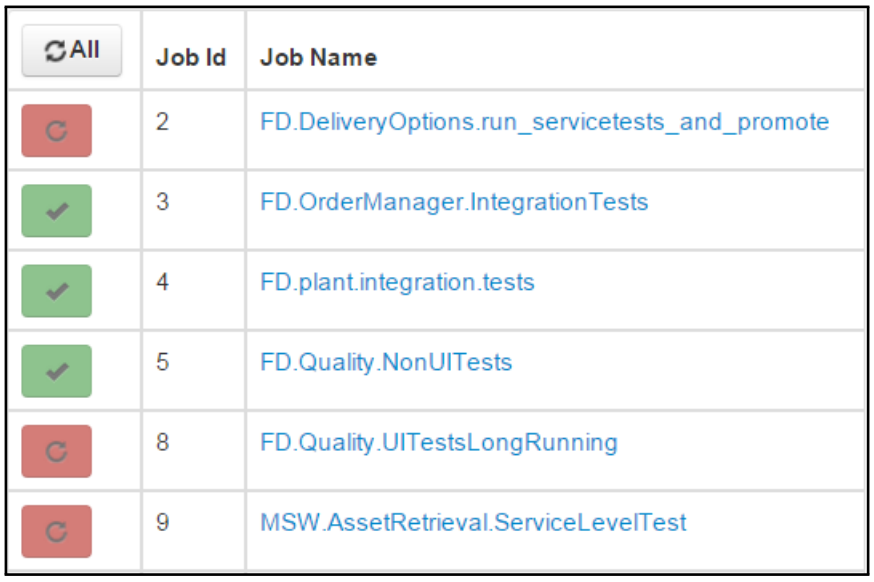


Figure V.25 – Performing global search

3.3.3 Add new job view

This view provides a form for adding a new Jenkins job to the system. The system administrator only provides the job's url, and the web application does the rest. In the following figure, the system administrator has provided the job's url and has clicked the "Get info" button, which now has a "Loading..." label because the information extraction is under process.

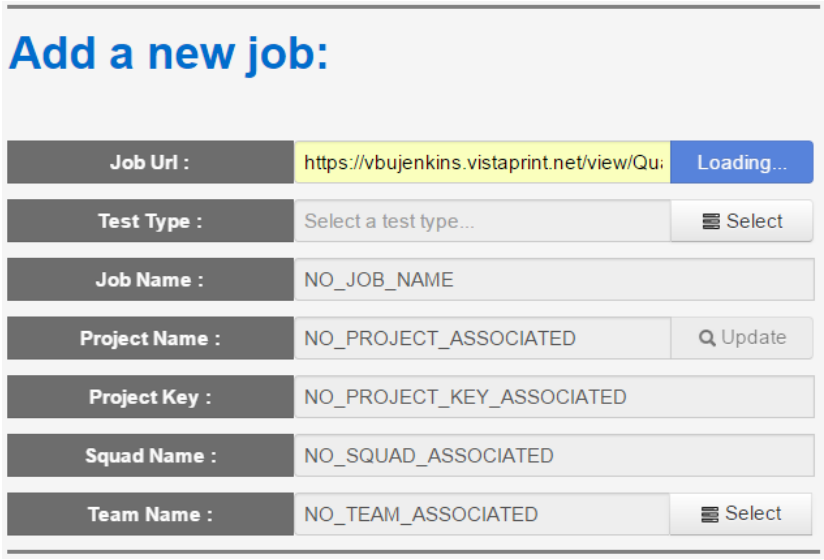


Figure V.26 – Fetching a job's information

In the following figure, the information extraction is complete ("Get info" button is back to its original state). The green labels indicate the successfully extracted fields.

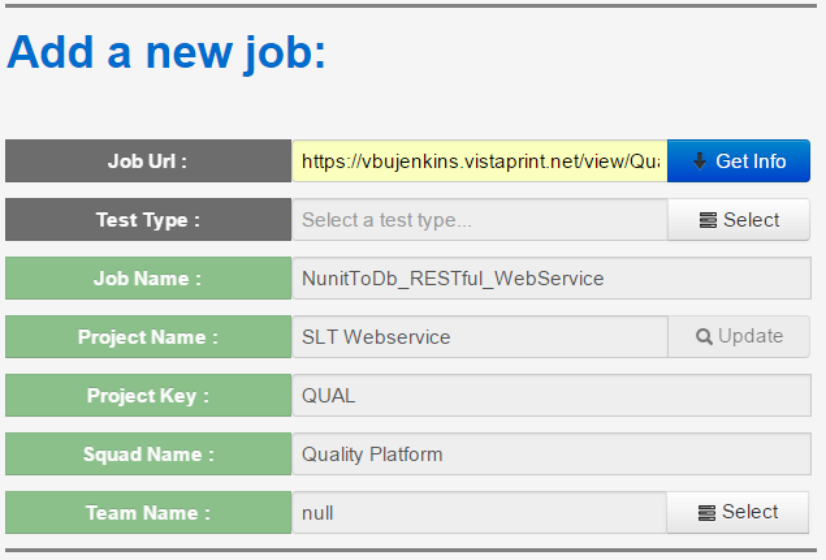


Figure V.27 – Information fetching results

4 Continuous integration

Continuous integration is a very important step in the software developing process. It allows the early detection of defects throughout the source code that is under development.
For this reason, we have used the Jenkins platform to ensure the continuous integration, along with Stash as a Git repository and Ant for the task scripts.Every build is triggered either manually, or after each change in the Stash repository.

4.1 Jenkins job

The Jenkins job that we have created automates the project's building, testing and deployment on the remote server.

– The following figure shows the Jenkins job that we created.

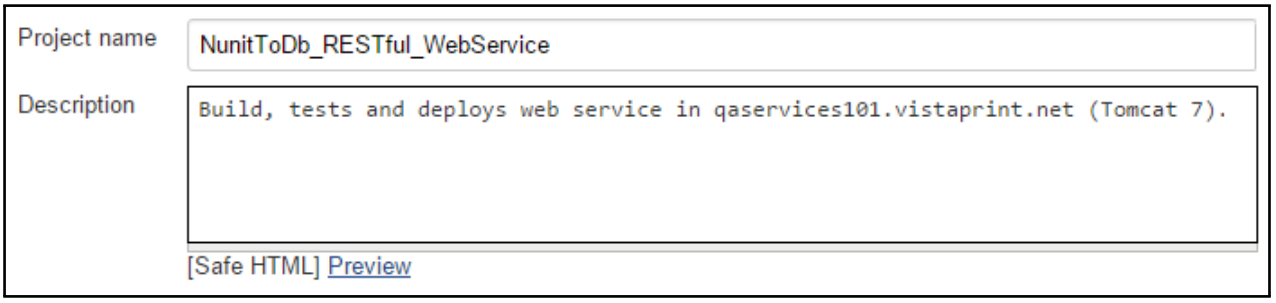


Figure V.28 – Jenkins job name

– The following figure shows how the Jenkins job is configured to import files from Stash before each build.

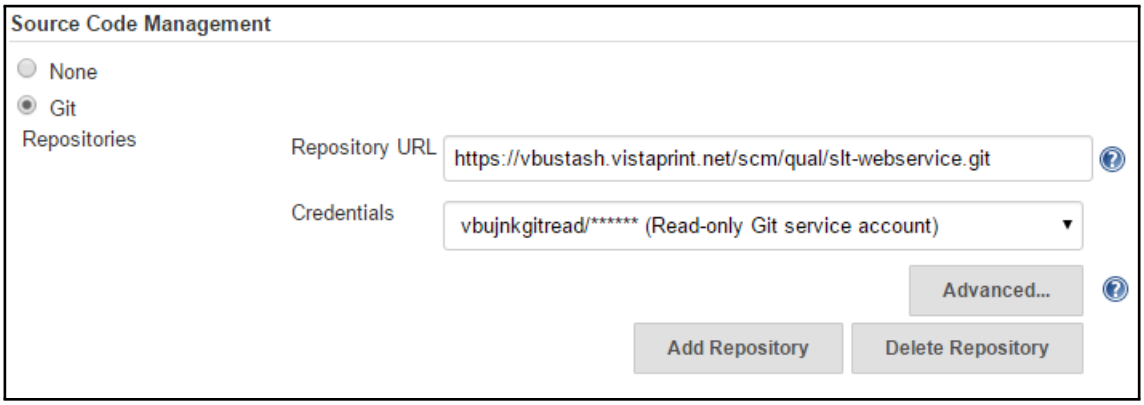


Figure V.29 – Jenkins job configuration to include Stash

– The following figure shows how the Jenkins job invokes Ant,so as to clean, compile, test, dist (package) and deploy the project.

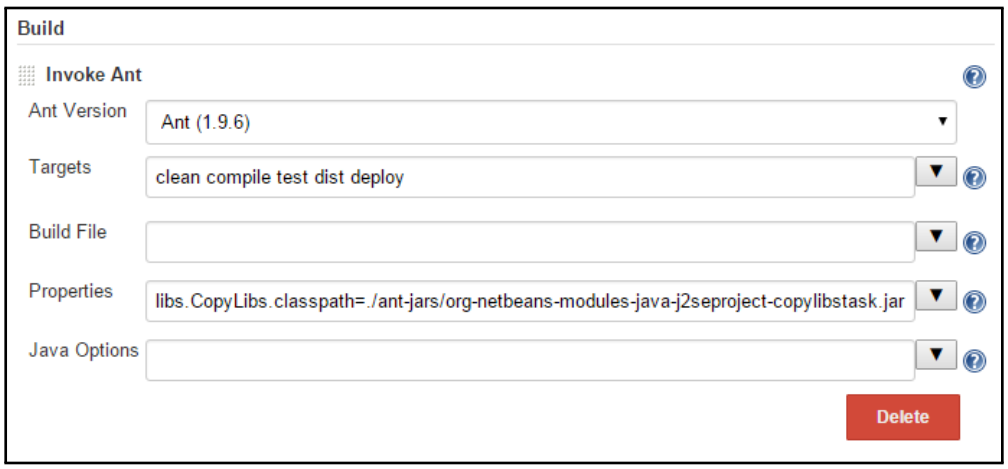


Figure V.30 – Invoking Ant script from Jenkins job

4.2 Stash repository

Stash was used as a Git repository and a version control system. It contains all the files that we have created throughout this project. The following figure shows the main interface of our Stash repository.

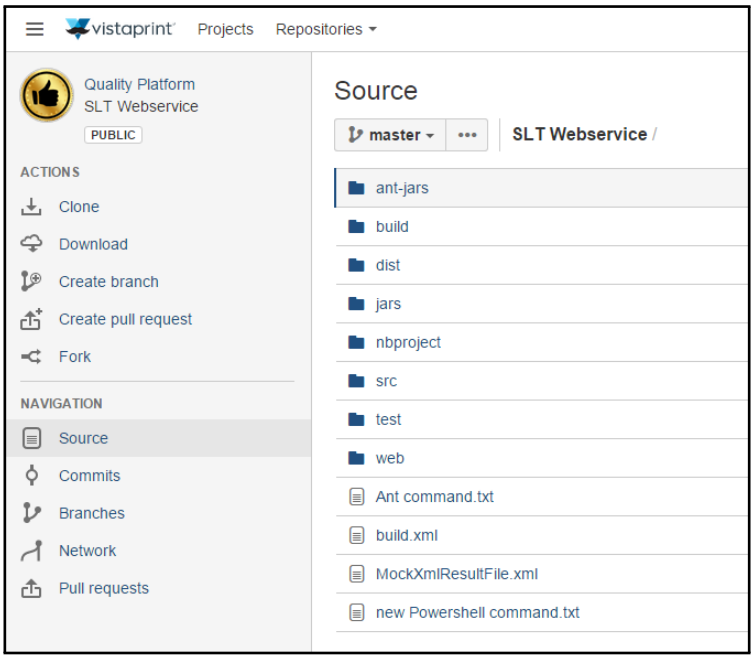


Figure V.31 – Stash repository

4.3 Ant script

As mentioned earlier, Ant was used for creating task scripts, such as "deploy" task, which is illustrated in the following figure (a portion of the "build.xml" file).

```
<path id="catalina-ant-classpath">
   <fileset dir="ant-jars">
      <include name="catalina-ant.jar"/>
   </fileset>
   <fileset dir="jars">
        <include name="tomcat-coyote.jar"/>
        <include name="apache-tomcat-util.jar"/>
        <include name="tomcat-juli.jar"/>
   </fileset>
</path>
<taskdef name="deploy"  classname="org.apache.catalina.ant.DeployTask" />
<taskdef name="undeploy"  classname="org.apache.catalina.ant.UndeployTask" />
<target name="deploy" description="deploy to tomcat">
    <mkdir dir="web\WEB-INF" />
    <echo>Application deployment started...</echo>
    <echo>Undeploying old war file...</echo>
  <undeploy url="http://qaservices101.vistaprint.net:8080/manager/text/" />
    <echo>Deploying new war file ... </echo>
    <deploy
            url="http://qaservices101.vistaprint.net:8080/manager/text"
            username="chedli"
            password="chedli"
            path="/NunitToDbRESTful"
            localwar="\\QABUILD101\dist\NunitToDbRESTful.war"
            />
    <echo>Application deployment done.</echo>
    </target>
```

Figure V.32 – Utils class model

4.4 Unit tests

Unit testing is a crucial step in the continuous integration. It ensures that all of the project's modules and function are working properly. For our case, JUnit was used as a unit testing tool. The Jenkins job executes these tests (hosted in Stash) through Ant scripts.

5 Project documentation

In Vistaprint, there is an internal documentation platform called "VistaWiki". We have created two linked wiki pages to describe the whole solution architecture with the newly implemented modules.Also, we described the internal structure and how to modify and build the source code,so as to facilitate future upgrades. The following figure illustrates a portion of the vistawiki page.

SLT reporting page (The global solution) :

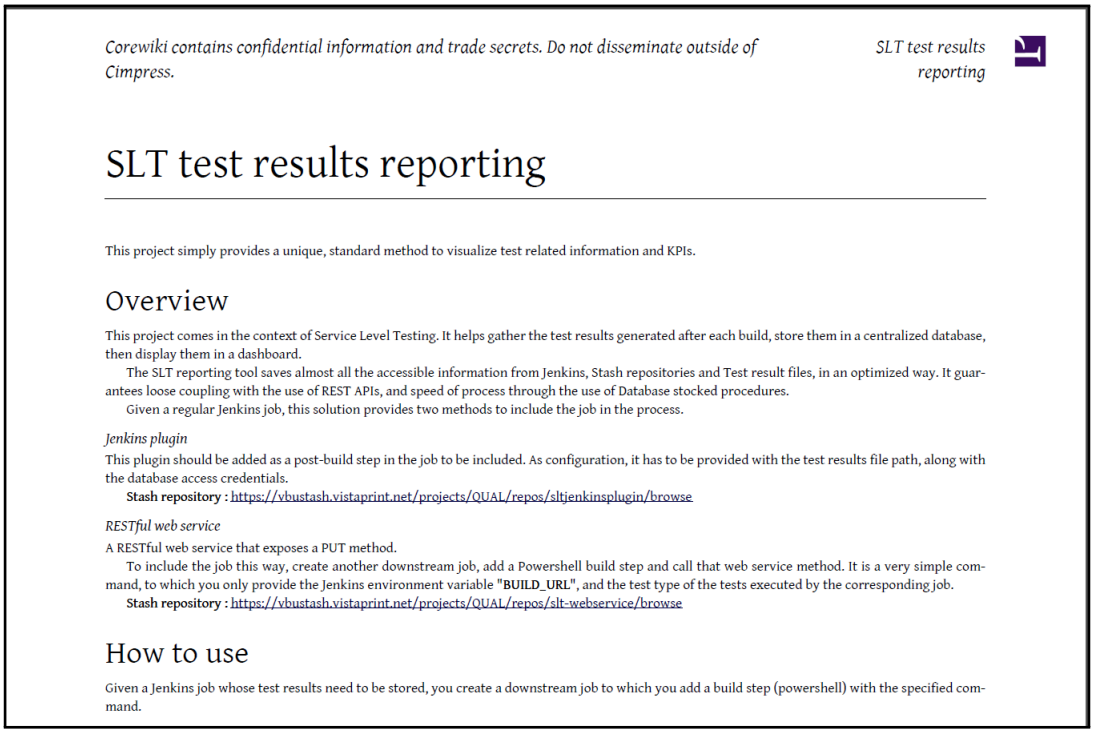


Figure V.33 – SLT reporting page

Conclusion

in this chapter we specified the software environment, which included the designing tools, the implementation tools, the continuous integration tools and the technologies and frameworks used during the project's development. Afterwards, we presented the technical architecture and the implementation achievements, which consists in a few screen-shots of the working solution. Then we continued with the project's own continuous integration and documentation. After this chapter we will proceed to the general conclusion.

General conclusion and perspectives

In this report we have presented the newly implemented reporting platform for service level testing. First we presented the project's background, going from the host company and the general context to the raised issues and the proposed solution. Afterwards we have developed the preliminary study, which included a thorough description of the technical context, a critical review of the existing solution and a detailed listing of the major technical choices for the proposed solution. At this point we had a global view over the project's general context and the specific problems yet to be solved.

After presenting the project's background and the preliminary study, we analyzed the various functional and non-functional requirements first by identifying the different actors and roles, then by establishing the use case diagrams and the system sequence diagrams. This has allowed us to better understand the requirements on which the solution's design was mainly based.

The solution's design had multiple aspects and levels. First we started by the global design, which included a global architecture, the global package model and the design patterns utilized in the solution. Then we moved to the domain design where we presented the domain models for each module, namely the Jenkins plugin module, the web service module, the file parsing module, the data persistence module and the web application module. Afterwards we presented the behavioral design which consisted in the different object sequence diagrams, and then the database design. Last but not least, we presented the achievements realized throughout the project's lifetime. First we went through the software environment, including the design and implementation tools, along with the different technologies and frameworks used during the development. After that we presented the technical architecture and demonstrated the implementation achievements by showing the screen-shots of the different working modules. We then concluded the chapter by presenting the continuous integration of the solution.

This reporting platform has helped many teams across Vistaprint to regain the global analytic view they once had. Feedback is now only few clicks away, and decisions are once again easier to make.

However, the solution is still a few steps away from being complete. In fact, it is currently being used only by Manufacturing Software (MSW) teams. It would be much more productive if its use was company-wide.
Moreover, the solution could be fully automated in a way that it gathers data without manual intervention. This would cost some more resources, but it would immensely increase productivity and ensure a better loose-coupling between the related external modules.

Bibliography

[1] VISTAWIKI. Vistaprint's general description. [03-March-2015].

[2] VISTAWIKI. Vistaprint's history. [03-March-2015].

[3] VISTAWIKI. Team organization. [04-March-2015].

[4] VISTAWIKI. Manufacturing software. [15-April-2015].

[5] Service oriented architecture (www.service-architecture.com). [18-April-2015].

[6] VISTAPRINT ACADEMY. *Introduction to Continuous Delivery.*

[7] Integrate often (www.extremeprogramming.org/rules/integrateoften.html). [20-April-2015].

[8] What is continuous integration (www.searchsoftwarequality.techtarget.com/definition/continuous-integration).

[9] VISTAWIKI. Test pyramid. [22-April-2015].

[10] MIKE COHN. *Test Automation Pyramid (succeeding with Agile).*

[11] MIKE COHN. *User Interface Testing (succeeding with Agile).*

[12] MIKE COHN. *Service Level Testing (succeeding with Agile).*

[13] MIKE COHN. *Unit Testing (succeeding with Agile).*

[14] MIKE COHN. *Agile methodologies (succeeding with Agile).*

[15] MIKE COHN. *Software development using scrum.*

[16] Meet jenkins (wiki.jenkins-ci.org). [20-March-2015].

[17] Remote access api (wiki.jenkins-ci.org). [20-March-2015].

[18] Extend jenkins (wiki.jenkins-ci.org). [20-March-2015].

[19] Git (www.git-scm.com). [21-March-2015].

[20] Version control definition (www.techterms.com). [21-March-2015].

[21] Stash developer documentation (www.developer.atlassian.com). [21-March-2015].

[22] Stash remote access (www.developer.atlassian.com). [21-March-2015].

[23] Nunit (www.nunit.org). [22-March-2015].

[24] VISTAWIKI. Test automation. [22-March-2015].

[25] Nunit plugin (www.jenkins-ci.org). [22-March-2015].

[26] Data reporting systems (www.gartner.com). [25-April-2015].

[27] Visual intelligence (www.visualintelligence.co.nz). [25-April-2015].

[28] What is qlikview (www.qlik.com). [26-April-2015].

[29] Magic quadrant for business intelligence and analytics platforms 2015 (www.gartner.com). [27-August-2015].

[30] Qlikview benefits (www.qlik.com). [26-April-2015].

[31] Qlikview (www.gartner.com). [27-August-2015].

[32] State of the art in web services (link.springer.com). [28-August-2015].

[33] VISTAWIKI. Restful web services. [10-June-2015].

[34] Http methods for restful services (www.restapitutorial.com). [10-June-2015].

[35] Xml definition (www.techterms.com). [12-June-2015].

[36] Xml parsing (www.xml.silmaril.ie). [12-June-2015].

[37] ZICARI R. RASHID A. CHAUDHRI, A. *XML Data Management*.

[38] Design patterns (www.sourcemaking.com). [07-July-2015].

[39] Enterprise architect (www.sparxsystems.com). [25-July-2015].

[40] Unified modeling language (www.uml.org). [25-July-2015].

[41] Working with netbeans (www.netbeans.org). [26-July-2015].

[42] Welcome to eclipse (www.eclipse.org). [31-July-2015].

[43] Apache maven (www.maven.org). [01-August-2015].

[44] Use sql server management studio (msdn.microsoft.com). [07-August-2015].

[45] Apache ant (www.ant.apache.org). [07-August-2015].

[46] Java ee 6 (docs.oracle.com). [19-August-2015].

[47] javax.ws.rs (docs.oracle.com). [19-August-2015].

[48] D. COCHRAN. Twitter bootstrap web development how to.

[49] Mouronval Lecomte. Jquery.

[50] Jeremy McPeak Nicholas C. Zakas and Joe Fawcett. Professional ajax.